

Final-Year Project Documentation

AI-Powered Gym Assistant – PoseFit AI

Real-Time Exercise Recognition and Logging

By

Artem Shkurat (SHK22612576)

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

for the degree of

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

Declaration

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Artem Shkurat

Date: 30.01.2026

Signed (apply signature below)

A handwritten signature in black ink, appearing to be 'AS' or similar, on a light blue background.

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout this remarkable and challenging journey. Every person who showed even the slightest moral support helped me make a massive contribution to this work.

I would especially like to say thank you to all the teachers, especially Lisa Haskel and my supervisor, Wei Li, who supported me throughout the project and was always there during difficult times.

I would also like to thank all my friends for their feedback. It played a key role in the development of this project. Special thanks to Ivan Ivanov for designing such a wonderful logo especially for me.

Thank you again to everyone for everything you've done for me.

Abstract

In this project, the development of an AI-powered fitness app for mobile devices, named PoseFit AI, is presented which use real-time machine learning and camera processing to recognize gym exercises. The overall objective of the project was to explore the possibility of integrating computer vision, pose estimation and developing a mobile application to form a viable system for exercise recognition and tracking.

The mobile application interface was created using the Flutter and Dart framework, while the application's backend was written in Python with a MySQL database to store user authentication, workout logs, exercise data, and machine learning feature data. To achieve this, MediaPipe pose estimation and TensorFlow were embedded in the system to enable real-time prediction of exercises from smartphone camera input, and to detect body landmarks and calculate body angle measurements.

To enhance usability and utility in a fitness context, additional features such as workout logging, tracking of Personal Records, camera permission management, and account management were added. The application successfully recognized the trained exercises in controlled environment with testing and evaluation, but some factors like camera positioning, lighting condition, limitation of database and movement variation influenced the recognition accuracy of the application.

Overall, the project was a success in showing the feasibility of using a combination of mobile development, machine learning, computer vision, and workout tracking within one AI-powered fitness application, as well as giving the students valuable hands-on experience in software engineering, backend development integration, database development, and real-time systems development.

Table of Contents

1. Introduction	9
Problem Statement and/or Research question	9
Aims	9
Objectives	10
2. Background and Context	11
Background	11
Literature review	11
Competitor Analysis/ Related Work	34
3. Tools and Methods	36
Rationale for the technical tools selected	36
Rationale for the research methods used	37
Project management and Product lifecycle	38
Risk register	43
4. Requirements/Specifications	46
Personas	46
User stories	49
User journeys	50
Use case diagrams	53
Wireframes	54
User flow diagrams	57
5. Solution design	60
System Architecture	61
Entity-Relationship Diagram (ERD)	63
Activity diagram - Camera-Based Exercise Recognition Workflow	65
6. Solution implementation	67
Initial Project Setup	67
Frontend Development	69
Backend Development	71
Database Implementation	74
Camera Integration and Permissions	77
Pose Estimation and Exercise Recognition	78
Workout Logging System	81
Authentication and Security	83
Challenges Encountered During Development	85
7. Solution testing/ evaluation	88

Functional Testing	88
Usability Testing.....	89
Exercise Recognition Evaluation	90
Performance and Reliability	91
Evaluation Against Objectives	95
Limitations.....	97
8. Discussion.....	99
Discussion of Results	99
Reflection on Development	100
9. Conclusion	102
Summary	102
Recommendations for further work.....	103
Reflection.....	104
10. References	106
11. Appendices	107

List of Figures

Figure 1: MediaPipe example image.....	12
Figure 2: MediaPipe real-life use example	12
Figure 3: MediaPipe body skeleton keypoints	Figure 4: MediaPipe settings..... 13
Figure 5: OpenPose main menu.....	14
Figure 6: OpenPose keypoints detection	14
Figure 7: OpenPose body keypoints examples	15
Figure 8: Tempo intro question	16
Figure 9: Tempo personal AI coach free trial	16
Figure 10: Tempo price for a month.....	16
Figure 11: Freeletics ratings	17
Figure 12: Freeletics intro question #2	18
Figure 13: Freeletics intro question #1	18
Figure 14: Freeletics intro question #3	18
Figure 15: StrengthLog most popular programs	Figure 16: one of the StrengthLog programs19
Figure 17: StrengthLog workouts	Figure 18: StrengthLog workout example 20
Figure 19: StrengthLog workout technique	Figure 20: StrengthLog workout instructions 21
Figure 21: Muscle & Strength muscle groups sorting #1	22
Figure 22: Muscle & Strength muscle groups sorting #2	23
Figure 23: Muscle & Strength all chest exercises	23
Figure 24: Muscle & Strength video exercise guide	24
Figure 25: Muscle & Strength exercise profile and target muscle group	Figure 26: Muscle & Strength exercise instructions 24
Figure 27: Strong workout menu	Figure 28: Strong workout logging..... 26
Figure 29: Strong finish workout screen	Figure 30: Strong after workout logging screen 27
Figure 31: Strong save workout?	Figure 32: Strong workout results..... 28
Figure 33: Hevy workouts main screen	Figure 34: Hevy exercise choice..... 29
Figure 35: Hevy workout logging screen #1	Figure 36: Hevy workout logging screen #2 30
Figure 37: Hevy muscle groups used	Figure 38: Hevy set types 31
Figure 39: Hevy save workout?	Figure 40: Hevy workout results screen 32
Figure 41: Kanban board.....	38
Figure 42: Gantt chart.....	39
Figure 43: Risk assessment.....	44
Figure 44: User Journey Diagram - Beginner Gym User.....	50
Figure 45: User Journey Diagram - Home Workout User	51
Figure 46: User Journey Diagram - Advanced Gym User	52
Figure 47: Use Case Diagram - PoseFit AI System.....	53
Figure 48: PoseFit AI Starting, Sign up, Log in screens Wireframe	54
Figure 49: PoseFit AI Home, Camera screens Wireframe	54
Figure 50: PoseFit AI Exercises screens Wireframe.....	55
Figure 51: PoseFit AI Workout logs, Add records screens Wireframe	55
Figure 52: PoseFit AI Account, Settings screens Wireframe	56
Figure 53: User Flow Diagram - Exercise Recognition.....	57
Figure 54: User Flow Diagram - Manual Exercise Search	58
Figure 55: User Flow Diagram - Workout Logging.....	59
Figure 56: System Architecture Diagram	61
Figure 57: Entity-Relationship Diagram (ERD)	64
Figure 58: Activity Diagram - Camera-Based Exercise Recognition Workflow	65
Figure 59: Flutter Application Navigation Structure	67

Figure 60: PoseFit_AI Kanban board Projects	68
Figure 61: Application Home screen Figure 62: Application Exercises screen	69
Figure 63: Application Camera screen Figure 64: Application Workout Logs screen	70
Figure 65: Application Account screen	71
Figure 66: Application navigarionsl bar.....	71
Figure 67: app.py terminal launching	73
Figure 68: part of the app.py code.....	73
Figure 69: PoseFit AI database	75
Figure 70: users database table and data	75
Figure 71: workout_logs database table and data.....	76
Figure 72: exercise_features_samples database table and data	76
Figure 73: Application Camera screen Figure 74: Camera Permission Interface	78
Figure 75: Real-Time Exercise Detection Camera Interface	80
Figure 76: Successful Exercise Recognition Using MediaPipe and TensorFlow.....	80
Figure 77: Exercises Info Page and Workout Log Confirmation After Exercise Recognition.....	80
Figure 78: SQL Query and Stored Exercise Feature Sample Dataset in MySQL Database.....	81
Figure 79: Workout Logs Interface	82
Figure 80: Add Workout Record Window	82
Figure 81: Personal Records Information Screen.....	82
Figure 82: Workout Logs Stored in the MySQL Database	83
Figure 83: Create Account Interface	84
Figure 84: User Login Interface	84
Figure 85: Forgot Password Window	84
Figure 86: User Account Data Stored in the MySQL Database.....	85
Figure 87: Flutter Runtime Assertion Failure During Testing.....	92
Figure 88: Workout Retrieval Loading Issue	93
Figure 89: Keyboard Overflow Error in the Help & Support Interface	94
Figure 90: PoseFit AI Application Logo.....	107
Figure 91: PoseFit AI Application Name Branding.....	107
Figure 92: Project Risk Assessment Matrix.....	108
Figure 93: GitHub Project Management Board	108

List of Tables

Table 1: Literature review application comparison table	33
Table 2: Risk register table	44
Table 3: Persona 1.....	46
Table 4: Persona 2.....	47
Table 5: Persona 3.....	48
Table 6: Functional Testing Results of the PoseFit AI Application	88
Table 7: Exercise Recognition Accuracy and Observation Results.....	91
Table 8: Evaluation of Project Objectives and Final Outcomes	96

1. Introduction

The development of artificial intelligence and the computer vision have made it possible to analyse human movement with the help of smartphones. When it comes to fitness, it becomes a problem to be able to recognize exercises properly, learn how to use the right technique, and monitor the progress of training. The goal of this project is to develop an AI powered mobile app that uses a phone's camera to recognize the exercise a person is performing. After identifying the exercise, the app can display useful information, such as the exercise name, target muscles and the correct technique. This app will also allow users to manually log sets, reps and weights, helping them keep track of their workouts. The aim is to make gym workouts easier, more accessible and more informative for anyone who wants to understand and track their progress.

Problem Statement and/or Research question

Many gym-goers, particularly beginners, struggle with identifying exercises, especially when learning new ones. Remembering exercise names, understanding which muscles are involved, and tracking sets, reps, and weights can also be challenging and not always practical during a workout. Most existing fitness apps don't automatically recognize exercises, meaning users must know their names before saving, which disrupts the workout process. Additionally, many apps lack clear instructions or explanations, forcing users to search for this information elsewhere. These issues can make workouts less effective, more time-consuming, and tedious for users.

Aims

The primary aim of the AI-Powered Gym Assistant (PoseFit AI) application is to make gym users and home exercisers more aware of **what they are doing, how to do it correctly**, and **what they are capable of** through a simple, accessible, and secure platform.

Objectives

- **Implement automatic exercise recognition using AI**

Develop a system that will use pose estimation and machine learning methods to automatically identify gym exercises from videos captured by a mobile device camera.

- **Improve user understanding of exercises and technique**

After recognising an exercise, provide clear and accessible information about it, including the exercise name, target muscle groups, and instructions and tips on proper technique. This will help users better understand what they are doing and how to do it correctly.

- **Enable simple and efficient exercise logging**

Allow users to manually record sets, reps, weights, and their personal records after recognising an exercise. This will help them remember their capabilities the next time they perform that exercise.

- **Ensure data privacy and security**

Develop the app with privacy in mind and adherence to GDPR regulations. Handle camera data responsibly, requesting explicit user consent and avoiding unnecessary storage or transmission of video recordings. This will help app users feel confident that nothing happens without their permission.

- **Develop a simple and practical app**

Create an app that is easy to navigate and enjoyable to use on both iOS and Android. This will ensure users have a flexible experience.

- **Evaluate system performance and usability**

Evaluate the accuracy, usability, and practicality of the app through testing and evaluation, ensuring that the app meets the stated goals and user needs before its full launch.

2. Background and Context

Background

The recent achievements in artificial intelligence and computer vision have given mobile devices much more ability to analyse human movement. Detecting and tracking key body joints in images or videos for pose estimation is finding increasing application not only in science, computer vision and security but also in sports analysis and fitness. Many different cutting-edge frameworks have shown that human pose tracking is possible using standard cameras without any specialized equipment.

Correct execution of exercises in fitness has a great influence on improving results, as well as preventing injuries. However, most gym-goers don't have easy access to professional guidance during their workouts. Therefore, users tend to use mobile applications, online videos, or written instructions to learn exercises and track their progress. Although these tools offer some useful materials, they usually demand manual exercises selection and require users to be already familiar with the name and methods of the exercises.

Current fitness applications tend to concentrate on a single part of the work out experience like instructional material, workout planning or progress tracking. More developed AI-based platforms might provide feedback in real-time but might also often require specialized equipment, subscriptions, or processing via a cloud. Such restrictions may decrease the availability and create an issue of cost, privacy, and data security.

The widespread use of smartphones enables a lightweight, privacy-conscious mobile solution that combines exercise recognition, guidance, and workout logging.

Literature review

To understand the current state of AI-powered pose and movement recognition for the PoseFit AI app, a study was conducted of existing mobile apps and websites that provide one or more of these core features:

- 1. Pose estimation techniques**
- 2. AI-guided fitness platforms**
- 3. Exercise info platforms**
- 4. Weight, reps, sets logging**

The review examines the strengths and weaknesses of each application and compares them to determine the most useful features that may be useful in future application development.

Feature 1: Pose estimation techniques

MediaPipe

MediaPipe[1], [2], [3] is a framework developed by Google developers that can estimate human poses in real time. It can detect body landmarks (such as shoulders, elbows, hips, and knees) on multiple human bodies at the same time.

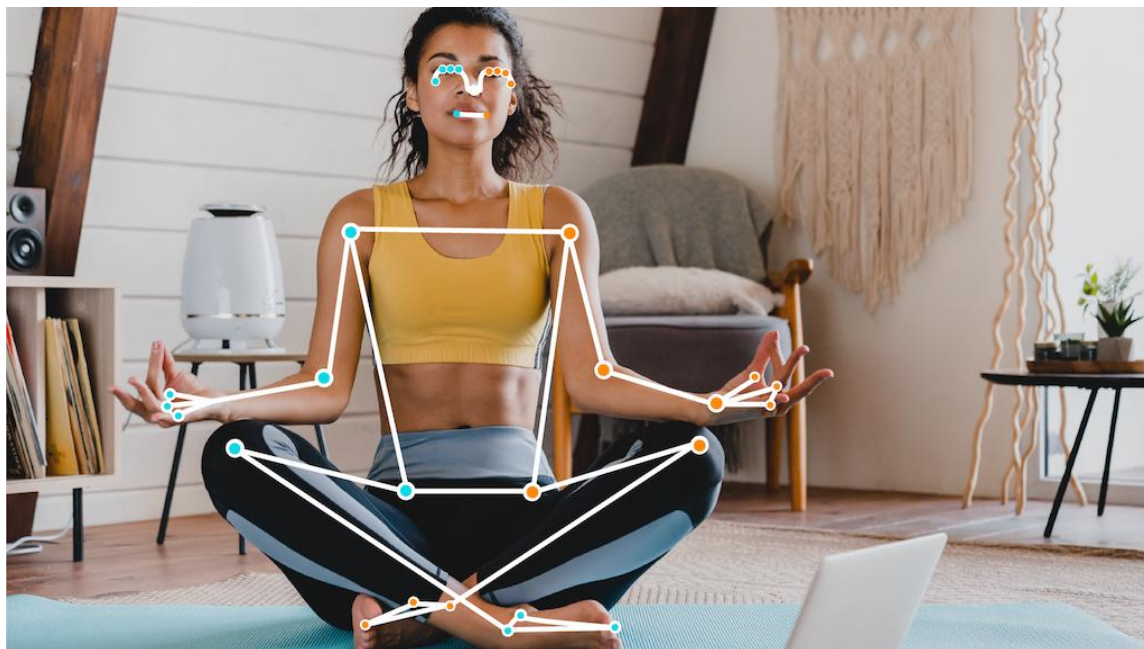


Figure 1: MediaPipe example image

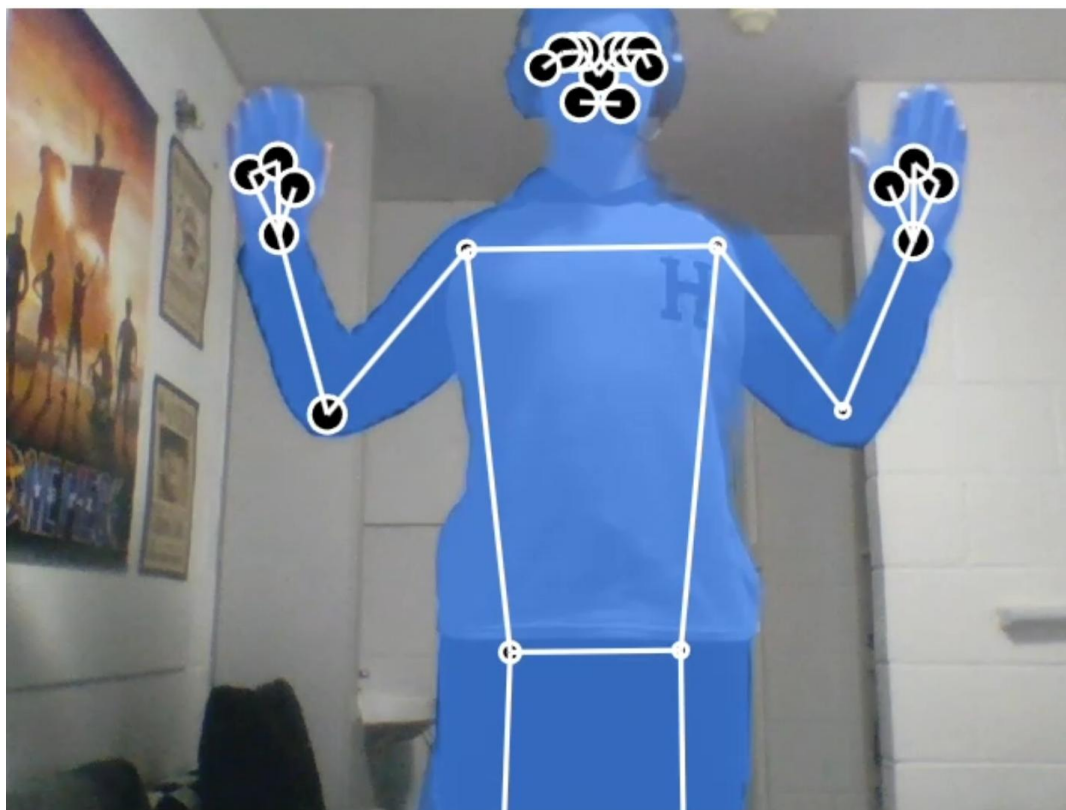


Figure 2: MediaPipe real-life use example

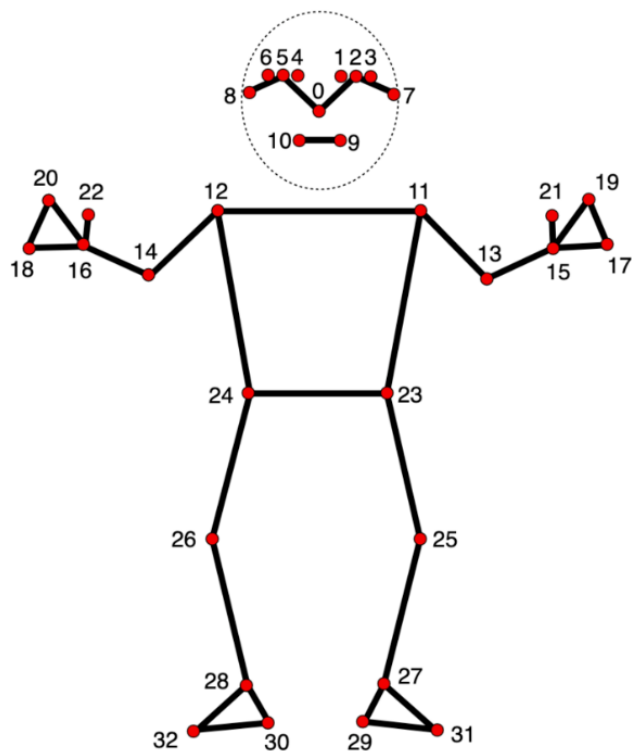


Figure 3: MediaPipe body skeleton keypoints

The sample parameters below can be changed. See [documentation](#) for more details

Inference delegate:

Model selections:

Segmentation mask:

Demo num poses:

Minimum pose detection confidence:

Minimum pose presence confidence:

Minimum tracking confidence:

Figure 4: MediaPipe settings

Summary:

MediaPipe uses a trained model that maps your body parts into landmarks in real time. This feature can be used on both a phone and a computer. Landmark detection accuracy is quite high, and the model keeps up with even the most rapid movements of your limbs. However, accuracy drops significantly when it can't accurately detect the ends of your body (for example, when turning sideways). When some parts of your body are out of camera view, the model estimates their approximate location. Lag is almost imperceptible, and this technology doesn't require significant hardware. The model doesn't analyse mouth movements or finger landmarks, only your fist. Additional settings are available, but they're not very extensive.

OpenPose

OpenPose[4], [5] is a pose estimation framework developed by researchers at Carnegie Mellon University (CMU) that can detect and track the human body using photos or videos and accurately determine its pose in 3D space. It is well known for being the first real-time multi-person pose estimation system to accurately detect human body, hand, facial, and foot key points (a total of 135 key points) on single images.

Let's create your project.

Artem Shkurat > Public

Project Name

E.g., 'Human Pose' or 'Car Par

Annotation Group

E.g., 'humans' or 'cars' or 'ani

Visibility

Private

Public

Licenses

CC BY 4.0

Name cannot be empty.

Project Type

Object Detection

Bounding Boxes # Counts Tracking

Identify objects and their positions with bounding boxes.

Classification

Image Labels Filtering Content Moderation

Assign labels to the entire image.

Single-Label Multi-Label

Instance Segmentation

Polygons Measuring Odd Shapes

Detect multiple objects and their actual shape.

Keypoint Detection

Skeleton Structure Pose Estimation

Identify keypoints ("skeletons") on subjects.

Multimodal

Prompts Visual Question Answering Captions

Describe images using text pairs.

Show More



Figure 5: OpenPose main menu

← ARTEM SHKURAT

Human pose

Keypoint Detection

DATA

↑ Upload Data

Annotate

Dataset 0

Versions Train

Analytics

Classes & Tags

MODELS

Models

Visualize

DEPLOY

Deployments

Annotate

Sort By: Newest

Unassigned

0 Batches

Upload More Images

Annotating

1 Job

Video: video_2026-01-09_13-55-29.mp4

Labeler: Artem Shkurat

33 Images

1 Annotated 32 Unannotated

Start Annotating →

Figure 6: OpenPose keypoints detection



Figure 7: OpenPose body keypoints examples

Summary:

During the utilisation of this tool, it was found that the site and its functionality were not easy to navigate. To work with OpenPose, it requires users to shoot a short video and upload it to their website. These steps include coming up with a project, the project name, an annotation group, and the activities that the uploaded video intends to perform (e.g., Keypoint Detection). Once these steps are completed, the user gets redirected to the main interface in order to accomplish this functionality. This is where the user may encounter several usability issues. The automatic pose keypoint extraction was hard to find and engage in its extraction, and no obvious workflow enabling this option was found. Consequently, it was concluded that keypoint annotation should be done by hand. Also, the absence of real-time pose estimation capability greatly restricts the use of the tool in mobile exercise recognition. All in all, it can be concluded that OpenPose is not an appropriate choice for this project since it features a complicated workflow, as well as lacks the possibilities of real-time processing.

Comparison MediaPipe vs OpenPose:

Although usability concerns were found regarding OpenPose, the general functions of both systems were compared. MediaPipe can be used on both a PC and a phone, while OpenPose is PC-only. MediaPipe also supports real-time processing, which is a huge advantage and well-suited for my project. OpenPose, on the other hand, lacks this capability and only uses photos and videos. Although MediaPipe has high accuracy, statistically, OpenPose's accuracy is even higher; however, the former has significantly lower hardware requirements than the latter. Thus, based on these considerations, it was determined that MediaPipe is the most suitable platform for implementing the proposed system.

Feature 2: AI-guided fitness platforms

Tempo

Tempo[6] is an AI-powered fitness platform that uses computer vision to recognise exercises and guide users through different types of training workouts. It has a built-in AI training assistant that helps users with workout plans, guides them throughout the session, gives tips on improving their technique, and so much more.

What is your fitness goal right now?

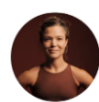
Losing weight

Getting toned and reducing body fat

Getting stronger

Figure 8: Tempo intro question

TEMPO



Hi Artem, Coach Mel here!

Based on your answers, here is your **strength gain Plan.**

We'll adjust as we go to make sure you stay on track to get results by April 9.

Your Goal	Session Length
Strength gain	35-45 mins
Focus Area	Training Frequency
Full body	3-4 days/week



Figure 9: Tempo personal AI coach free trial

Virtual Personal Training Starting at \$39/month

Figure 10: Tempo price for a month

Summary:

Since the application is paid and most features are unavailable until a virtual trainer is purchased, it was not possible to fully explore its functionality. Until the paid version is obtained, the application offers few advantages beyond providing an estimate of potential muscle mass gain over a given period. In addition, the subscription is relatively expensive and requires a monthly payment. Another drawback is that the application is primarily designed for home use, and some assistant features require specialised equipment, which must be purchased through the provider’s website. If the paid version is purchased, the application offers a wide range of features, including a virtual assistant that provides guidance and support throughout workouts. Overall, the application follows a similar concept to the proposed project.

Freeletics

Freeletics[7] is a fitness app that uses artificial intelligence to create personalised workout plans based on the user’s goals, fitness level, and performance history. Unlike Tempo, it doesn't use a camera to evaluate posture; instead, it collects user data (completed workouts, performance metrics, and feedback) and uses AI-powered algorithms to tailor recommendations for future workouts.

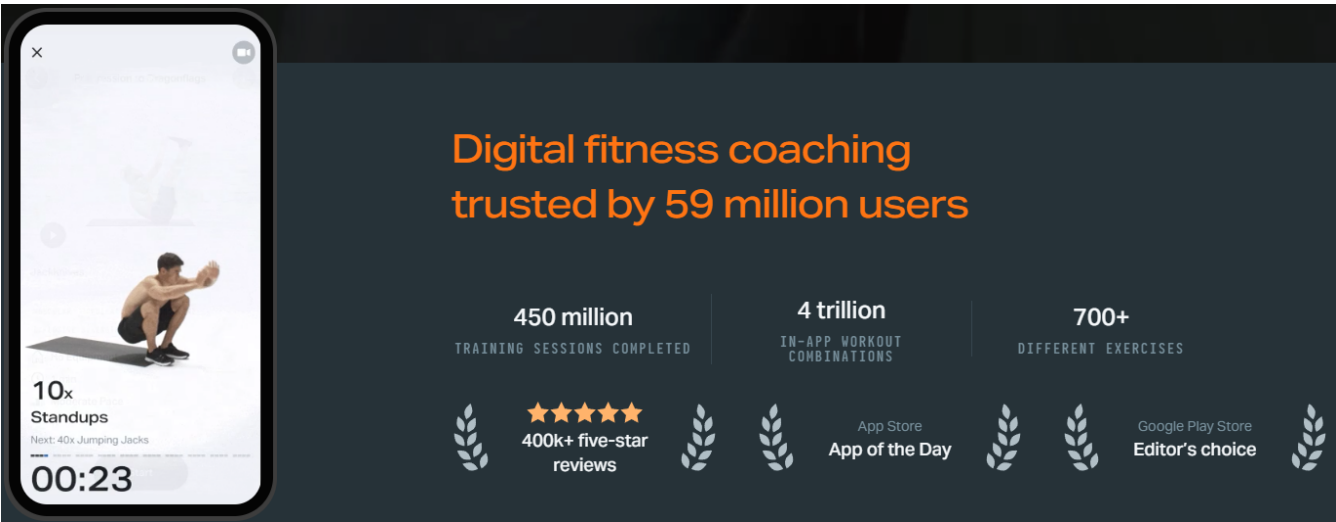


Figure 11: Freeletics ratings

How would you like to train?

Select in your order of preference.

- Bodyweight**
No equipment needed 1x
- Dumbbell(s)**
Any weight that works for you 2x
- Running**
No equipment needed 3x
- Barbell**
+ Bench, rack, & wide range of plates 4x
- Kettlebell**
Any weight that works for you 5x

Next →

Figure 13: Freeletics intro question #1

Time to push your limits

What is your height?

CM **FT**

Figure 12: Freeletics intro question #2

You won't sweat it alone!

What is your current weight?

KG **LB**

Figure 14: Freeletics intro question #3

Summary:

This is a mobile app that requires a fee to use. While the price is lower, this comes at the cost of fewer features. The app doesn't use the camera or AI for posture assessment. It's more focused on building a training program using AI. While it doesn't have a personal assistant, the training plan can be easily modified. Compared to Tempo, this app relies too much on manual control and lacks the AI influence.

Comparison Tempo vs Freeletics:

Although these applications were not fully explored in practice, their overall features were compared. These apps are targeted at different audiences: one is designed for those who need real-time guidance, the other for those who need a training plan. Both apps do their job for a reasonable price. Tempo has a built-in AI assistant that uses the phone's camera, while Freeletics doesn't. On the other hand, Freeletics can be used both at home and at the gym, while Tempo is almost exclusively used at home (due to the necessary equipment). Overall, Tempo's approach aligns more closely with the concept of the proposed project, although certain design changes would be required.

Feature 3: Exercise info platforms

StrengthLog

StrengthLog[8] is a mobile fitness application with multi-functional content about program following, goals reaching and workout planning. It focused on strength training tracking, and your gym progress. It provides a structured exercise database alongside tools for logging workouts and monitoring progress over time.

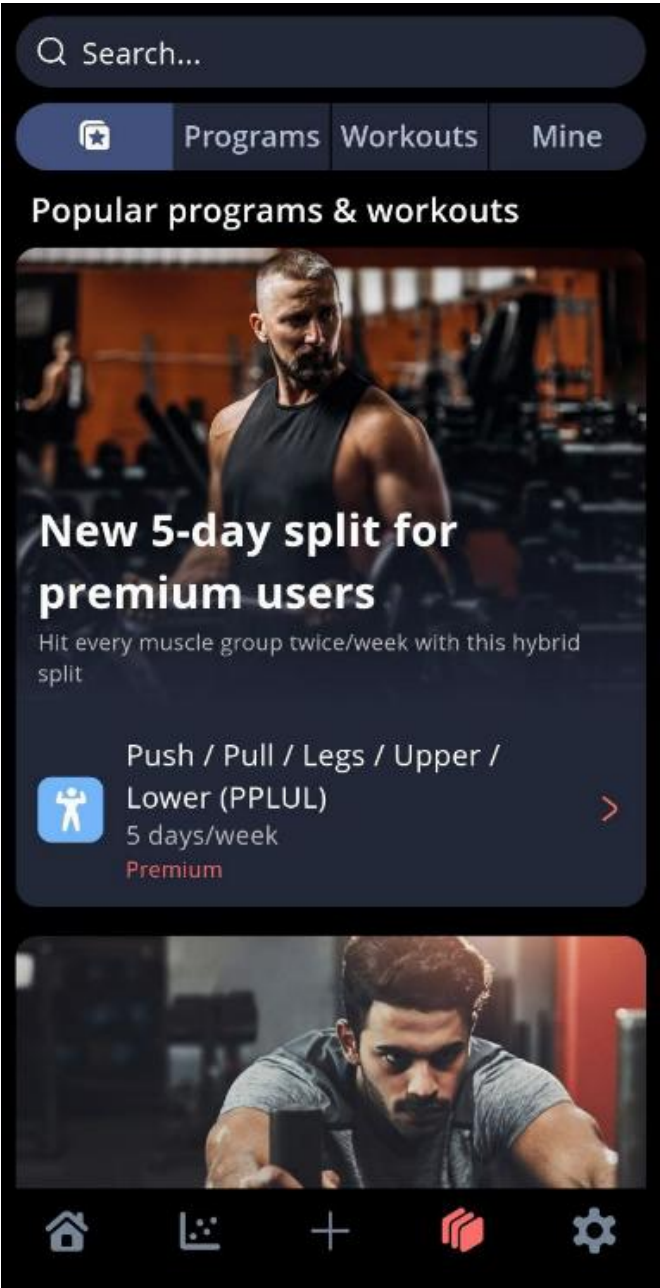


Figure 15: StrengthLog most popular programs

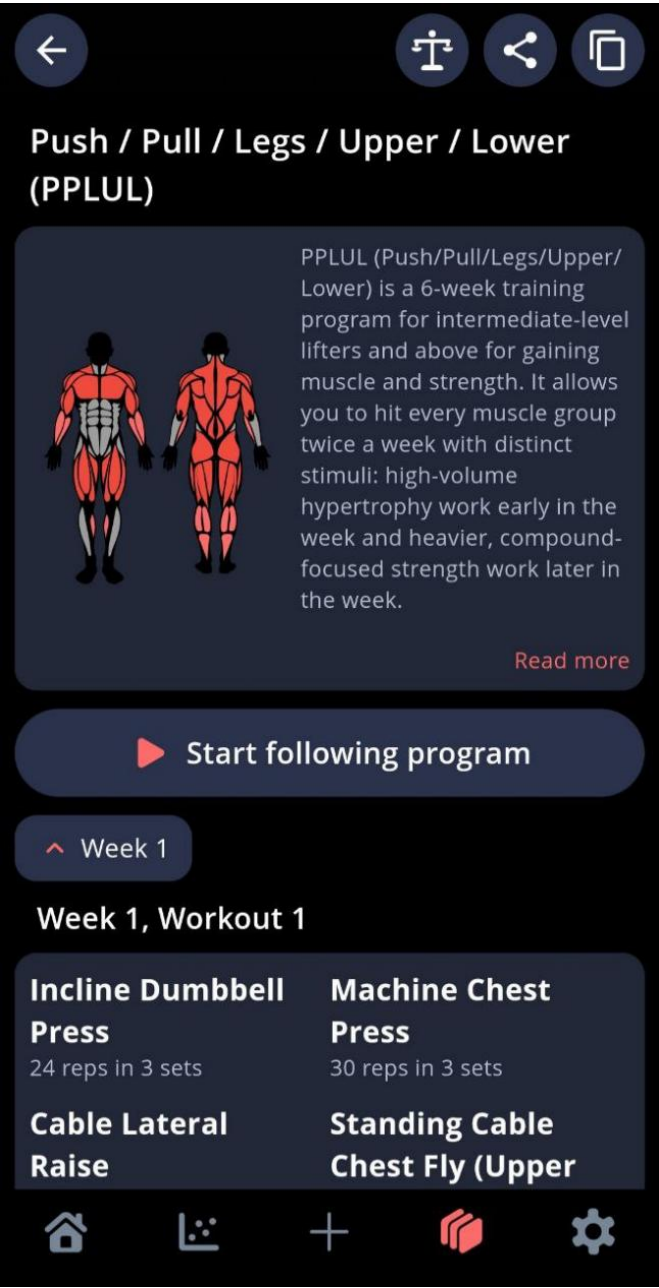


Figure 16: one of the StrengthLog programs

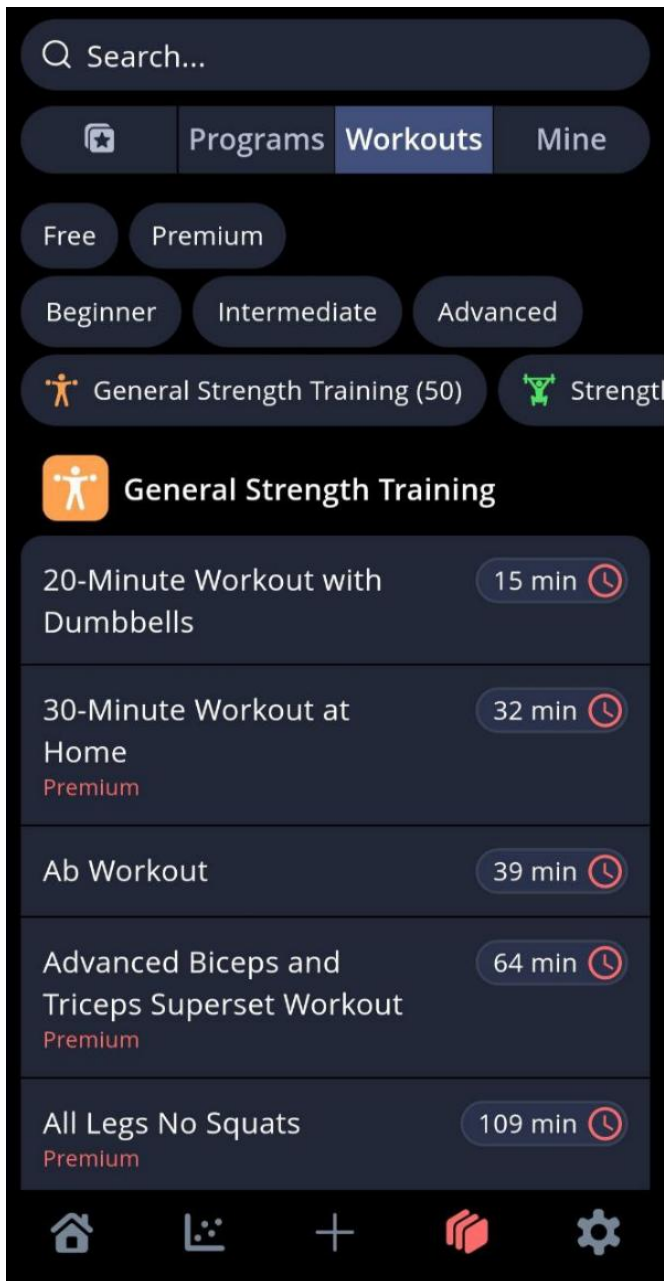


Figure 17: StrengthLog workouts

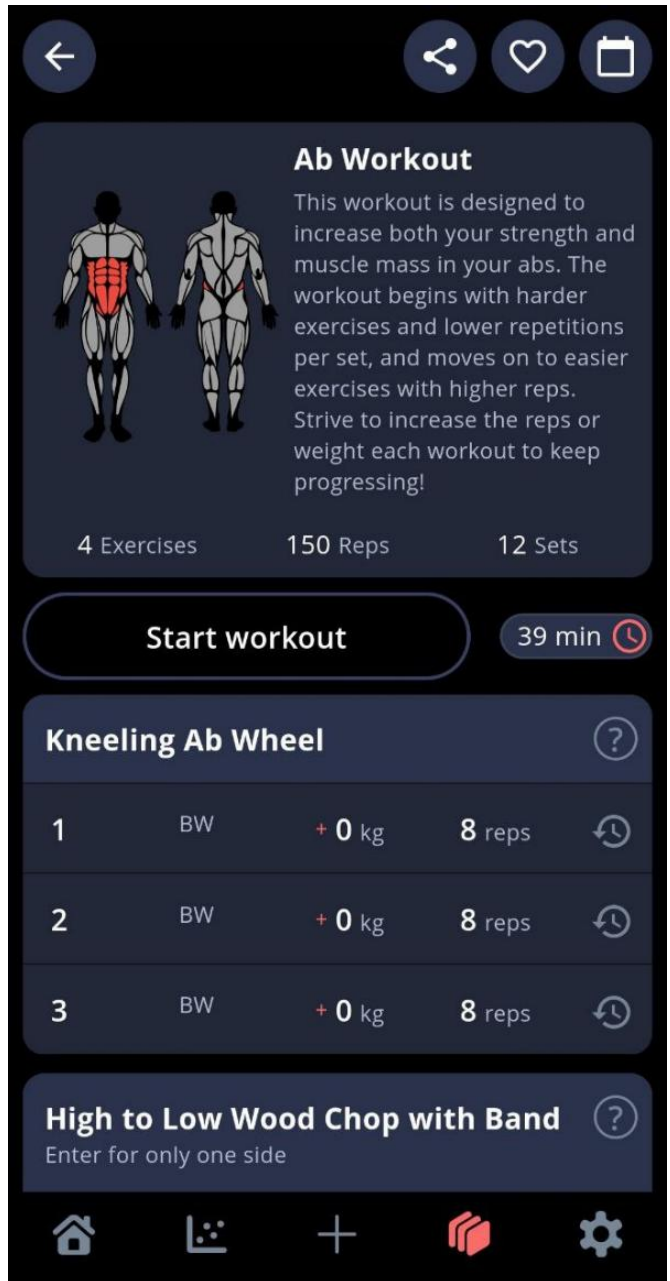


Figure 18: StrengthLog workout example

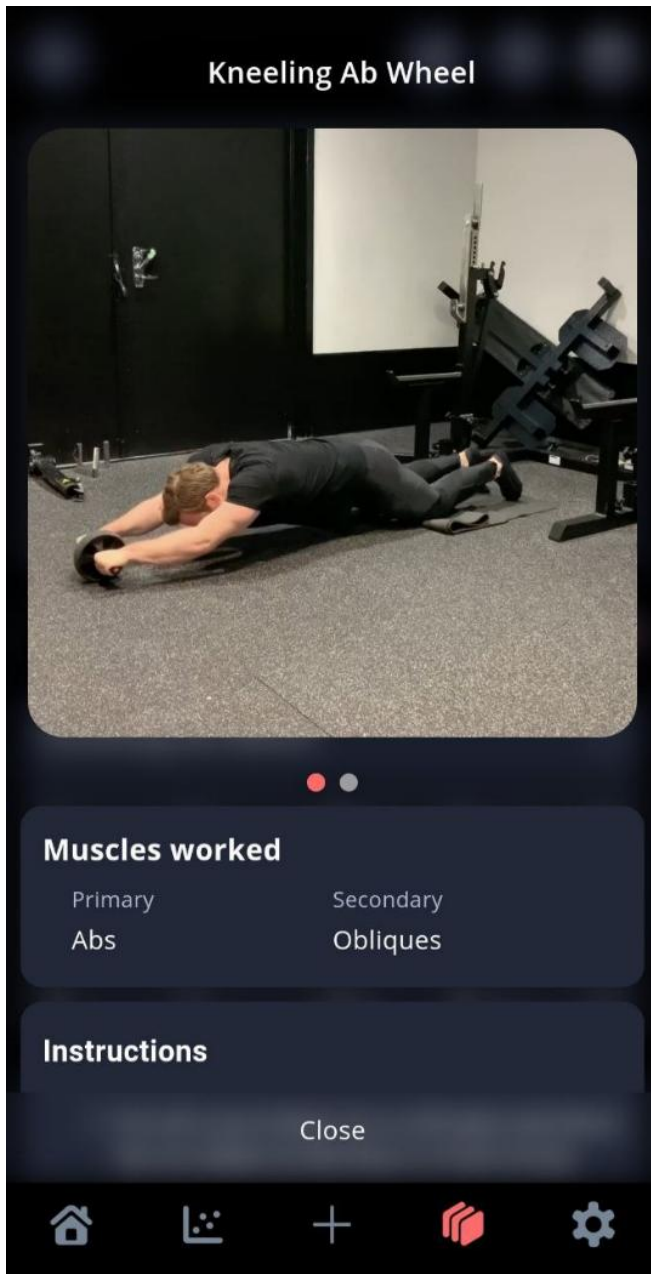


Figure 19: StrengthLog workout technique

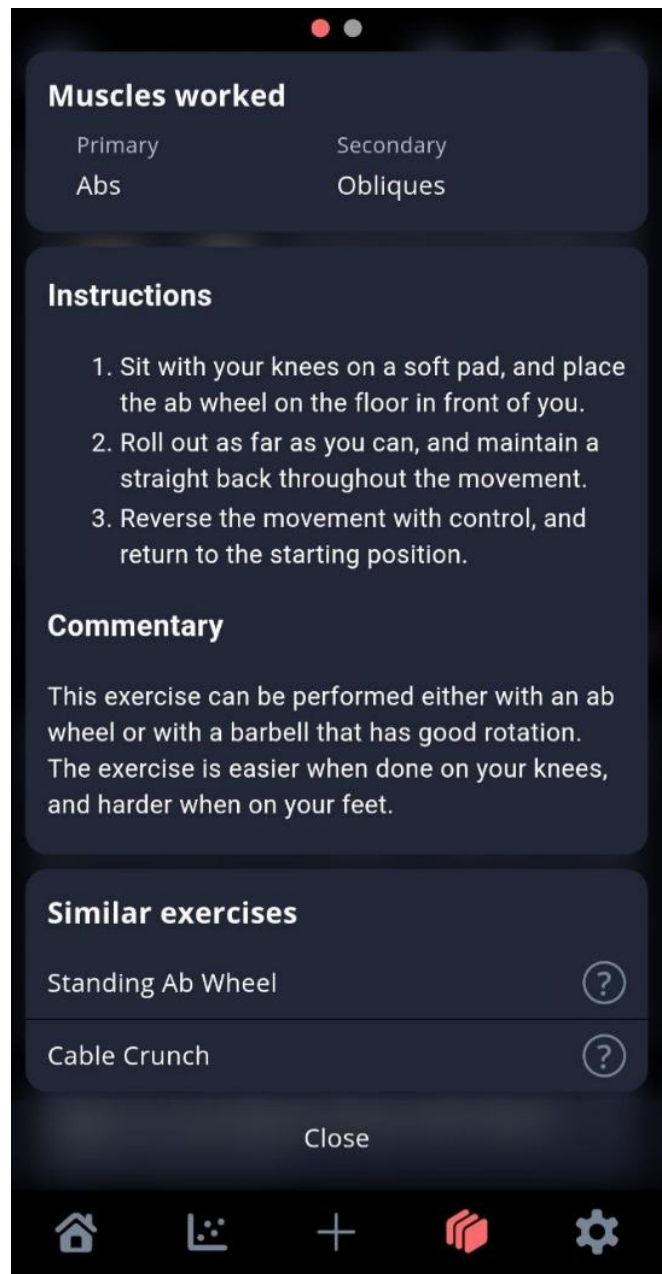


Figure 20: StrengthLog workout instructions

Summary:

When opening the mobile application, users are expected to register. Upon registration, the user is forced to make some data permissions to enable the application to operate properly. Popular programmes and workouts are clearly presented in a section and the user interface is usually easy to use and understand. In choosing a workout, the application will explain what the workout entails and its duration. Certain exercises are a part of premium membership, with some of the content being free. Once a workout is opened, the application shows the muscle groups to be used, the exercises to be used and the number of sets and repetitions needed.

Every exercise incorporates a help icon, which allows a demonstration video and the written instructions on how to do it properly. Altogether, the app has a good user interface as it includes not only guided workouts and educational material but also other support functions.

Muscle & Strength

Muscle & Strength[9], [10] is an educational website dedicated to fitness, offering exercise guides, workout programs, and training tips. The platform provides written guides, images, videos, and downloadable workout plans. Exercises can be categorised by muscle group, equipment used, and workout goals.

EXCERCISES BY MUSCLE GROUP

Choose the muscle group you want to target. Once in the muscle group, use the sort and filter options to find the best exercises for the equipment you have, your training experience, and goals.

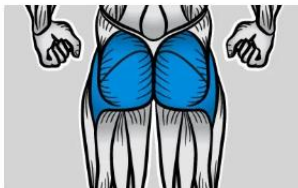
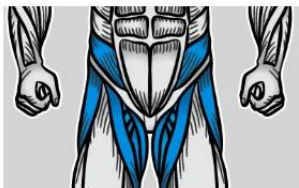
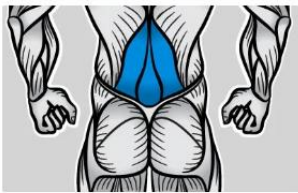
 Abductors	 Abs	 Adductors	 Biceps
 Calves	 Chest	 Forearms	 Glutes
 Hamstrings	 Hip Flexors	 IT Band	 Lats

Figure 21: Muscle & Strength muscle groups sorting #1



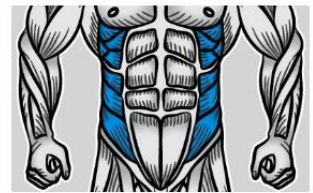
Lower Back



Upper Back



Neck



Obliques



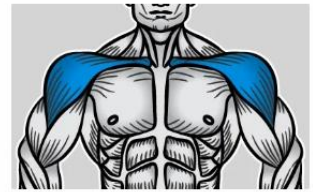
Palmar Fascia



Plantar Fascia



Quads



Shoulders



Traps



Triceps

Figure 22: Muscle & Strength muscle groups sorting #2

ALL CHEST EXERCISES

TRENDING

WHAT'S NEW

DISCUSSED

A-Z NAME

TYPE
STRENGTH

EQUIPMENT
DUMBBELL

MECHANICS
COMPOUND

EXP. LEVEL
BEGINNER

Dumbbell Bench Press

7.1M Views 108 Comments

TYPE
STRENGTH

EQUIPMENT
DUMBBELL

MECHANICS
COMPOUND

EXP. LEVEL
BEGINNER

Incline Dumbbell Bench Press

6.1M Views 64 Comments

Figure 23: Muscle & Strength all chest exercises

Dumbbell Bench Press Video Exercise Guide

Categories: Exercise Videos Chest
7.1M Views



Figure 24: Muscle & Strength video exercise guide

EXERCISE PROFILE	
Target Muscle Group	Chest
Exercise Type	Strength
Equipment Required	Dumbbell
Mechanics	Compound
Force Type	Push (Bilateral)
Experience Level	Beginner
Secondary Muscles	Shoulders, Triceps

TARGET MUSCLE GROUP
Chest

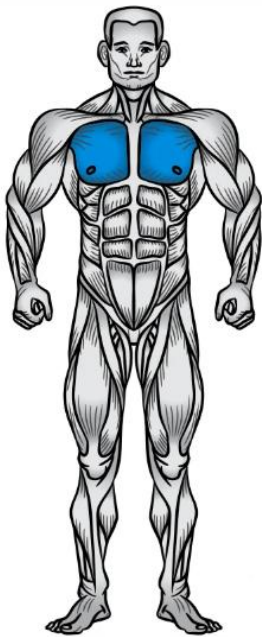


Figure 25: Muscle & Strength exercise profile and target muscle group

Dumbbell Bench Press Overview

The dumbbell bench press is a variation of the [barbell bench press](#) and an exercise used to build the muscles of the chest.

Often times, the dumbbell bench press is recommended after reaching a certain point of strength on the barbell bench press to avoid pec and shoulder injuries.

Additionally, the dumbbell bench press provides an ego check in the amount of weight used due to the need to maintain shoulder stability throughout the exercise.

The exercise itself can be featured as a main lift in your workouts or an accessory lift to the bench press depending on your goals.

Dumbbell Bench Press Instructions

1. Pick up the dumbbells off the floor using a neutral grip (palms facing in). Position the ends of the dumbbells in your hip crease, and sit down on the bench.
2. To get into position, lay back and keep the weights close to your chest. Once you are in position, take a deep breath, and press the dumbbells to lockout at the top.
3. Slowly lower the dumbbells under control as far as comfortably possible (the handles should be about level with your chest).
4. Contract the chest and push the dumbbells back up to the starting position.
5. Repeat for the desired number of repetitions.

**Dropping the dumbbells to the side is discouraged unless you are experienced with the technique or using excessively heavy weights.

**Ideally you should twist the dumbbells back to neutral (palms facing each other), bring your knees up so the ends of the dumbbells are touching your thighs, then use the weight of the dumbbells to rock back to an upright, seated position.

Dumbbell Bench Press Tips

1. Maintain more tension through the pecs by not locking out the elbows entirely.
2. Keep the weights slightly tilted at a 45 degree angle in order to keep the elbows in a neutral position.
3. Don't allow the dumbbells to collide at the top of each rep - bouncing them together may cause you to lose stability within the shoulder and injure yourself.
4. Squeeze the dumbbells as tight as possible to improve a phenomenon known as "irradiation" which promotes greater shoulder stability.
5. Keep your shoulder blades pinched together to ensure the shoulders remain in a safe position.
6. Imagine you're trying to push yourself away from the weights rather than pushing the weights away from yourself.
7. If you're feeling pain within the shoulder joint itself (specifically at the front), ensure your shoulder blades are slightly retracted and try to keep the shoulder girdle "packed".
8. Ensure you maintain some tension in your abs and don't allow your lower back to arch excessively.
9. Keep your feet flat on the floor and don't allow the lower body to move during the set.

Figure 26: Muscle & Strength exercise instructions

Summary:

This website provides videos and written instructions for a wide range of exercises using various types of equipment. Exercises are categorised by muscle group, popularity, equipment, and even mechanics (Compound Exercises and Isolation Exercises). For each muscle group, multiple exercise variations are available, allowing users to train the same muscle in different ways. When selecting an exercise, a video demonstration showing correct execution is displayed immediately. Additional detailed information about the exercise is provided below the video. Overall, the website serves as an effective resource for users seeking clear instructions for specific exercises.

Comparison StrengthLog vs Muscle & Strength:

Both of these websites do their job. The difference between them is that StrengthLog isn't focused on providing exercise information and also includes many other useful features.

Muscle & Strength, on the other hand, is entirely focused on demonstrating, explaining, and sharing everything there is to know about the exercise. StrengthLog is strictly a mobile app and can be used in the gym. Muscle & Strength can be accessed from any device.

StrengthLog requires registration; Muscle & Strength doesn't.

Feature 4: Weight, reps, sets logging

Strong

Strong[11] is a mobile fitness application focused on simple and efficient strength-training logging. It is designed for gym users who want a fast way to record workouts without unnecessary complexity. Users manually select exercises and record the number of sets, reps, and weights during their workouts. The app stores historical data and displays key progress statistics.

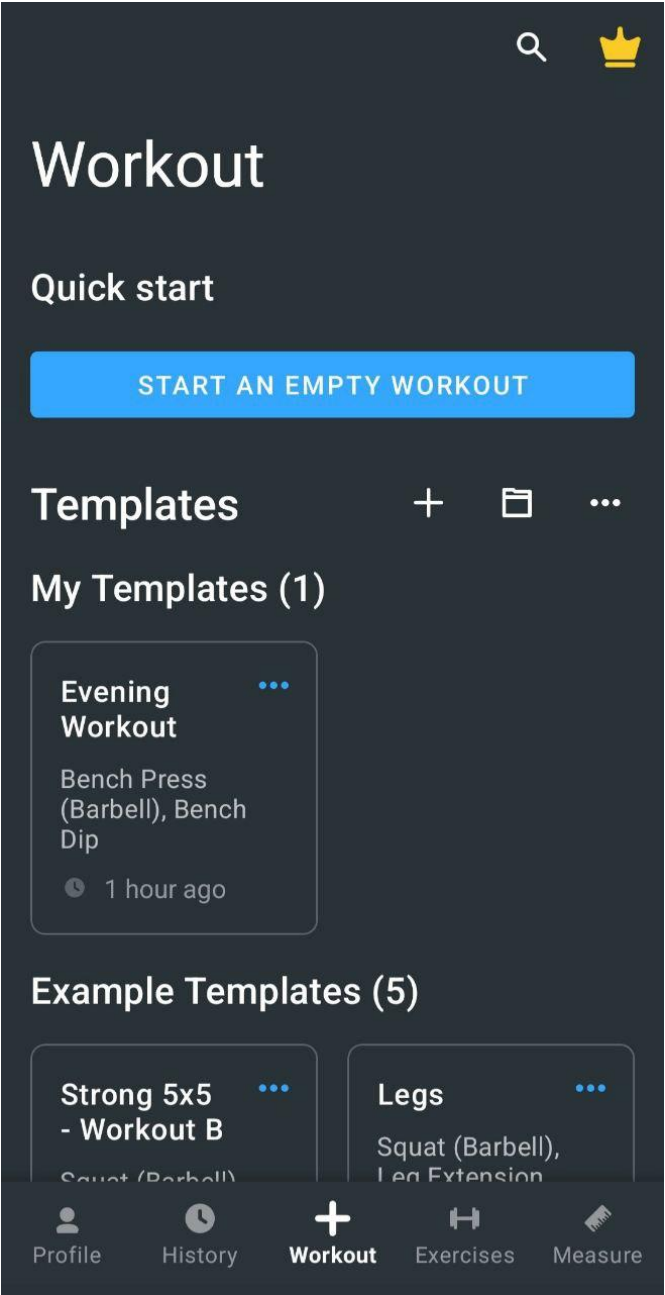


Figure 27: Strong workout menu

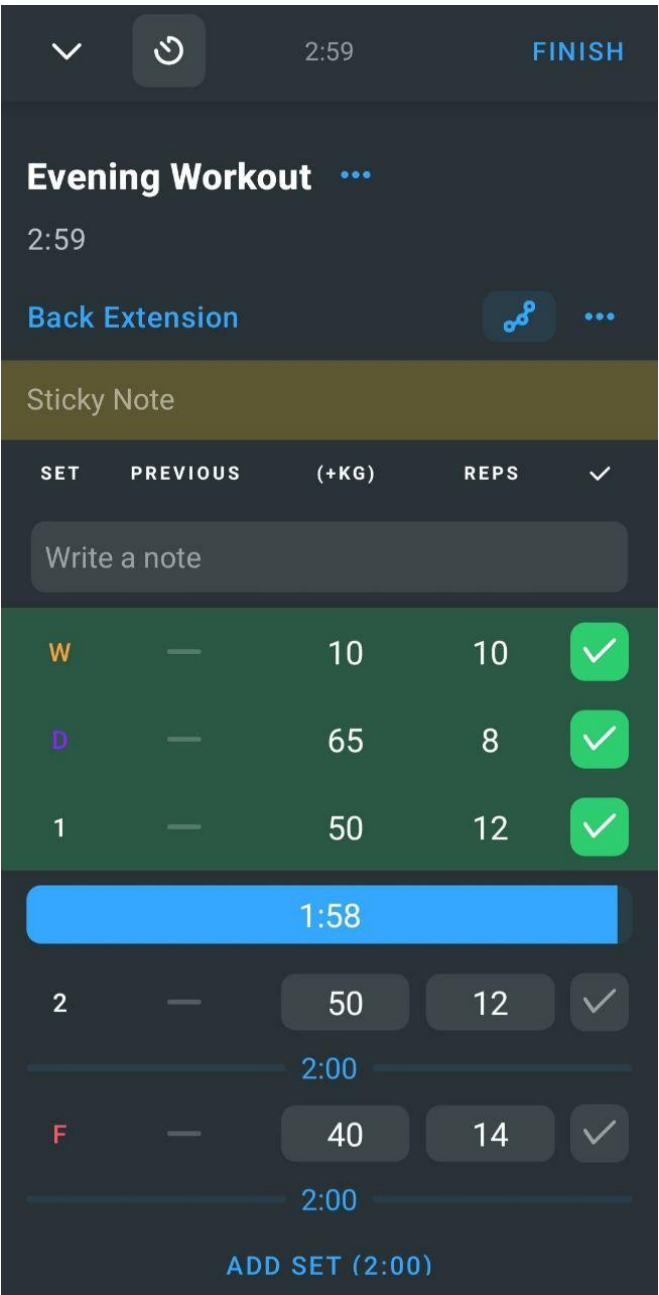


Figure 28: Strong workout logging

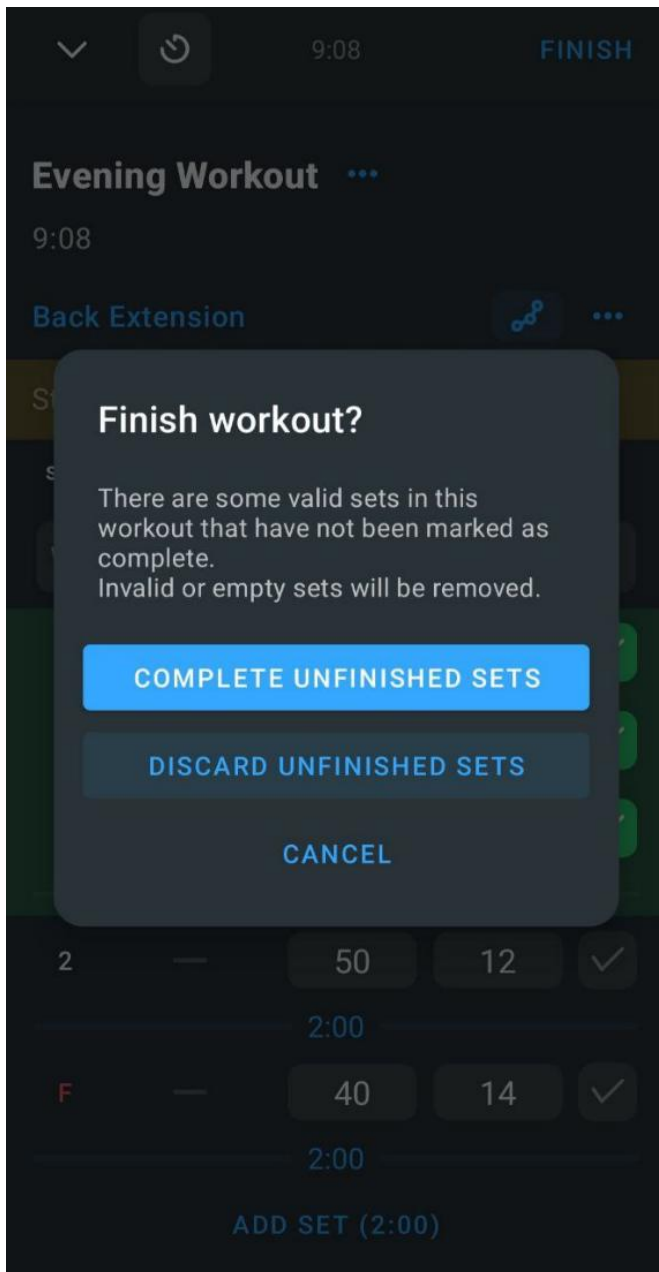


Figure 29: Strong finish workout screen

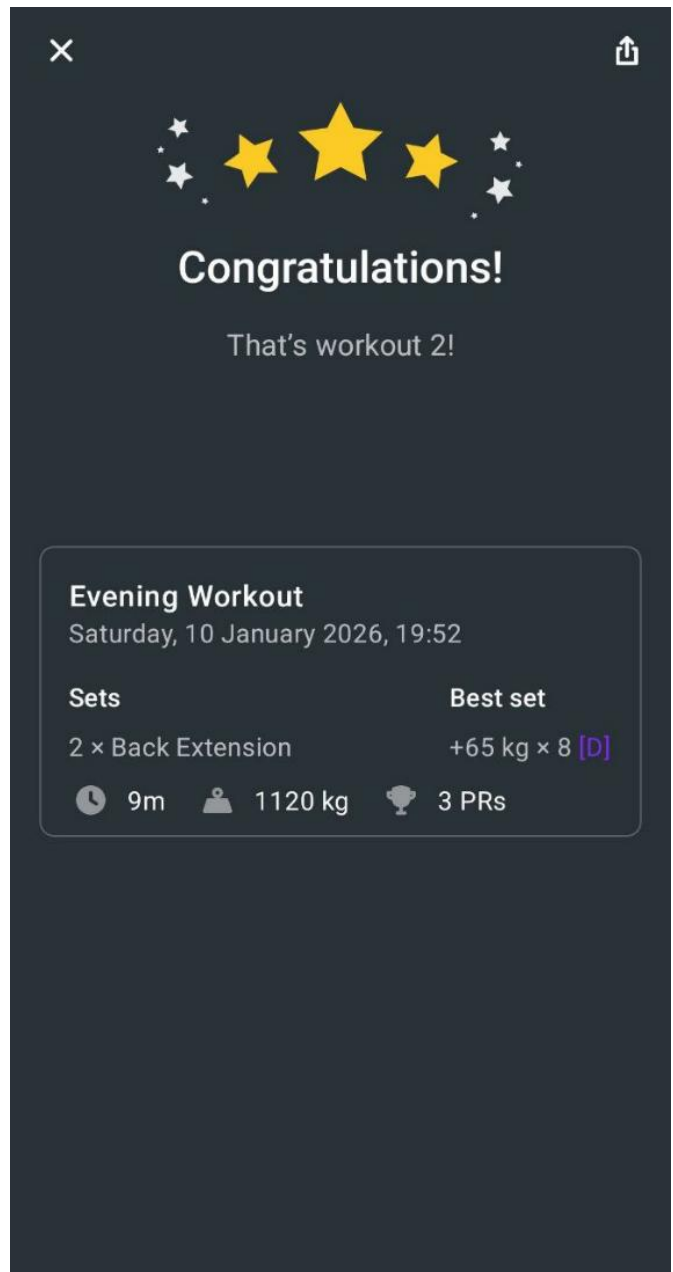


Figure 30: Strong after workout logging screen

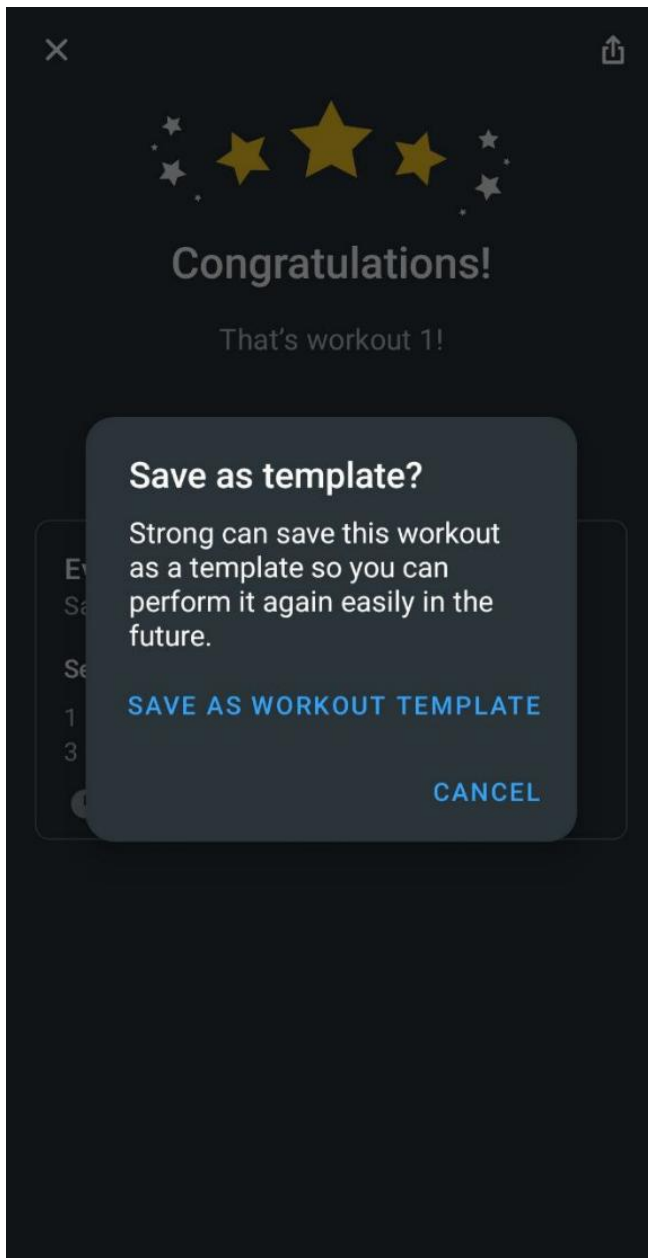


Figure 31: Strong save workout?

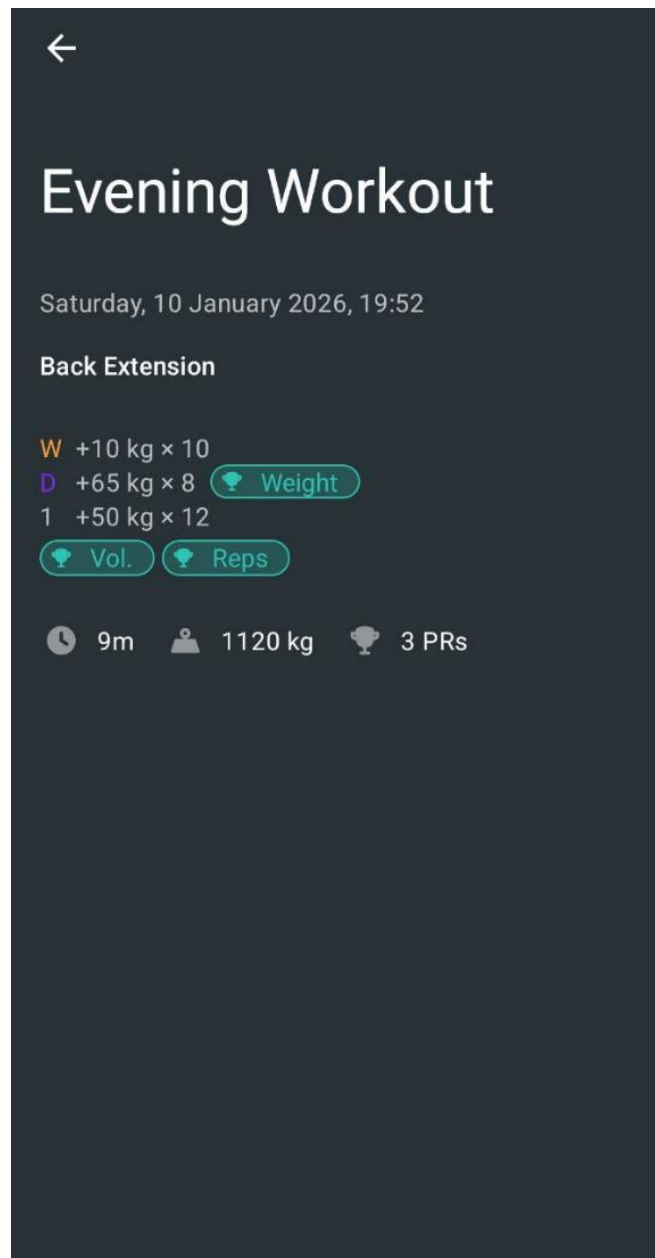


Figure 32: Strong workout results

Summary:

To access the application, users are required to register and create an account. The user interface is relatively simple, making it easy to locate the workout logging feature. Users can select an existing workout programme to follow or create a custom workout. Exercises can be added easily, with options to select weights, repetitions, and the number of sets. Sets can also be categorised, such as warm-up sets, drop sets, or sets to failure. The application does not allow users to manually define a personal record (PR); instead, PRs are automatically assigned when the heaviest weight is recorded, which may be inconvenient for some users. After each set, a default two-minute rest timer is applied, although this can be adjusted or disabled. Once the rest period ends, the device vibrates and plays a notification sound.

Additional features include the ability to add notes, delete unfinished exercises, or mark them as completed. After completing a workout, users can view detailed exercise statistics and save the session to their workout history. Overall, the application provides a convenient and feature-rich workout logging experience and presents useful ideas relevant to the proposed project.

Heavy

Heavy[12] is a strength training app that combines workout logging with analytics and performance tracking. Users manually record exercises, and the app analyzes performance trends throughout the workout, providing statistics such as volume progression and personal records.

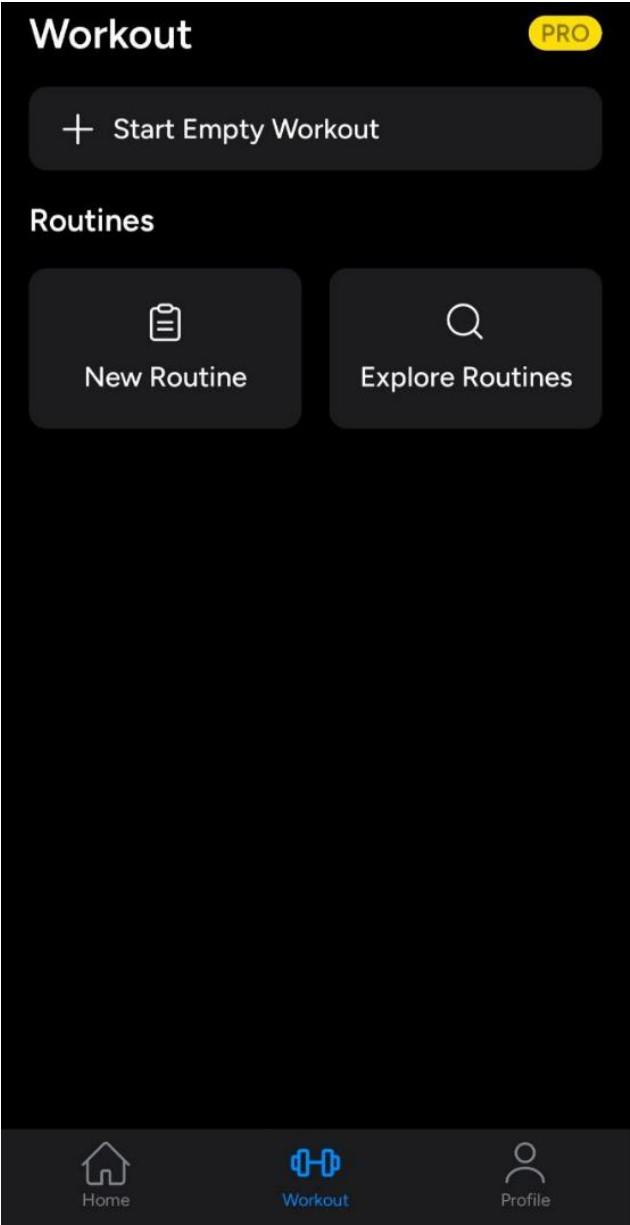


Figure 33: Heavy workouts main screen

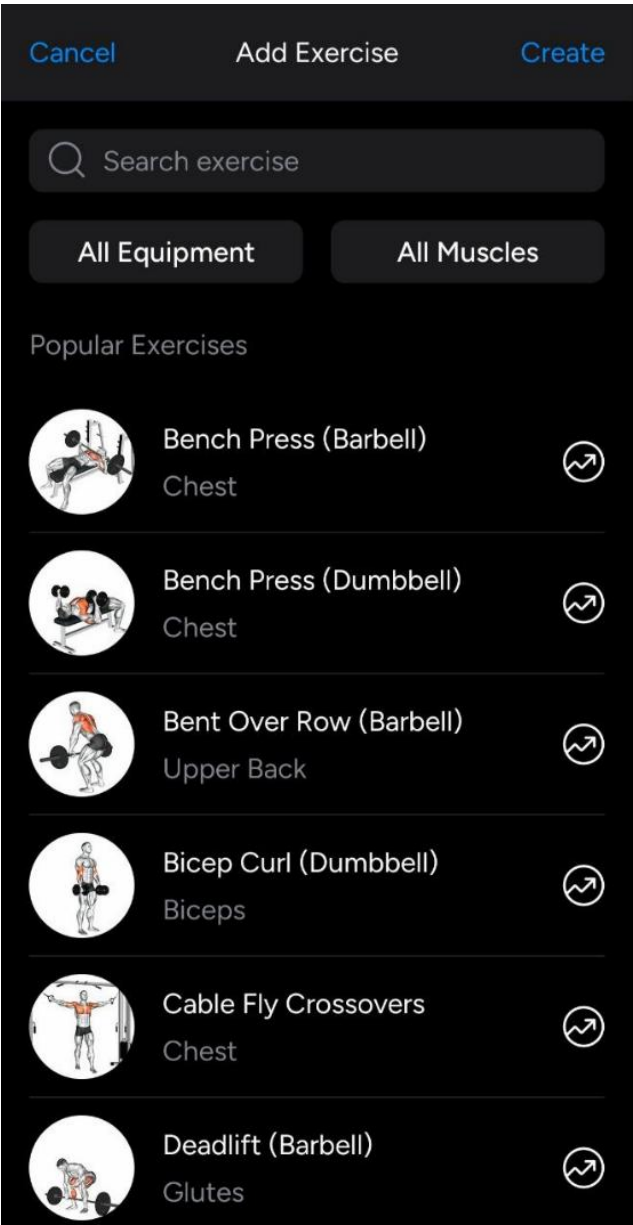


Figure 34: Heavy exercise choice

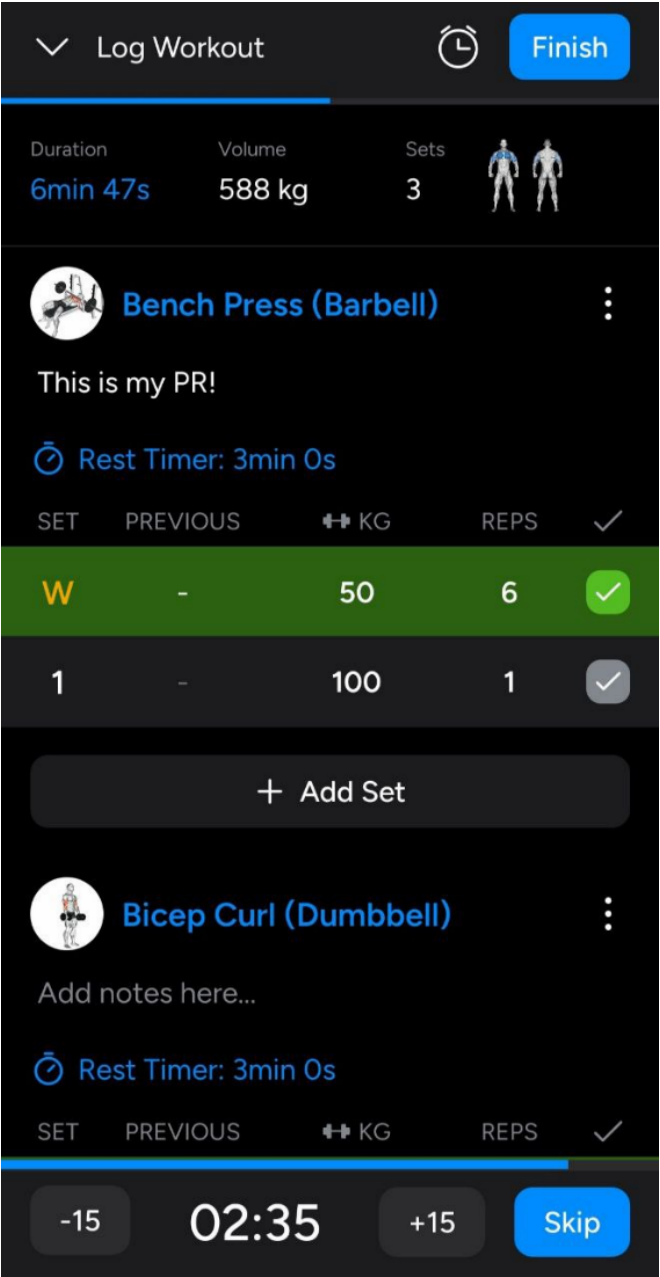


Figure 35: Hevy workout logging screen #1

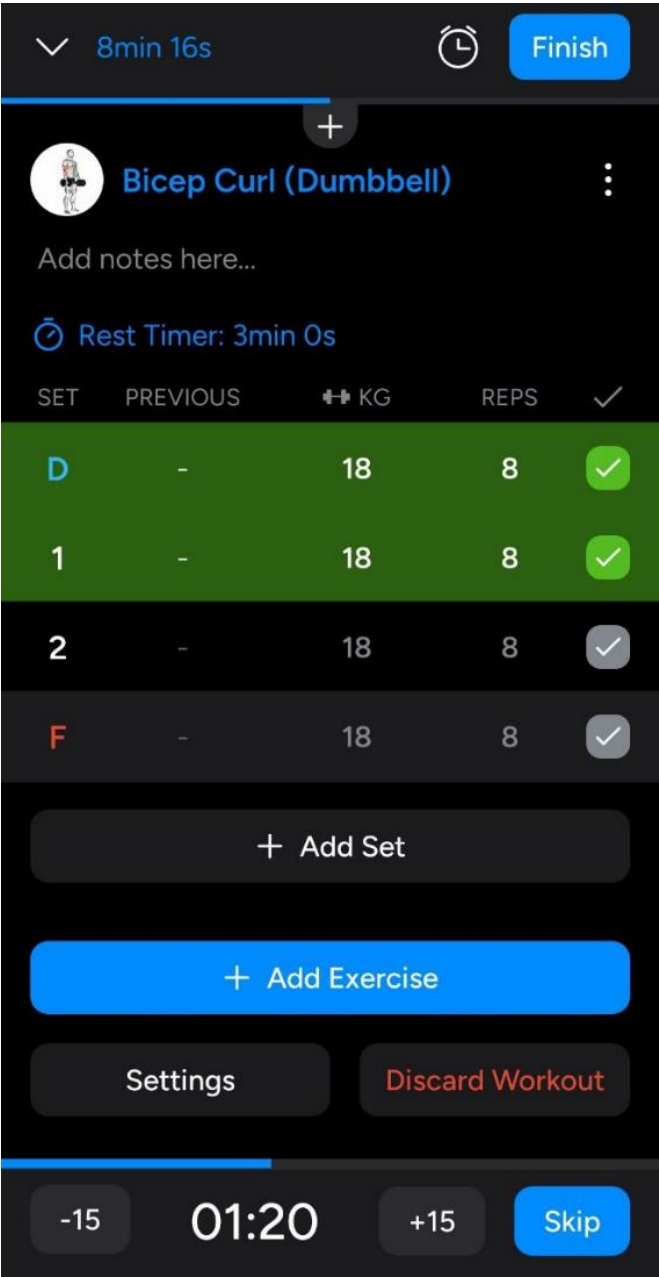


Figure 36: Hevy workout logging screen #2

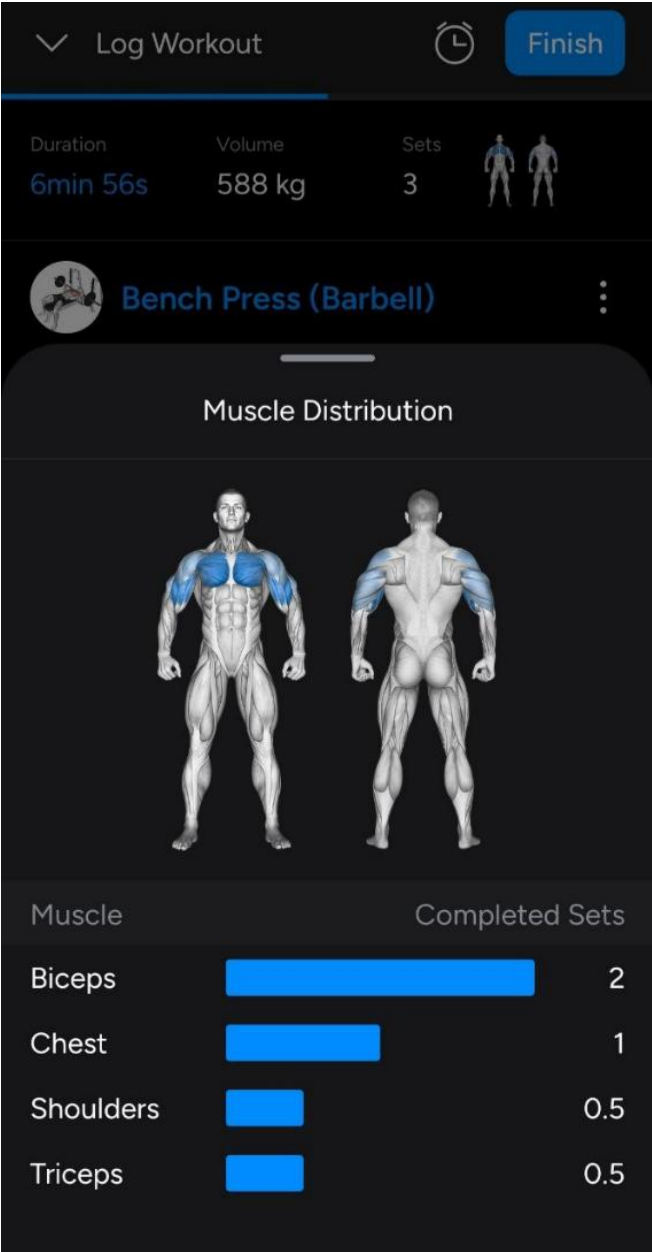


Figure 37: Heavy muscle groups used

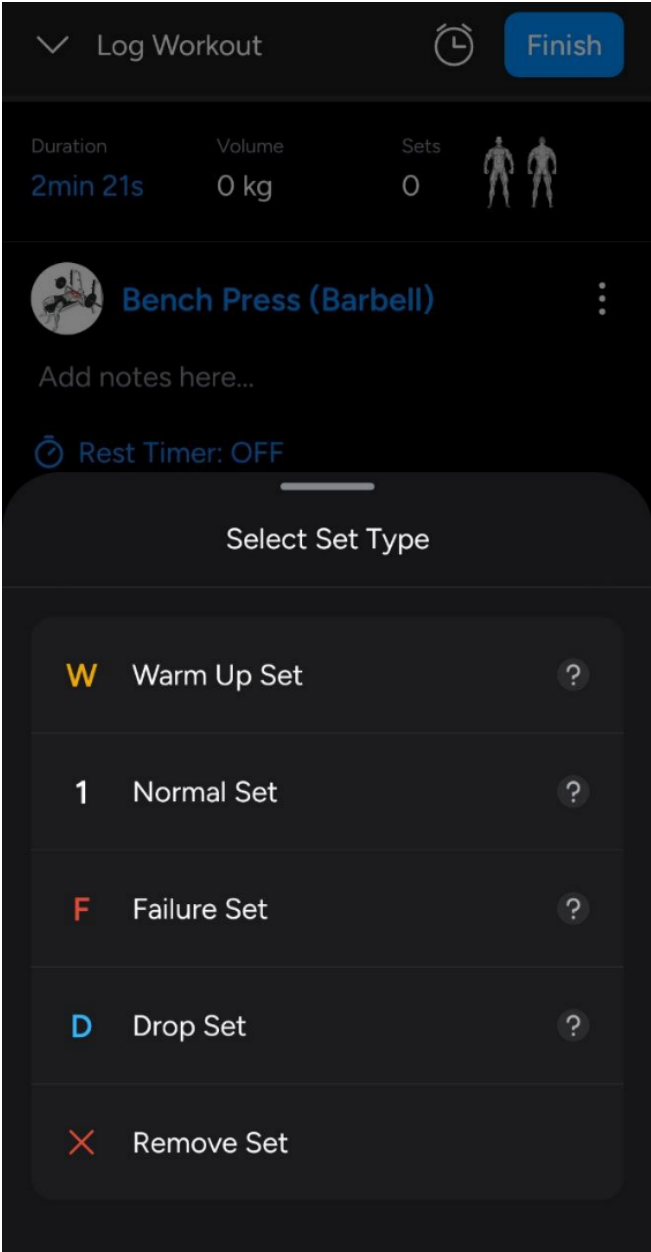


Figure 38: Heavy set types

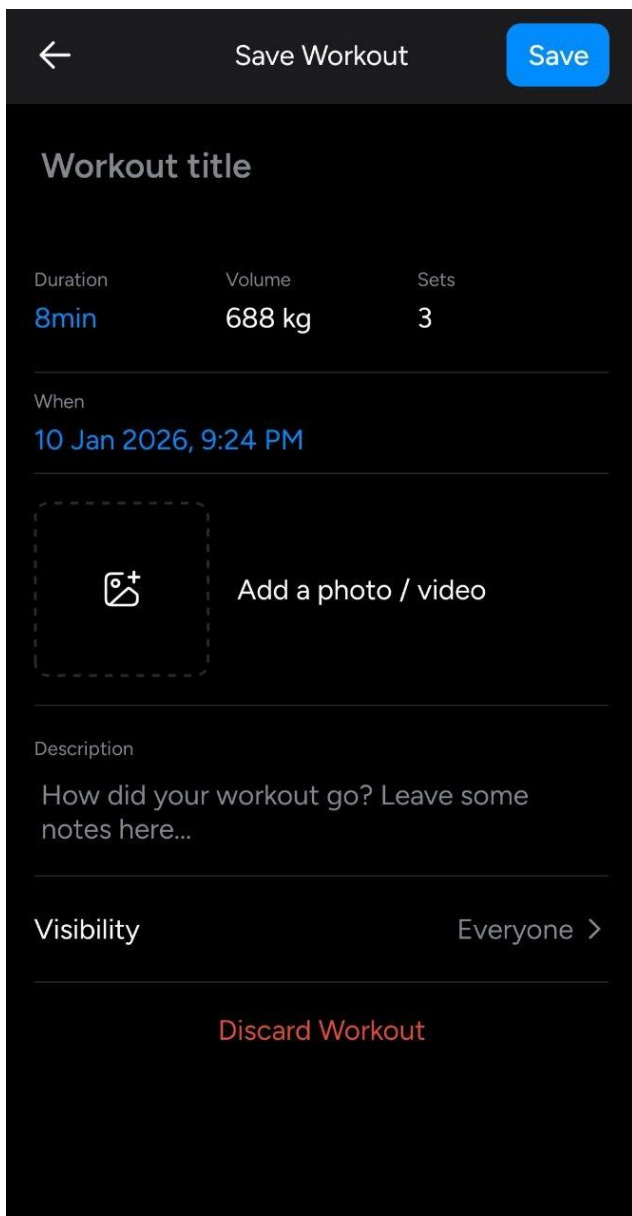


Figure 39: Hevy save workout?

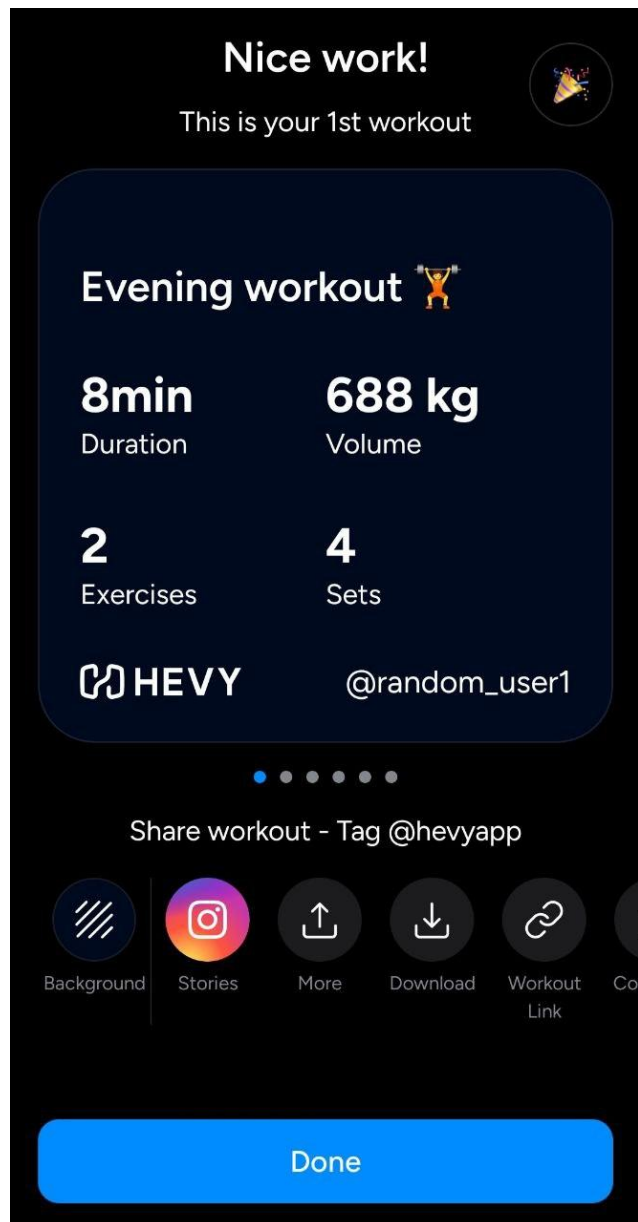


Figure 40: Hevy workout results screen

Summary:

Literally the same functionality and the same idea as the previous application.

Comparison Strong vs Hevy:

The apps were virtually identical, but there were some minor differences. Hevy had a simpler and more user-friendly interface than Strong. In Hevy, you had to set a manual rest timer for each exercise, whereas in Strong it was automatic. In Hevy, every time you pressed the end button for an exercise, the muscles you were using would be highlighted in the human model.

Comparison Table

Table 1: Literature review application comparison table

Feature	Application	Platform	Key Strengths	Key Limitations
Feature 1: Pose estimation techniques	MediaPipe	Mobile (Android, iOS), Web, Desktop	Real-time pose estimation, lightweight, optimised for mobile devices, easy integration with mobile apps	Limited customisation of models, less accurate than heavier frameworks in complex scenes
Feature 1: Pose estimation techniques	OpenPose	Desktop, Server	High accuracy multi-person pose estimation, detailed keypoint detection	Computationally expensive, suitable for real-time mobile use without optimisation
Feature 2: AI-guided Fitness platforms	Tempo	Mobile + Dedicated hardware	Automatic exercise recognition, real-time feedback, strong AI integration	Requires a controlled environment, often hardware-dependent, subscription-based
Feature 2: AI-guided Fitness platforms	Freeletics	Mobile	Personalised AI-driven workout plans, adaptive training programs	No camera-based exercise recognition, relies on manual exercise selection
Feature 3: Exercise info platforms	StrengthLog	Mobile	Clear exercise descriptions, muscle group information, and a clean logging interface	Half of the content is paid, the user interface can be confusing, not ideal for model training
Feature 3: Exercise info platforms	Muscle & Strength	Web	Detailed exercise instructions, educational content, and a large exercise database	No disadvantages, perfect for model training
Feature 4: Weight, reps, sets logging	Strong	Mobile	Fast and simple workout logging, minimalistic design, good usability during workouts	Automatic setting of personal record (Not always convenient)
Feature 4: Weight, reps, sets logging	Hevy	Mobile	Advanced progress tracking, analytics, personal records, strong visualisations	Automatic setting of personal record (Not always convenient)

Competitor Analysis/ Related Work

This section analyses existing technologies and applications related to exercise recognition, posture estimation, and AI-powered workout tracking. The goal is to identify the strengths, limitations, and gaps of existing applications and to justify the design decisions made in this project.

As no single application was identified that satisfies all the defined goals and requirements, several applications with different objectives are compared. Their advantages, relevant features, and limitations are examined in order to identify elements to incorporate into the proposed application and aspects to avoid.

The applications reviewed are categorised into the following groups:

1. Pose estimation techniques (MediaPipe, OpenPose)
2. AI-guided fitness platforms (Tempo, Freeletics)
3. Exercise info platforms (Strong, Hevy)
4. Weight, reps, sets logging platforms (StrengthLog, Muscle & Strength)

Pose estimation techniques

As the proposed application relies on camera-based motion capture in real time, MediaPipe is considered the most suitable framework. It allows for the use of mobile phone cameras and provides fast data processing with relatively low computational requirements. However, MediaPipe provides a limited number of body keypoints, which may limit its performance in some cases. For example, when a group of friends is working together. However, this shouldn't be a problem as long as only one of them is in the camera's field of view.

An alternative framework considered is OpenPose. It offers a more detailed skeletal model with a larger number of keypoints, which will ensure higher accuracy of detailed motion analysis. However, this comes with the risk of requiring significant computational resources. Also, unlike MediaPipe, OpenPose is primarily designed for desktop computers or servers and doesn't use real-time mode, which is completely unsuitable for the PoseFit AI project. So, the high accuracy of detailed motion analysis isn't worth choosing this framework.

AI-guided fitness platforms

Tempo and Freeletics use artificial intelligence for different purposes. PoseFit AI's use of AI will also differ. Tempo uses computer vision and machine learning to provide real-time

exercise feedback through camera analysis. Freeletics focuses on personalising workouts using AI.

The proposed application adopts a different approach. It is designed to use a trained machine learning model to recognise and analyse human movements and postures in real time, before providing the exercise name and relevant instructional information. It's somewhat similar to Tempo, as it also uses machine learning and a real-time camera.

Exercise info platforms

StrengthLog and Muscle & Strength provide excellent resources for users interested in what exercises are, how to do them correctly, and which muscle groups they target. The problem is that to find the right app, users need to know at least its name, which isn't always clearly given the vast number of different exercises.

The proposed application eliminates this problem, as simply using the camera while performing the exercise will show all the information users need.

Weight, reps, sets logging platforms

Strong and Hevy are similar apps, and both feature a custom workout feature that lets users add various exercises, reps, weights, and sets. They can not only create and save their own workout but also use pre-made ones. These applications also offer a bunch of other features, such as adding notes, setting a timer for rest periods, and so on.

The proposed application does not aim to include a custom workout feature, and it'll be much simpler. Instead, it focuses on allowing users to be able to add only their personal bests, so they can see how much weight they can lift.

Identified gaps in existing platforms

The analysis of current competitors reveals several consistent limitations:

- Lack of **automatic exercise recognition** in standard gym environments
- Dependence on **manual exercise selection**
- Heavy reliance on **subscriptions or proprietary hardware**
- Limited consideration of **privacy and on-device processing**
- **Fragmentation of features** across multiple applications

No reviewed solution combines real-time pose estimation, exercise identification, instructional content, and workout logging into a single, lightweight mobile application.

3. Tools and Methods

In this section, the way of how the project is performed and the main factors which impact on the technical and methodological choices are described. The development of the project is based on the design-and-build principle, while the research and experimentation, as well as the continuous improvement support the project development. The choices made concerning tools, frameworks, and methods are based on the results of the literature review, competitor analysis, practical limitations in terms of performance, usability, and privacy.

The artefact is created by taking the form of a mobile application that captures human motion with the use of a smartphone camera and applies pose estimation algorithms to identify exercises. The data of human poses is collected in real time and processed by means of machine learning in order to detect exercise patterns. The identified exercise is then displayed to the user with the associated information on how to perform it and not required but optional manual entries into the workout data including sets, repetitions and weights.

The project data collection comprises of recorded exercise videos that were used in the experimentation and testing of pose estimation and classification methods. None of the personal information is captured unless users agree to it and all processing is made to happen on the device where it can. This strategy complies with the legal and ethical demands concerning data protection and privacy.

The process of development adheres to the evidence-based design methodology, with system architecture, user interface design and functionalities being smoothly improved, depending on the technical feasibility, the usability guidelines, and the comments provided during the supervision sessions. Technical decisions are constantly judged in accordance with the legally, ethically, and professionally identified, and the system is safe, open, and responsible.

Rationale for the technical tools selected

A number of technologies and tools were chosen in order to facilitate the adoption of the proposed system with regard to appropriateness, performance, and matchability with the requirements of the project.

Flutter has been selected as the main development framework of the mobile application since it can be used to develop both Android and iOS modules based on one codebase. This makes the development less complex and also provides a consistent user interface and performance

across platforms. Other native development models were also thought of but they were not adopted because they required more time to develop and maintain.

MediaPipe was chosen to be used in human pose estimation because it can be used to provide real-time processing on mobile with comparatively small computation needs.

MediaPipe is smaller in size than other options like OpenPose and can be deployed to an on-device processing system and a mobile platform. Even though OpenPose has a more detailed skeletal model, it has more hardware requirements and is not supported by real-time mobile applications, thus not fitting this project.

The training and evaluation of machine learning models to classify exercises are supported by the TensorFlow. It is flexible and scalable with good community support, so it is suitable to be used in experimentation and extended in the future. Other simpler rule-based methods were also taken into consideration and discarded because of the lack of adaptability and accuracy in the ability to identify more complex patterns of movements.

The non-sensitive user data and workout logs are stored in a SQL-based local database that ensures the management of data in a structured and efficient way. Platform-specific secure storage procedures are used to process sensitive authentication data and ensure confidentiality and adherence to the principles of data protection.

GitHub provides support to project management and documentation, version control, issue tracking, and Kanban boards. This allows the process of clear organisation of tasks, monitoring of progress and indication of iterative development in the project life cycle.

Rationale for the research methods used

The project will be based on an applied research approach that involves a qualitative and technical research design.

To examine the current literature regarding pose estimation, exercise recognition, and AI-based fitness apps, a literature review is carried out. This will give a basis of evidence regarding the choice of technologies and the gaps in the existing solutions.

An evaluation of strengths and limitations is carried out through a comparative analysis of available applications and frameworks, which aids in refining the project requirements and making decisions in the design.

The technical implementation is developed in the form of an experiment. This involves generation of datasets, pose estimation accuracy testing, and cyclic process of refining the exercise recognition model with regards to perceived performance.

The methods of evaluation will involve:

Accuracy of exercise recognition in functional mode.

Mobile interface usability test.

Ethical, legal and professional reflection.

Project management and Product lifecycle

The project management of this project is facilitated by the application of the GitHub Projects and Kanban boards, which is utilised to plan the tasks, monitor the progress, and control development activities on all project phases. The tasks are categorized into rational stages, which include research, development, testing, and documentation, and the development can be reviewed and reported on a regular basis.

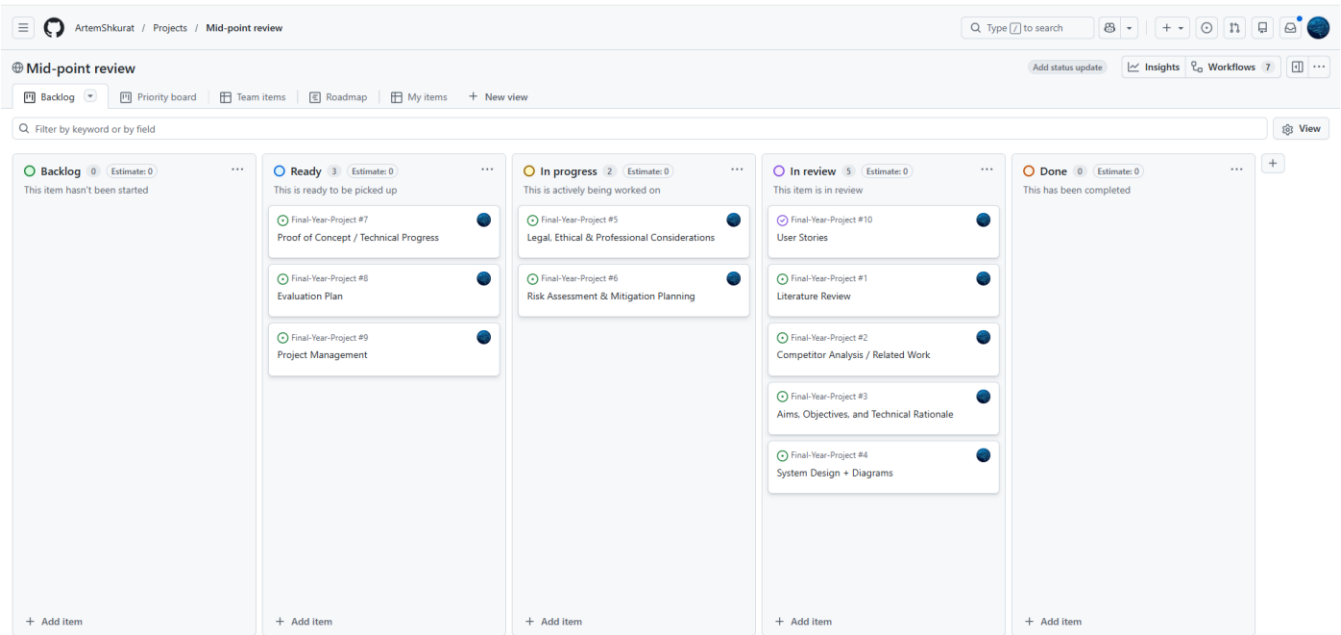


Figure 41: Kanban board

Beside a Kanban board, there is the Gantt chart that plans and visualises the timeframe of the project in general. The Gantt chart gives a logical view of the important milestones, the time allocated to various tasks and dependencies such as research and literature review,

application development, model training, integration, evaluation, and final submission. The method will enable efficient time management, chances of the probable risks that appear during the schedule, as well as alignment of the project with the academic due dates.

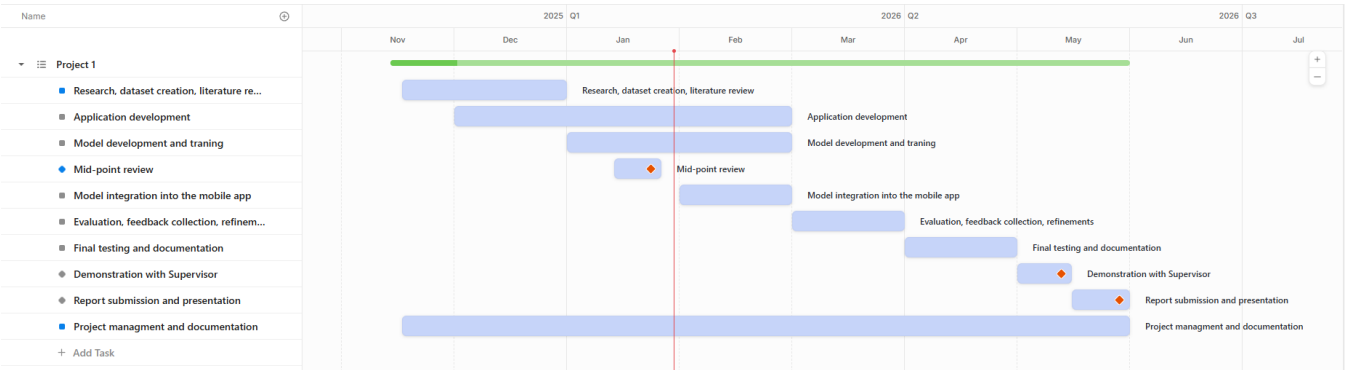


Figure 42: Gantt chart

The project is based on an iterative and agile-inspired development lifecycle, in which the feedback on supervision sessions and evaluation activities is applied to improve the design decisions and implementation during the development process.

Legal, ethical and Professional issues

This section discusses the importance of potential legal, ethical, and professional considerations related to the proposed mobile application. Addressing these considerations is essential to ensure that the project does not violate any laws, complies with relevant regulations and legal obligations, and remains responsible and fair to users. This section also outlines how these considerations are addressed and how responsibility is taken in various situations throughout the project.

Legal Considerations

(1) User Consent and Camera Access

Issue:

Using the camera without the user's consent may violate data protection laws.

Solution:

The app's camera won't be usable until the user consents. After consent, the app will clearly inform users when the camera is active, and its access can be easily disabled through the

app's settings.

(2) Data Privacy and Protection

Issue:

When using the camera to recognise an exercise, the video is processed by a model to identify key points on the user's body, which, according to the UK GDPR, may be classified as personal data. The user may not be aware of how the app uses this data and may be concerned that it could be shared without their knowledge.

Solution:

The app will contain a link to the Privacy Policy, which states:

1. The app does not save or upload any user videos.
2. Only basic workout information (e.g., exercise name, number of sets, reps, and weight) is saved and stored locally on the device.
3. What data is used, why it is needed, and how it is protected.

(3) Data Storage

Issue:

Storing personal data longer than necessary or without user control may breach GDPR data minimisation and storage limitation principles.

Solution:

All user data is stored locally on the device, so you can manually edit or delete workout records if you wish.

(4) Account Security

Issue:

Users may have concerns about insecure processing of authentication data and data leakage.

Solution:

User authentication credentials will never be stored in cleartext. Secure platform-specific storage mechanisms will be used for their storage.

(5) Liability

Issue:

Users may interpret exercise recognition and exercise technique recommendations as professional or medical advice, which could potentially lead to injury or misuse.

Solution:

The app will include unavoidable disclaimers next to the feature, which will state that the app does not provide professional or medical advice and that exercise information is provided for educational purposes only.

Ethical Considerations

(1) User Privacy

Issue:

The user may feel uncomfortable knowing that the camera is analyzing his/her movements.

Solution:

If the user feels uncomfortable using the camera, the app offers a manual option to find an exercise. However, if the user still wants to use the camera, they will first need to grant permission. Even then, the camera is used in real time and does not save video without the user's consent.

(2) Bias in AI Recognition

Issue:

Artificial intelligence models may be biased against certain groups of people, which could lead to unfair or inaccurate assessments of the effectiveness of exercise. Such groups may include people with certain body types, genders, clothing styles, and races.

Solution:

To reduce model bias, it will use training data that covers a wide and diverse range of users. Exercise recognition will also primarily focus on movement patterns rather than appearance.

(3) Risk of Misclassification

Issue:

If the app misidentifies the exercise, users may be misinformed or confused about what they are doing.

Solution:

The app won't show exercise recognition results if the model's confidence isn't high enough and will request a re-shoot. The user can also find or log the exercise manually if needed.

(4) User Safety**Issue:**

Users may rely on the app's recommendations but perform the exercises incorrectly, which could potentially increase the risk of injury.

Solution:

The disclaimer will clearly state the limitations of the system and state that the app is only intended to help users understand the exercises and is not a replacement for a personal trainer.

(5) Transparency**Issue:**

Users may not understand how and why the AI system recognises a particular exercise, which can reduce trust in the app.

Solution:

The manual will explain how the application functions and its main features.

Professional Considerations**(1) Professional Use of Third-Party Technologies****Issue:**

Incorrect integration and failure to adhere to the expiration dates of third-party libraries can lead to unexpected risks and problems with code.

Solution:

Always check dependencies for reliability, suitability, and expiration dates. Also, avoid using libraries from suspicious or unfamiliar websites.

(2) Help, Support, and User Guidance

Issue:

If users do not understand how to use the application, this may cause confusion, misuse of features, or incorrect assumptions about exercise performance.

Solution:

The application will provide clear in-app guidance, such as brief explanations and tooltips. This will help users understand key features and reduce the risk of user error.

(3) *Timely support***Issue:**

If users have had a poor experience using the app, they may be tempted to use the Help and Support features. A delayed response to user complaints can undermine their trust and loyalty.

Solution:

Always be ready to help users with problems. Treat all their concerns with respect and understanding.

Risk register

This section focuses on reviewing, assessing, and mitigating all risks that may be associated with the PoseFit AI application.

Risks:

R1: The pose-estimation model may incorrectly classify exercises (especially when movements are similar or partially occluded).

R2: Real-time position estimation may produce poor results on older or low-power mobile devices.

R3: Insufficient or undiversified training data may reduce the accuracy of the model when dealing with different body types, camera angles, or conditions.

R4: Improper handling of access to cameras or video data.

R5: Underestimating the time required to train, test, and refine a model.

R6: Failure to follow or deviation from the application development plan.

R7: Complex workflows or an unclear user interface can make the app difficult to use.

R8: Users may treat app feedback as professional fitness advice.

R9: Changes, deprecation, or licensing issues with MediaPipe, TensorFlow, or other libraries.

R10: Insufficient user testing or evaluation results.

R11: The posture estimation model may not be able to recognize the exercise in some cases.

R12: Workout logs or user data may become corrupted due to crashes or improper handling.

R13: Errors in pose detection or classification may be hard to trace.

		Severity				
		Negligible	Minor	Moderate	Significant	Severe
Likelihood	Very Likely	Low Med	Medium	Med Hi	High	High
	Likely	Low	Low Med	Medium	Med Hi	High
	Possible	Low	Low Med	Medium	Med Hi	Med Hi
	Unlikely	Low	Low Med	Low Med	Medium	Med Hi
	Very Unlikely	Low	Low	Low Med	Medium	Medium

Figure 43: Risk assessment

Table 2: Risk register table

Reference	Probability	Impact	Risk Score	Mitigation Actions
R1	Possible	Moderate	Medium	Train the model on a variety of different data until its accuracy increases too virtually error-free.
R2	Likely	Moderate	Medium	Add a feature to the camera settings for phones with poor performance that will limit camera processing to a single detected user.
R3	Possible	Significant	Med Hi	Collect a variety of training data covering different body types, angles, lighting conditions, and environments.

R4	Unlikely	Severe	Med Hi	Ensure explicit user consent, avoid storing video files, and comply with GDPR and UK data protection regulations.
R5	Likely	Significant	Med Hi	Find and prepare all the necessary data for training the model in advance. Prepare a plan to follow.
R6	Possible	Moderate	Medium	Break the plan into smaller chunks to make it easier to follow. Maintain and regularly update the project plan. Use Kanban boards and management feedback to track progress adjust tasks as needed.
R7	Unlikely	Significant	Medium	Apply usability guidelines. Create interface prototypes and then follow them. Collect user feedback and simplify workflows where necessary.
R8	Unlikely	Significant	Medium	Clearly state disclaimers that the app is not a substitute for professional advice.
R9	Unlikely	Severe	Med Hi	Monitor updates and licenses for third-party libraries. Prepare alternative frameworks.
R10	Possible	Minor	Low Med	Conduct usability testing with a large number of users. Analyze all the feedback received and draw conclusions.
R11	Possible	Moderate	Medium	If exercise recognition fails, make the system prompt the user to film the exercise from a different angle.
R12	Unlikely	Significant	Medium	Use database transactions, input validation, and exception handling. Regularly test save/load operations and handle app lifecycle events correctly.
R13	Possible	Significant	Med Hi	Add debugging mechanisms during the development phase. Use clear model evaluation metrics and step-by-step testing of pose detection and classification stages.

4. Requirements/Specifications

Personas

Table 3: Persona 1

Picture 	Description Name: Tom Demographic Age: 16 Gender: Male Tom is an ordinary teenager who decided he wanted to get a little stronger and achieve a more athletic physique. To that end, he bought a gym membership and started going there on his own.
Goals - To become stronger and achieve athletic physique. - To learn about gym exercises. Frustrations - No idea what the exercise names are. - No idea what exercises to do or how to do them. Wishes - To get a way to easily identify exercises. - To get a source with information on all exercises.	Scenario Since he was still a teenager, he didn't have enough pocket money to work with a trainer, so he worked out alone. He was new to using the gym's equipment, so he didn't understand how to do it correctly or know the names of the exercises. He wanted to learn about the exercises other people were doing, but he didn't want to be too intrusive or bother them. That's when he discovered an app called PoseFit AI, which could solve all his problems. With this app, he could get the names of exercises simply by using his phone's camera. By pointing it at anyone performing an exercise, the app could tell him exactly what the exercise was and its details. Using this app, he could perform exercises independently, without bothering others, and get useful information about any exercise he would need again and again.

Table 4: Persona 2

Picture 	Description Name: George Demographic Age: 43 Gender: Male Georgy is a mature man who spends most of his time at work. Recently, he's noticed he's gaining weight due to his sedentary lifestyle. Consequently, he's decided to take up exercise to improve his fitness.
Goals • To get back into shape. • To avoid spending a lot of time on exercises. • To not injure yourself. Frustrations • There is not enough time to go to the gym. Wishes • To filter exercises by those he could do at home. • To get useful tips on how to do exercises correctly. • To get all the useful info about exercises in no time.	Scenario Because Georgiy spends almost all of his time at work, he doesn't have much time for exercise. Working out at the gym is a more effective option, but it's too time-consuming, so he decides to work out at home. He didn't know many exercises and was afraid he might injure himself with improper form. He needed something that would show him how to do the exercises correctly without constantly searching online, and that's when he discovered PoseFit AI. This app had everything he needed. He was able to filter exercises by those he could do at home and those that only require a gym. Each exercise also had a video showing how to perform it correctly. He also found helpful tips on performing exercises that made it easier to work on them. As a result, he could work out at home, doing the exercises correctly, without spending a lot of time and money on a gym.

Table 5: Persona 3

Picture 	Description Name: Alexa Demographic Age: 19 Gender: Female Alexa is an advanced gym user. She's been going there several times a week for several years and already knows most of the exercises. Every time, before starting an exercise, she needs to remember the exact number of reps, weights, and sets she needs to do, and she can't always figure it out right away. This really irritates her.
Goals • To track training results. • To avoid wasting time remembering reps, weights, and sets every time she starts trains. Frustrations • She constantly forgets the results of her training. Wishes • A place where you can save your training results. • Filter training results.	Scenario One day, when Alexa was returning from a long vacation, she wanted to go to the gym, but when she got there, she couldn't remember most of what she'd done. She needed a way to track all her reps, weights, and sets without having to constantly remember them. Then she started looking for a solution and came across an app called PoseFit AI. Along with other useful features, she found a tab where she could save her workout results and review them at any time. It also allowed her to save her personal records and filter saved records by exercise and date saved. After Alexa started using this app, she no longer has problems with forgetting her workout results and she doesn't have to waste time remembering them.

User stories

1. As a **user**, I want to recognize exercises using the phone's camera, so that I can save time searching for it on the internet.
2. As a **user**, I want to see the name of the exercise when searching for it, so that I can memorize it and use it later.
3. As a **user**, I want to know which muscles are being used in different exercises, so that I can better understand what to train.
4. As a **user**, I want to see the instructions on how to perform the exercise correctly, so that I don't injure myself doing it wrong.
5. As a **user**, I want to see a visual representation of how to perform the exercise correctly, so that I can repeat after it.
6. As a **user**, I want to keep a record of the number of repetitions, sets, and weight I use during the training, so that I don't have to worry about memorizing them.
7. As a **user**, I want to keep track of my PR (Personal Record), so that I can be sure I'm progressing.
8. As a **user**, I want to delete or edit my own exercise entries, so that if I don't need them or want to change them.
9. As a **user**, I want to see a list of all exercise records, so that I can see what weight I used last time.
10. As a **user**, I want to see the date when an exercise entry was saved so that I can easily identify the last time I performed that exercise.
11. As a **user**, I want to filter exercises by name and date, so that I can easily find the one I need.
12. As a **user**, I want to have an option to choose between kilograms and pounds, so that I can choose the most suitable one.
13. As a **user**, I want to create or log in to an account, so that I can personalize the app.
14. As a **user**, I want an app to ask for permission to use the camera or save user data, so that I feel safe in relation to personal information.
15. As a **user**, I want to see an explanation of how the user's personal information is used, so that I can make an informed decision about using the app.

User journeys

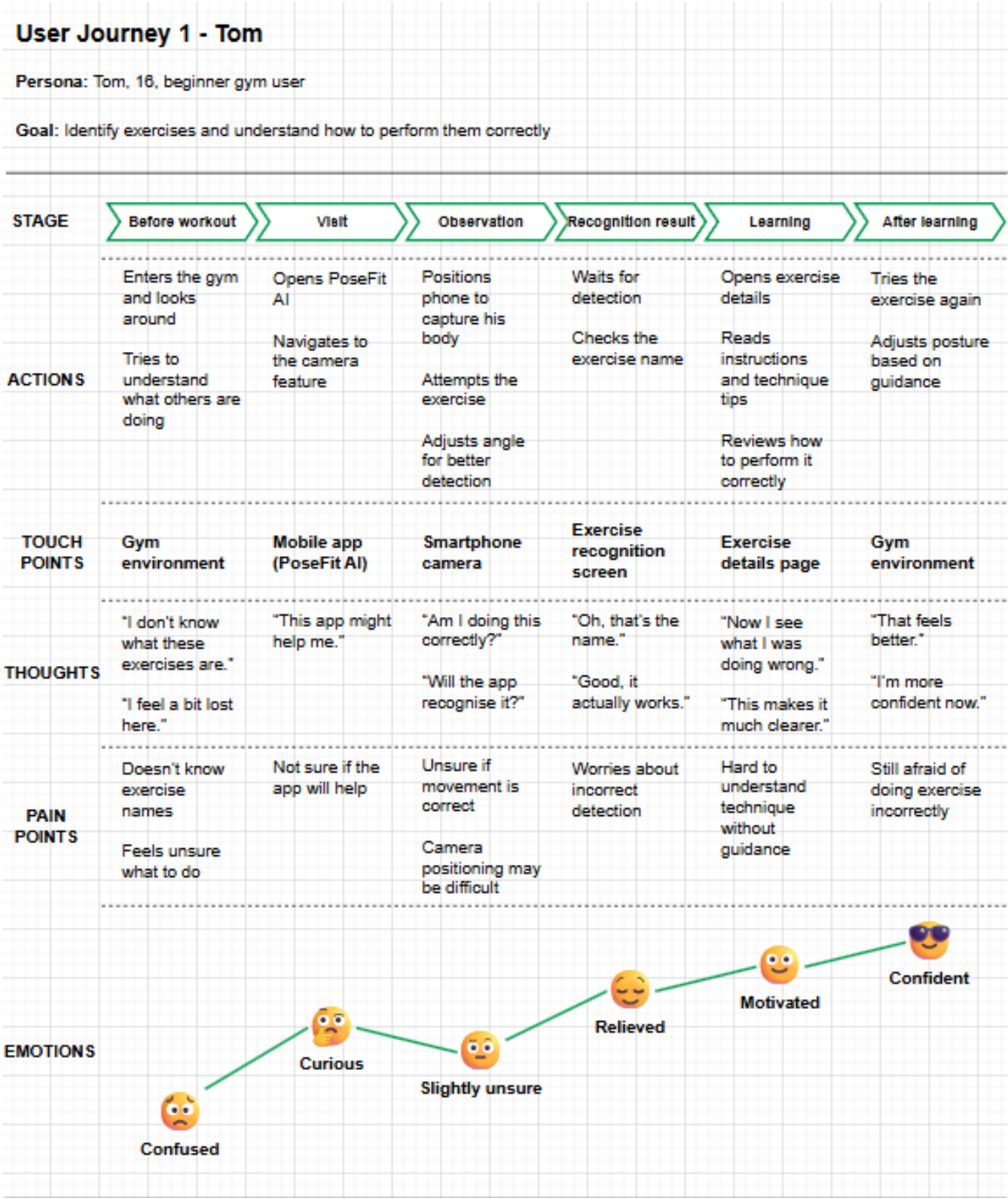


Figure 44: User Journey Diagram - Beginner Gym User

User Journey 2 - George

Persona: George, 43, busy adult working from home

Goal: Quickly find suitable exercises and perform them safely without wasting time

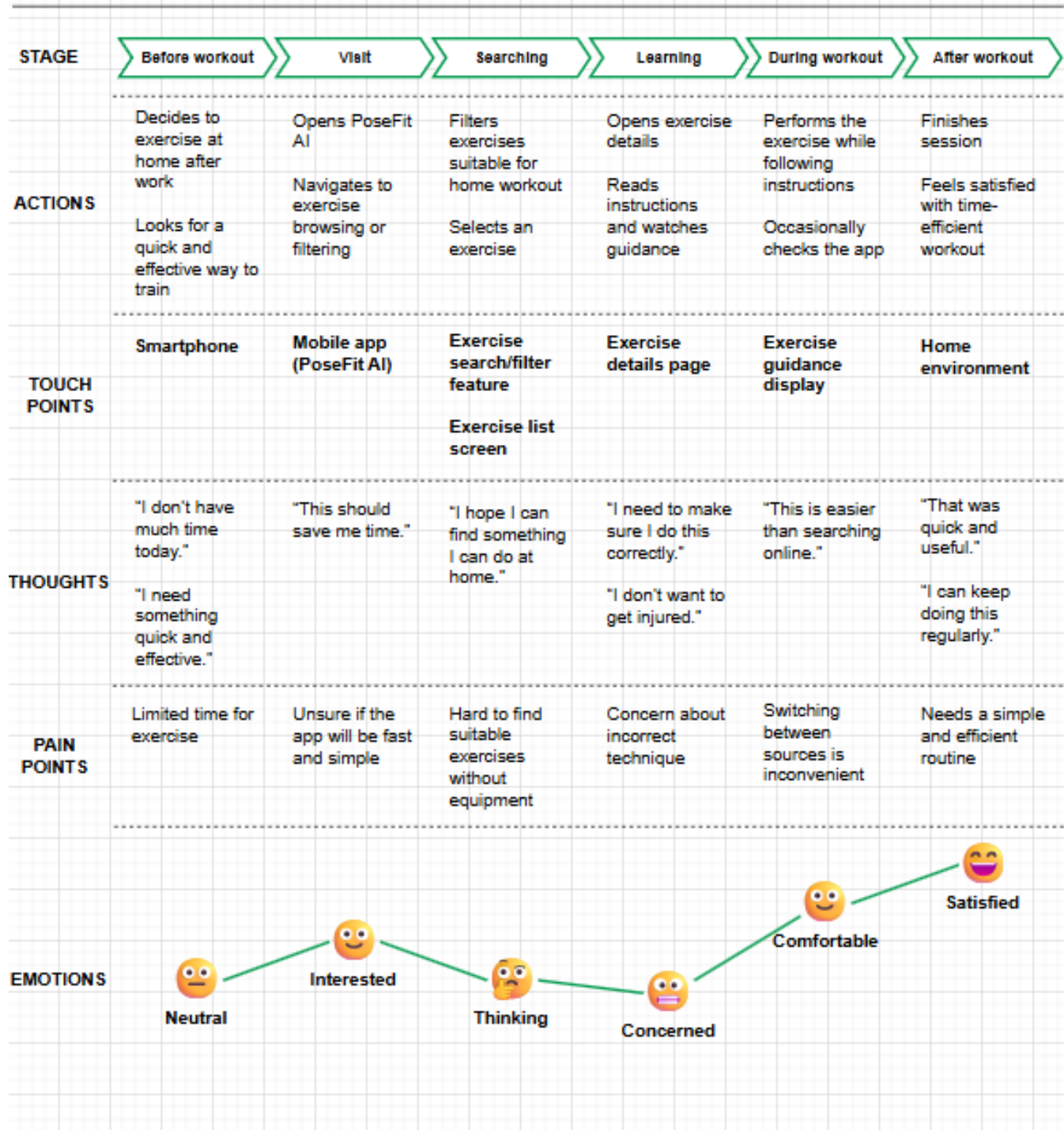


Figure 45: User Journey Diagram - Home Workout User

User Journey 3 - Alexa

Persona: Alexa, 19, experienced gym user

Goal: Track workout performance and quickly access previous results to improve progress

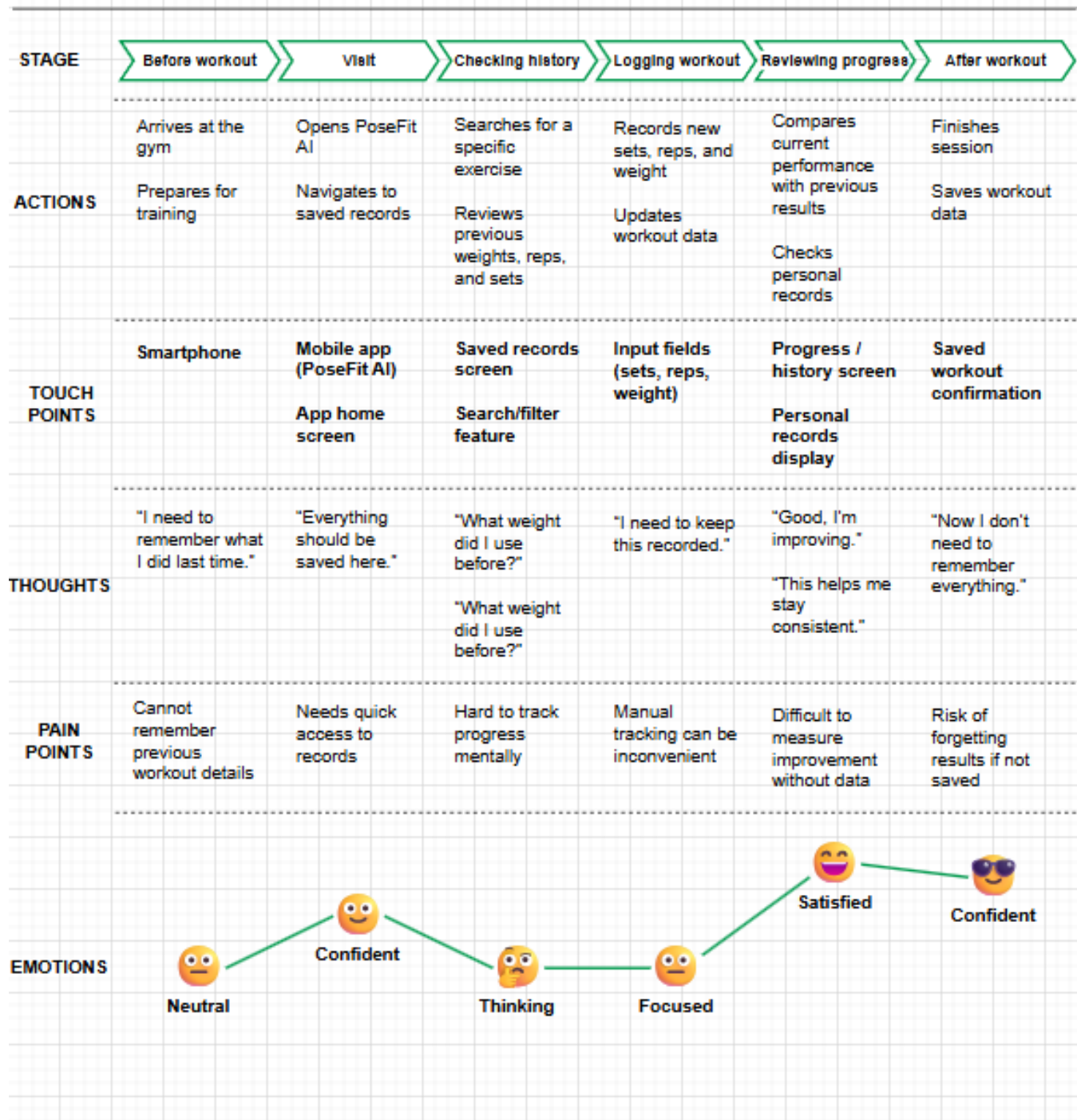


Figure 46: User Journey Diagram - Advanced Gym User

Use case diagrams

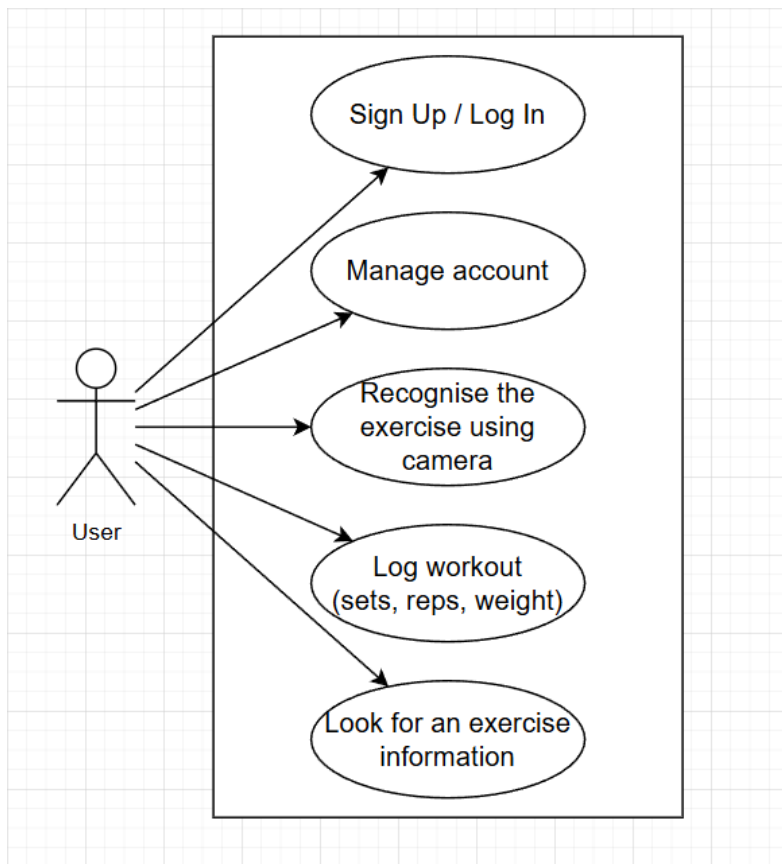


Figure 47: Use Case Diagram - PoseFit AI System

Wireframes

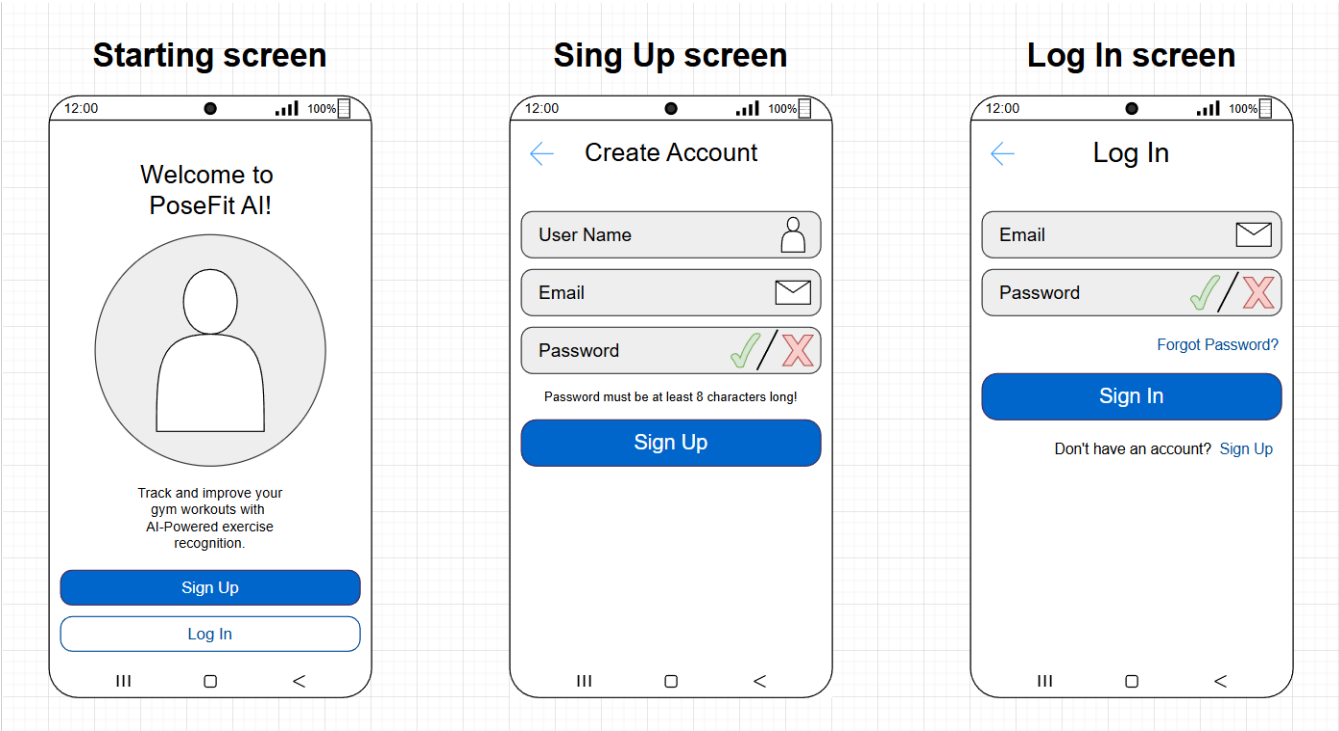


Figure 48: PoseFit AI Starting, Sign up, Log in screens Wireframe

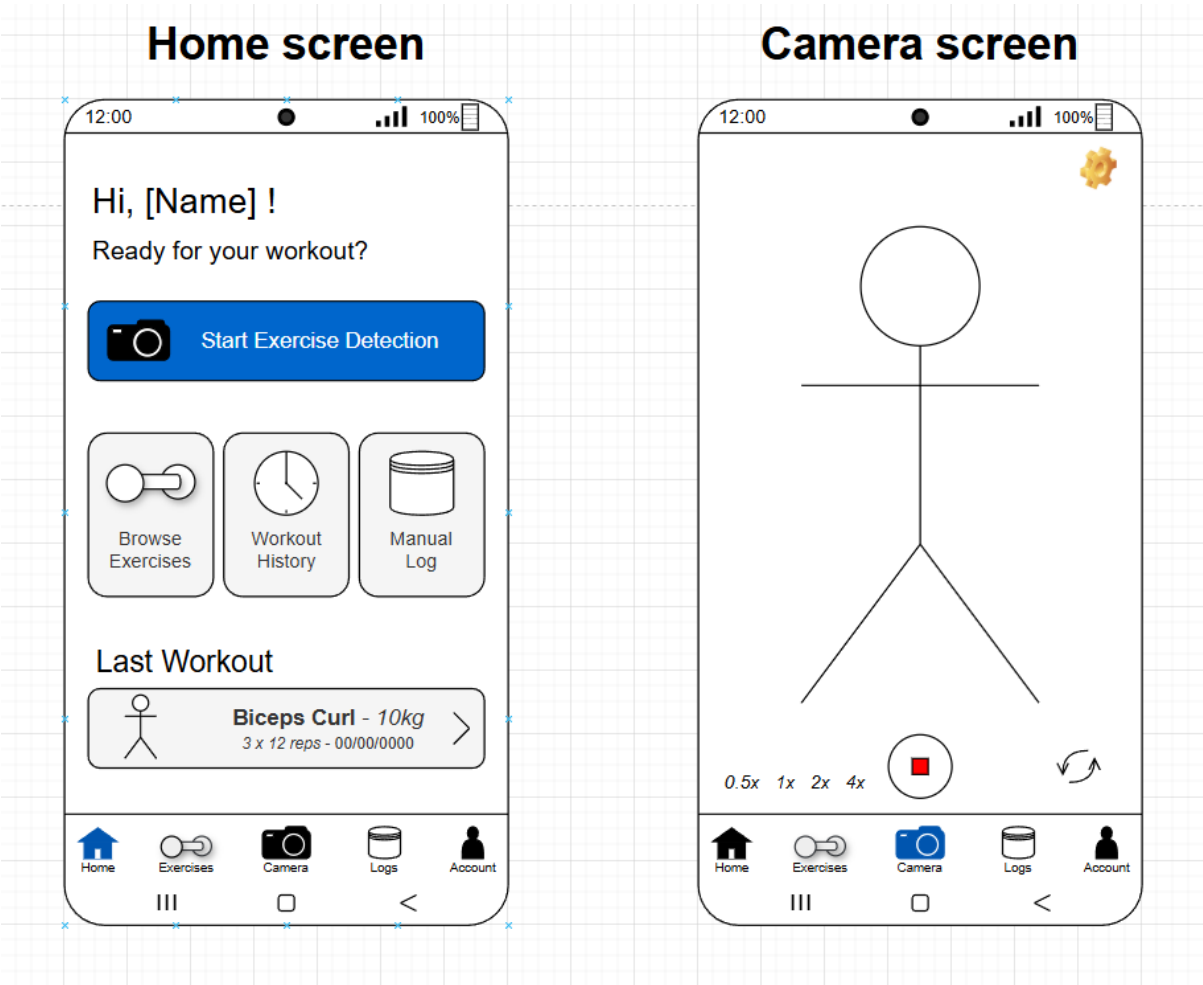


Figure 49: PoseFit AI Home, Camera screens Wireframe

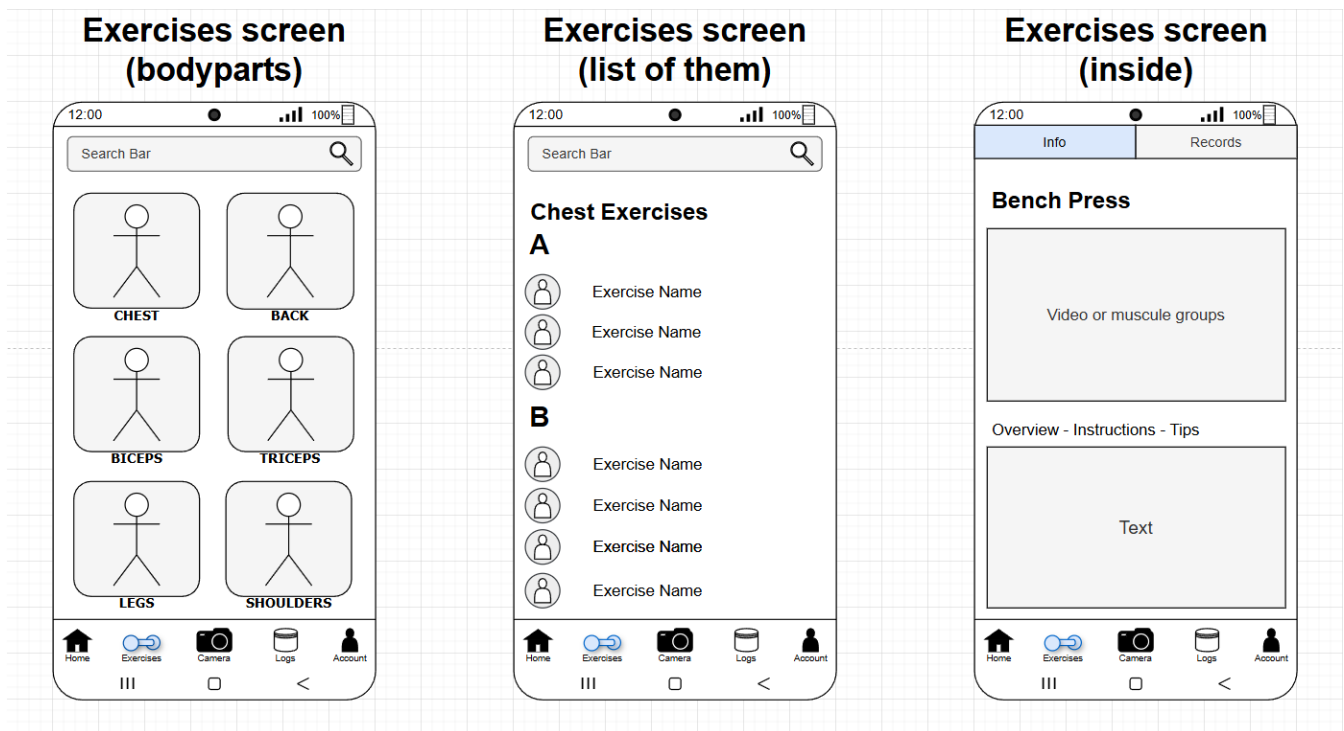


Figure 50: PoseFit AI Exercises screens Wireframe

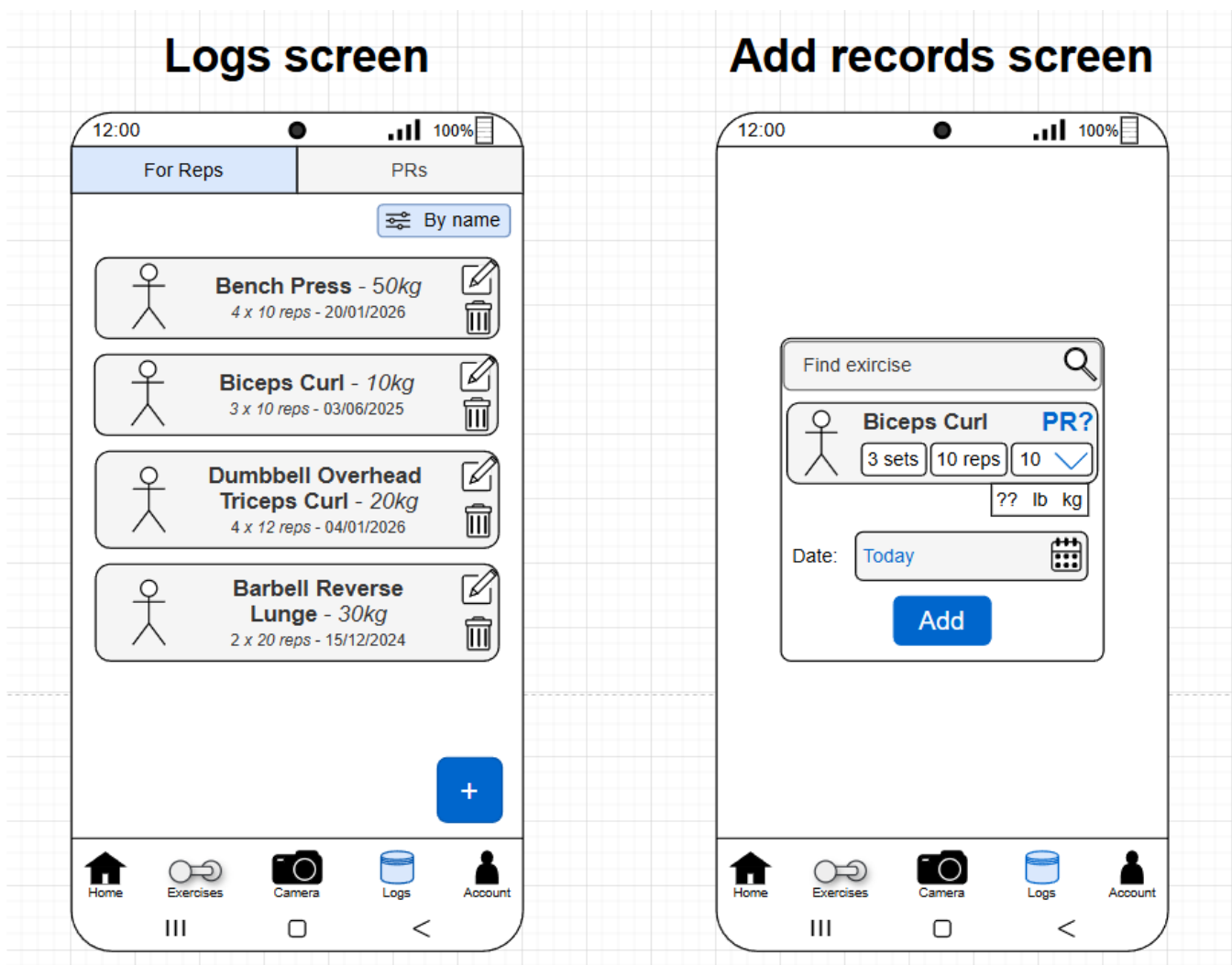
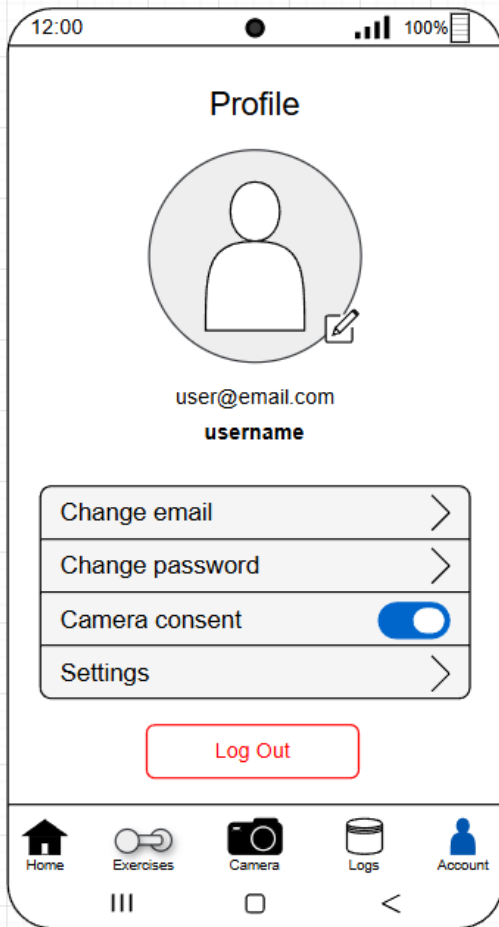


Figure 51: PoseFit AI Workout logs, Add records screens Wireframe

Account screen



Settings screen

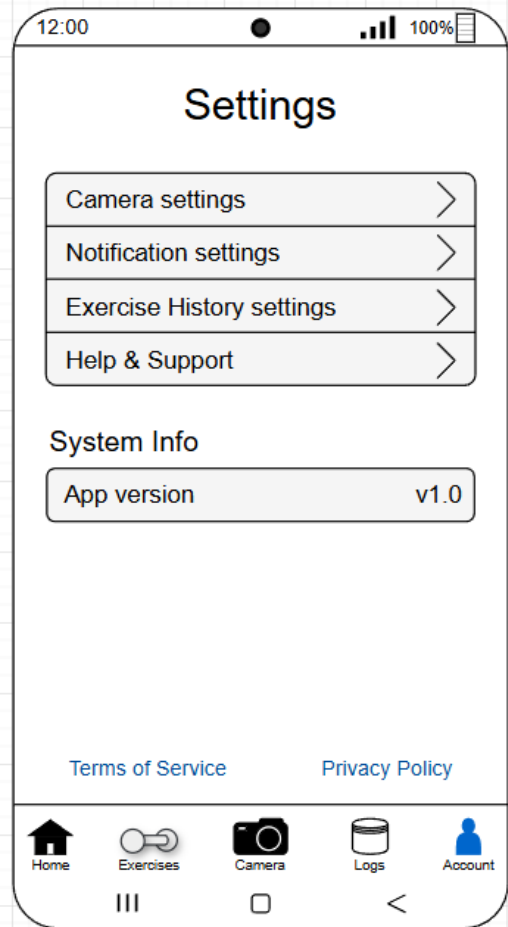


Figure 52: PoseFit AI Account, Settings screens Wireframe

User flow diagrams

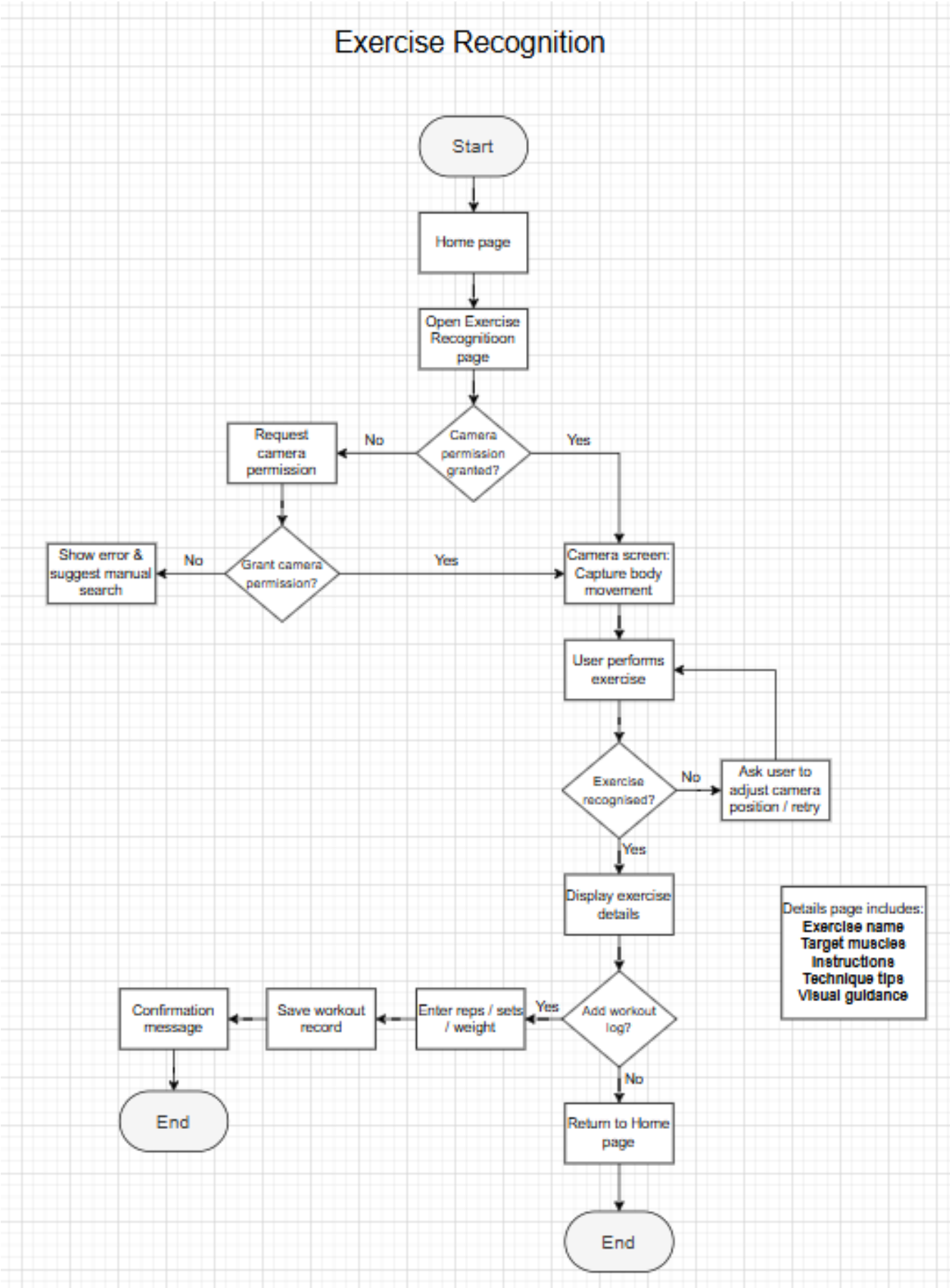


Figure 53: User Flow Diagram - Exercise Recognition

Manual Exercise Search

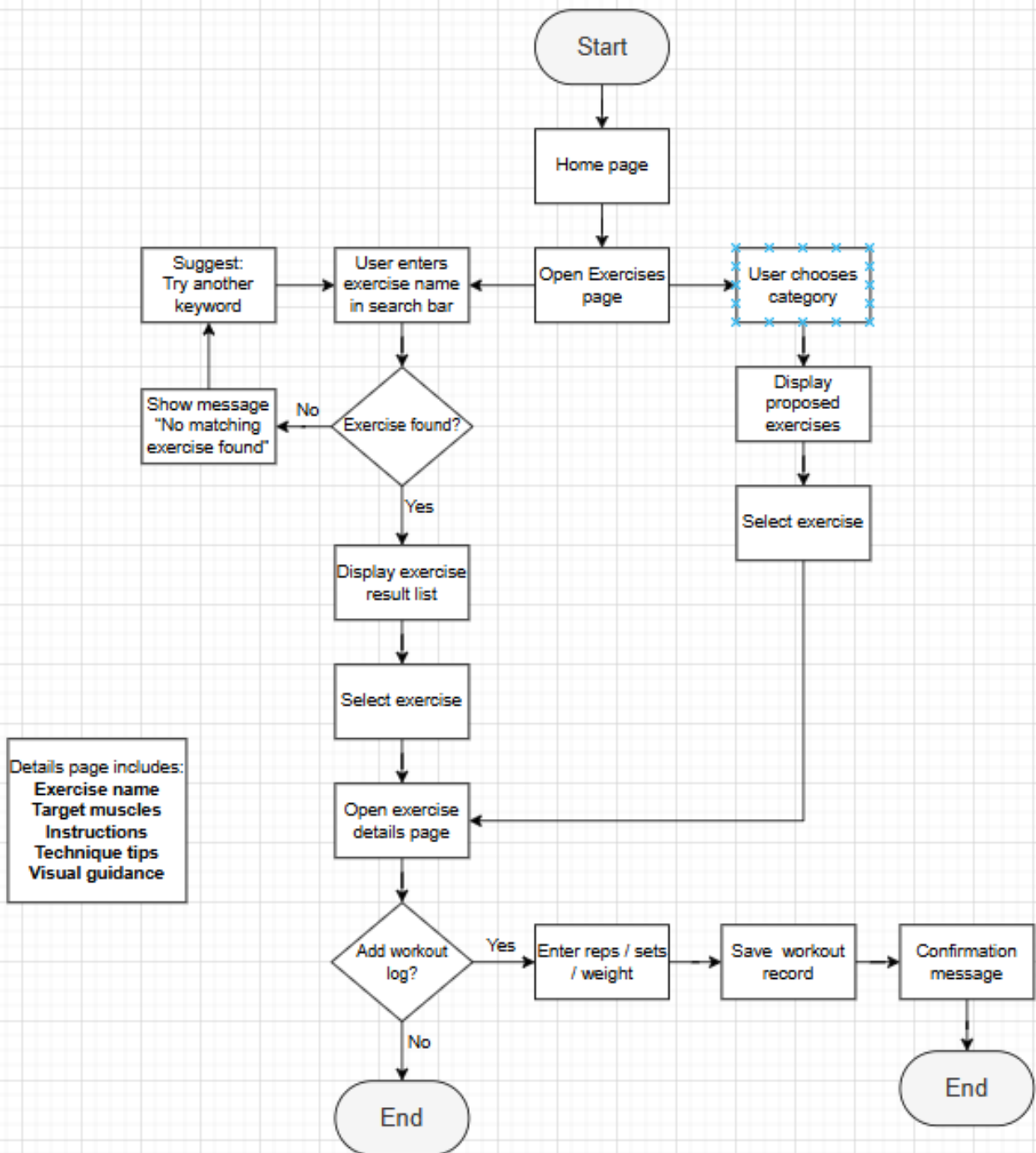


Figure 54: User Flow Diagram - Manual Exercise Search

Workout Logging / Progress Tracking

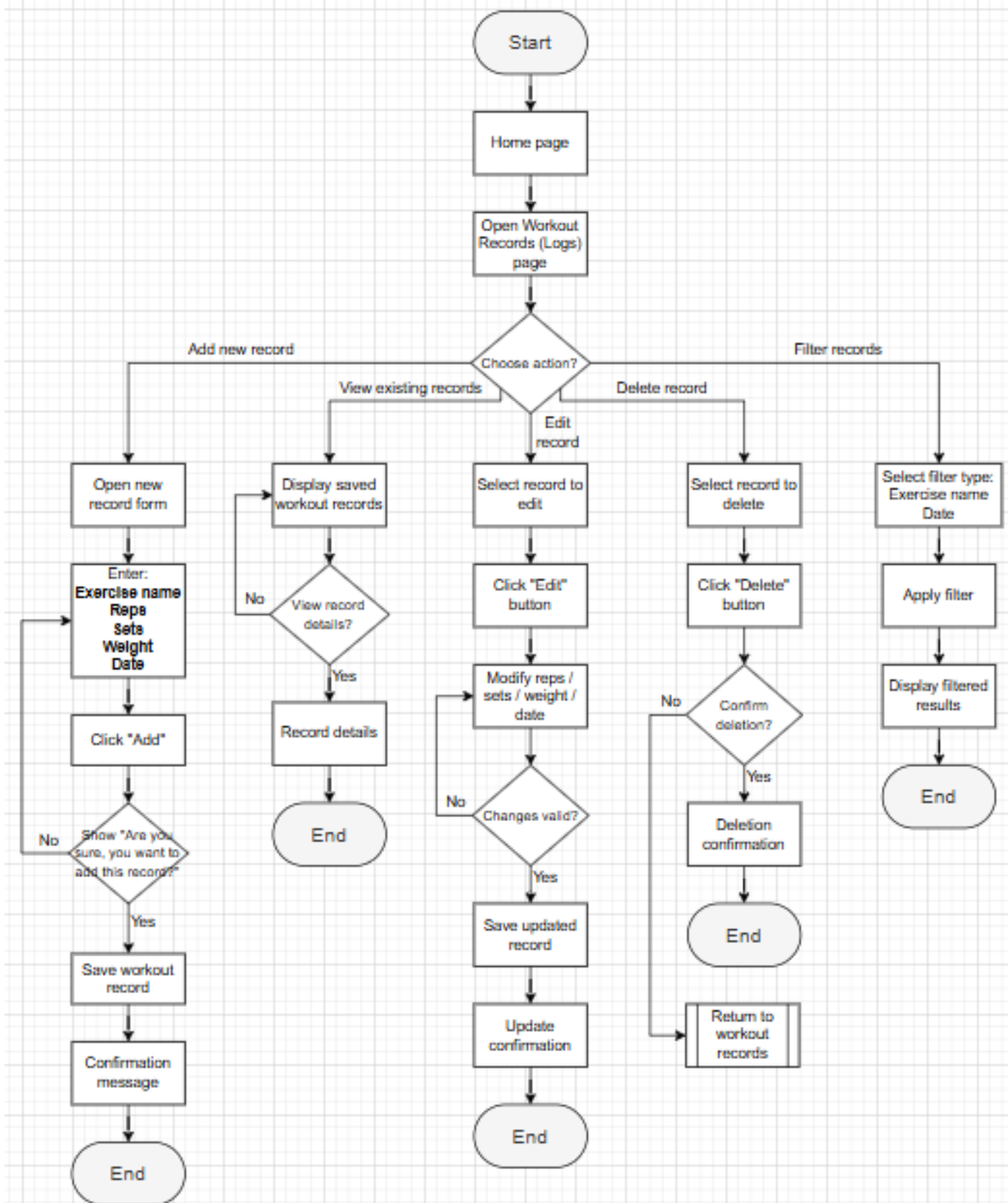


Figure 55: User Flow Diagram - Workout Logging

5. Solution design

In this section, the technical design of the PoseFit AI application is discussed, along with the planning of the overall system architecture from which the software components and data flow were derived before implementation started. The solution design section aims at showing the proposed system was designed to meet the aims and objectives set out earlier in the project. It also gives a clear idea of how the frontend application, backend services, machine learning components, database systems, and security mechanisms work together in the proposed architecture.

Solution design is a crucial area of the project as it transforms theory research and needs into a sensible technical solution. The design process enabled experimentation with various technologies, including Flutter, MediaPipe, TensorFlow, MySQL, and the back-end API services and how these could be combined into a single system. The design phase also afforded the opportunity to consider issues like usability, security, real-time processing, scalability, and maintainability, before development even started.

The system design was influential throughout the project for decisions on implementation and project development. The architecture served as a tool for the project organization and helped to structure the project into logical parts, which made the development process more structured and manageable. It also facilitated the early identification of technical challenges, especially in regard to real-time pose estimation, camera integration, the backend communication, and the secure handling of data. Additionally, the design process was carried out in such a way that the design implementation will be consistent with the initial project goals, legal and ethical issues addressed in the previous design stages and the user needs.

System Architecture

This section describes the overall system architecture adopted for the development of PoseFit AI application. The architecture diagram shows the relationships between the various system components, and how they all work together in order to provide the functionality of real-time exercise recognition, secure user authentication, workout logging, and communication with the database. The system is built using a single client-server architecture, Flutter mobile application, backend API services, a secure local storage, a machine learning element, and a MySQL database. The aim of this architecture was to show how various technologies and services interact along the application and to illustrate the overall architecture prior to implementation. The architecture design is also influenced by the important considerations identified during the project such as usability, real-time mobile processing, maintainability, security and data protection.

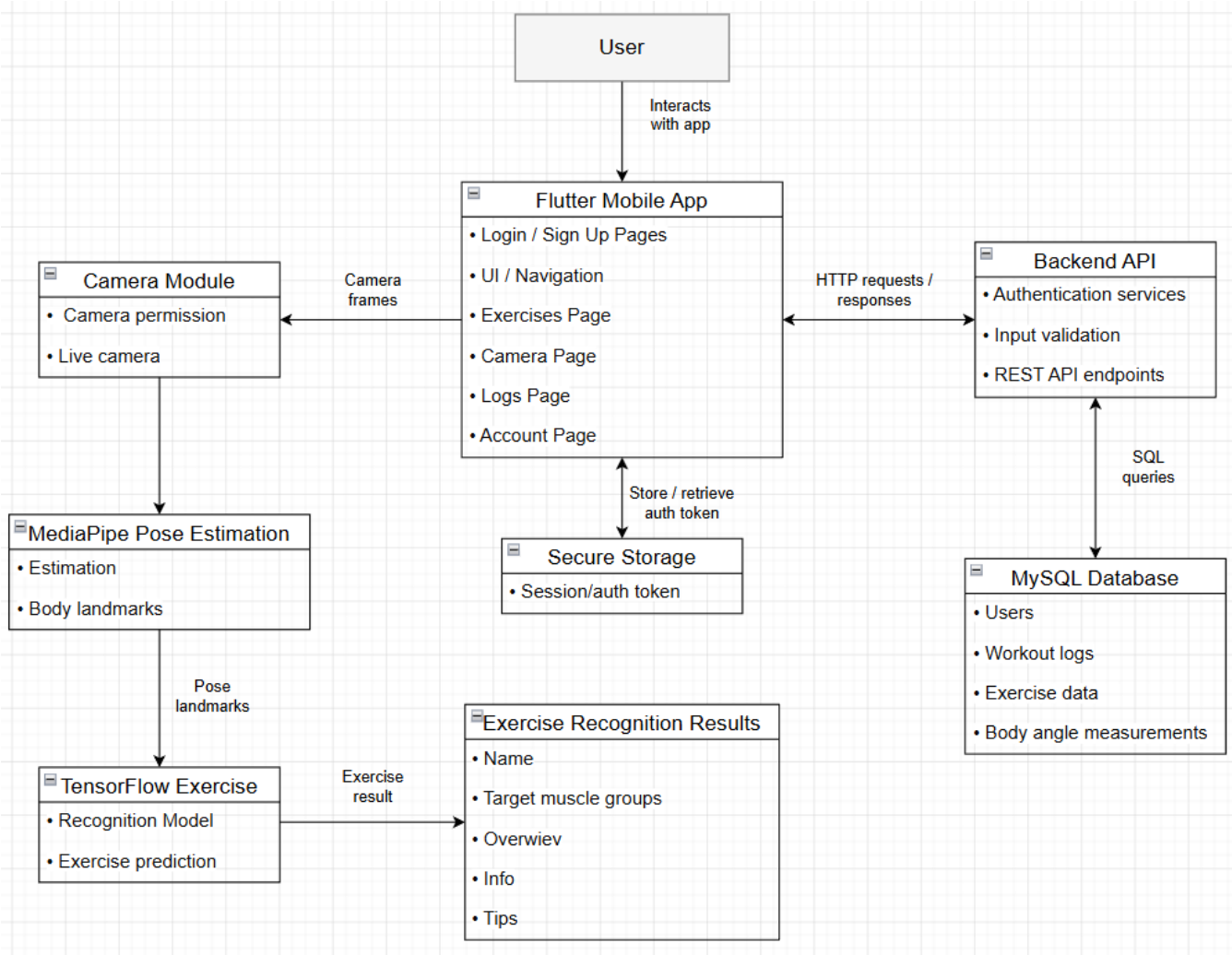


Figure 56: System Architecture Diagram

Flutter Mobile Application

Frontend of the system written in Flutter to enable cross-platform mobile application development from a single source of code. The main user interface and way to navigate all the screens are included in the Flutter app, such as login and sign-up pages, exercise pages, camera feature, workout logs, and account management screens. The reasons for choosing flutter are that it is flexible, has a robust mobile support, and it delivers a consistent user experience across Android and iOS devices.

Camera Module

The camera module provides access to the camera module of the smartphone and takes live camera frames for exercise recognition. The module is also responsible for runtime camera permissions, which means that users will have to give permission for the camera to be used before it is activated. The real time camera frames are fed into the pose estimation system for further processing.

MediaPipe Pose Estimation

In this example, Pose Estimation is used to detect human body landmarks in real-time with the mobile device camera using MediaPipe. The method analyzes the user's movements and outputs the body landmark co-ordinates (coordinates of key body joints and positions). MediaPipe was chosen because it offers pose estimation that is lightweight and efficient with real-time performance that is appropriate for mobile devices with relatively low hardware requirements.

TensorFlow Exercise Recognition

The pose landmark data from MediaPipe is utilised to aid the exercise recognition process using TensorFlow. The model estimates body landmark positions and angle measurements for a body in each of the model's data points to determine exercise movement patterns and the exercise being performed. The recognized exercise results are then shown back to the mobile application interface and presented to the user along with other information about the exercise.

Backend API

The back-end API serves as the communication channel between the mobile application built using Flutter and the MySQL database. The backend handles authentication services, request processing, validating the input, and the REST API endpoints. The front-end application and the back-end services communicate with each other via HTTP requests and responses.

MySQL Database

Non-sensitive application data, such as user accounts, workout logs, exercise information, and exercise angle data are stored in a MySQL database. This database can offer efficient data management, track workout history, and store user-specific data.

Secure Storage

Secure local storage is used to safely store session and authentication tokens on the user's device. Platform-specific secure storage methods keep sensitive authentication information secure and mitigate the security problems of using plain text authentication data. This allows for GDPR and data protection principles outlined above in the report.

Entity-Relationship Diagram (ERD)

This part introduces the Entity-Relationship Diagram (ERD) that was utilized to design the structure of the database for the PoseFit AI application. The diagram below shows the organization of the data in the relational database system and the relationships between the data in the different tables used in the system. The ERD aims to illustrate how structured data essential for authentication, workout logging, exercise information management and machine learning features processing are stored by the application and managed. The database design aimed to optimize queries, minimize data redundancy, and ensure data consistency using primary and foreign key relationships. The relational design also enables scalability and maintainability for future integration of further exercises, workout logs, and recognition feature. The relational design also facilitates future scalability and maintainability of the project in the integration of additional exercises, workout logs, and recognition feature data as they are developed.

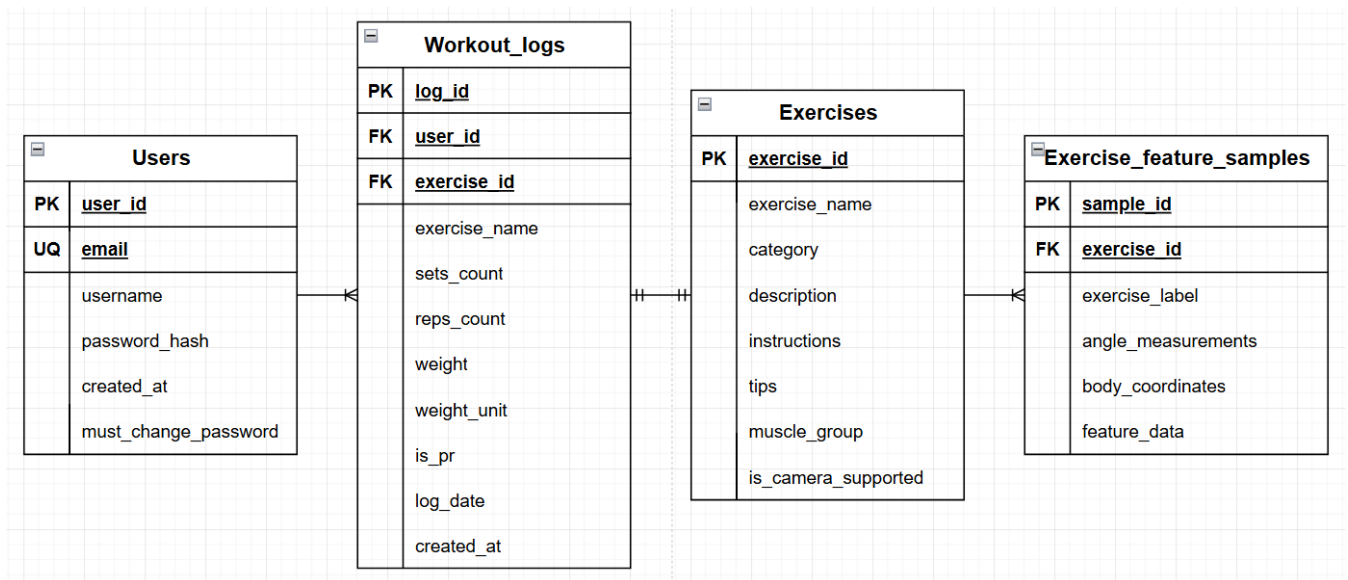


Figure 57: Entity-Relationship Diagram (ERD)

Users Table

The Users table contains information about accounts needed for authentication and user management. Individual users each have a unique user identifier and workout data is stored in other places in the database. The table also contains information about authentication, such as password hashes and password reset requirements.

Workout_logs Table

The Workout_logs table contains information regarding user workout history, such as exercise information, sets and reps, weight used, personal record (PR) indicators, and the date of the workout. The workout log is related to a user and exercise by foreign keys.

Exercises Table

The Exercises table contains details about supported exercises in the application, like exercise names, categories, descriptions, instructions, tips, and targeted muscle groups. This structure enables the reuse of exercise information in the entire application interface and exercise recognition system.

Exercise_feature_samples Table

The Exercise_feature_samples table contains the feature data that was processed during the exercise recognition. This includes the labels of the exercise performed, angle data, body

coordinate data, and other extracted feature data based on pose estimation. The table houses the machine learning parts utilized for exercise classification.

Activity diagram - Camera-Based Exercise Recognition Workflow

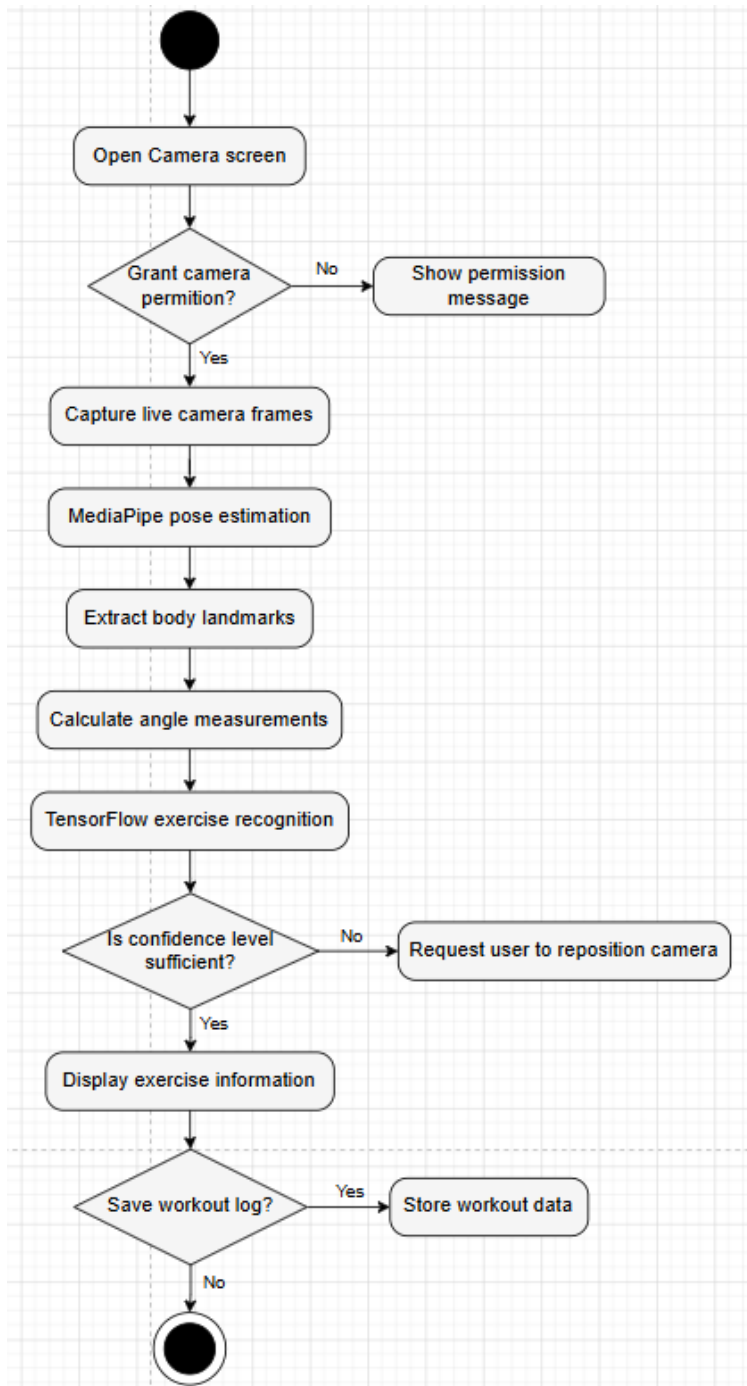


Figure 58: Activity Diagram - Camera-Based Exercise Recognition Workflow

This section describes the process of the activity workflow implemented in the PoseFit AI application for the exercise recognition functionality that uses the camera. This figure illustrates the steps a user would take to use the camera to recognise the exercise using the machine learning classification, including handling of permissions, capturing the image in real-time, estimating the pose from the image, extracting features from the image, using machine learning classification to determine the exercise, validating the confidence level, and logging the exercise. This workflow is meant to provide a general understanding of the interactions between the various parts of the system, as well as how the app handles user interaction and internal logic. Another key point in the workflow is that usability, reliability, and error handling were thought of from the outset of designing the exercise, as evidenced by the inclusion of permission validation and confidence checking steps before results of the exercise are presented to the user.

The operation starts with the user opening the camera screen in the app.

Firstly, it verifies if camera access has been granted. It verifies if the camera permission has been granted or not before it activates the live camera stream. If the user is denied permission, the user is notified with a permission message.

With the user's consent, the app records a live stream of images from the camera and passes them to the MediaPipe pose estimation framework to identify the body landmarks that correspond to the key joints and positions of the body. The landmark coordinates detected are then used to compute body angle measurements and additional feature data that are needed for exercise recognition.

This data gets fed into the TensorFlow exercise recognition model that analyses the motion and makes exercise predictions. The system then checks the prediction confidence level and if it is at or above that level, it shows the user information about the exercise.

When the confidence level is inadequate, the user is requested to adjust the camera to make the recognition more precise. If successful, the application will show information regarding the exercise, target muscle groups, instructions and tips.

The user can then optionally save workout log data such as sets, reps, weights and personal records to the database to be looked back on later.

6. Solution implementation

Initial Project Setup

The first phase of the PoseFit AI project centred on identifying an appropriate project idea, choosing suitable technologies and establishing a development environment for the realisation of this project. A couple of ideas were explored for the project in its early stages—desktop applications, websites, and mobile applications. Based on the analysis of personal interest, recent trends in technology and the potential application of the system, it was decided to create a mobile application focused on fitness and gym training.

The overall idea of the project came from the combination of the interest in gym classes and machine learning technologies learned in the machine learning module. Eventually, a new concept emerged, one that involved analyzing exercises, using mobile devices, and computer vision and machine learning to recognize exercises through a smart phone camera, which was the base for the development of a mobile app using AI. The aim was to provide a usable product that would facilitate users to identify exercises, learn the correct form, and to monitor workout progress in a more user-friendly and automated manner.

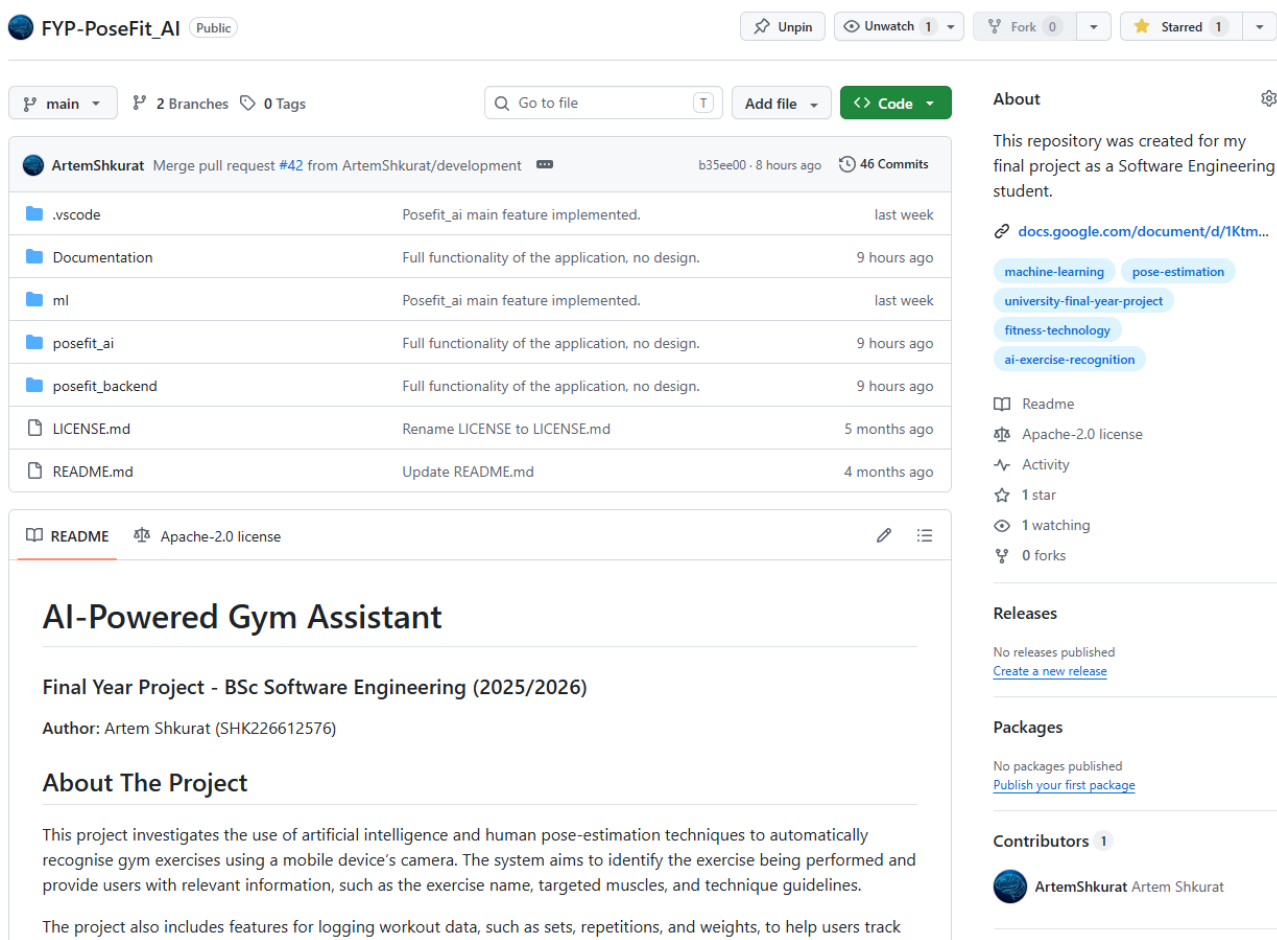


Figure 59: Flutter Application Navigation Structure

The decision to create a mobile-based solution led them to choose Flutter as their main development platform, as it is cross-platform, easy to learn, and has a fast development process. The “hot reload” feature was one of the major benefits of Flutter, as they could test the changes to the interface and implementation in real time during the development process. The Flutter development process also supported both Android and iOS platforms from one codebase, which minimized development complexity and ease up maintainability.

The main development environment used was Visual Studio Code. Development was supported by GitHub, where versions and project management were enabled, and where the progress of implementation, feature additions and debugging could be tracked. GitHub Projects and Kanban boards also used to structure tasks and track their progress in various stages of development.

In the initial phase of development, further research and literature survey were performed to find appropriate technologies, existing fitness applications and pose estimation frameworks that are applicable to the solution proposed. Existing applications and AI-based fitness systems were analyzed to understand their features, limitations and potential enhancements, all of which could impact the design and development of the PoseFit AI system.

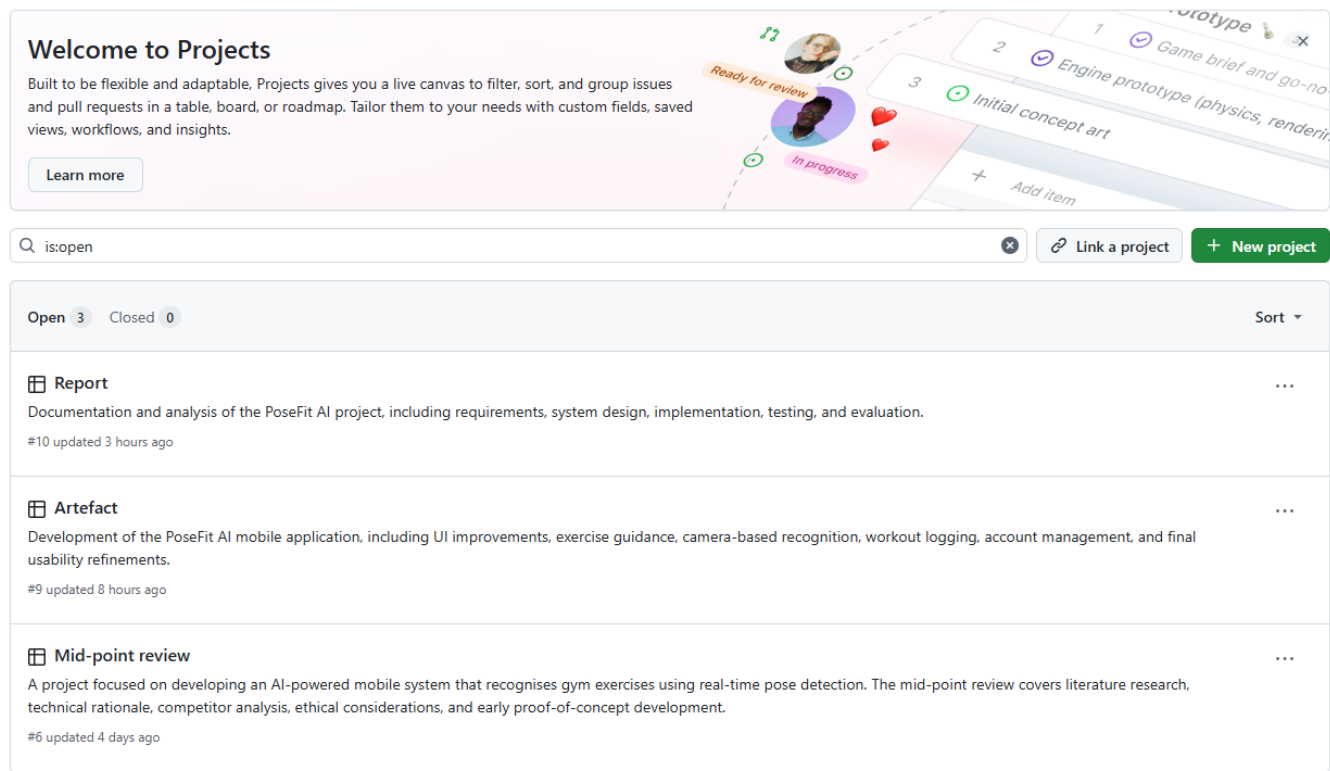


Figure 60: PoseFit_AI Kanban board Projects

Frontend Development

Throughout the project, Dart was used as the primary language, as Flutter is designed around Dart and makes use of its features. It was thought to be a good option because Dart is integrated with Flutter, has very good performance in the mobile world, and the syntax for cross-platform application development is simplified. Flutter and Dart are also both backed by Google, which offers long-term stability, tons of documentation, and a lot of support from the community. This also paid off in the development process, since the project utilized another Google technology, called MediaPipe, which was used for real-time pose estimation.

The main navigation structure and core application pages were created as the first steps in the frontend implementation of the PoseFit AI application. First five main pages were developed: Home Page, Exercises Page, Camera Page, Workout Logs Page, Account Page. These pages were the base of the user interface and the most important functions of the application.

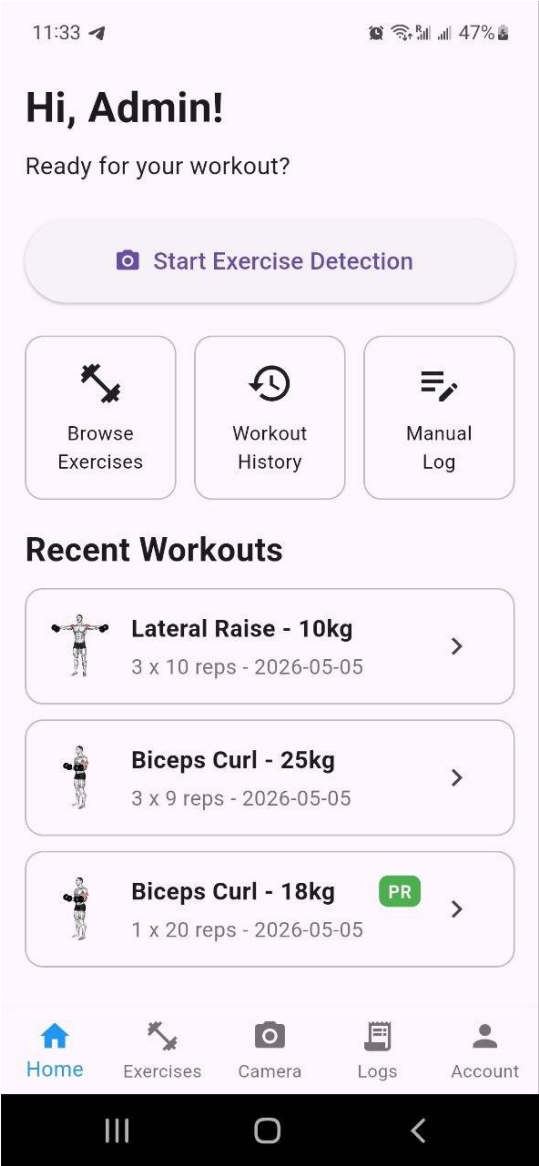


Figure 61: Application Home screen

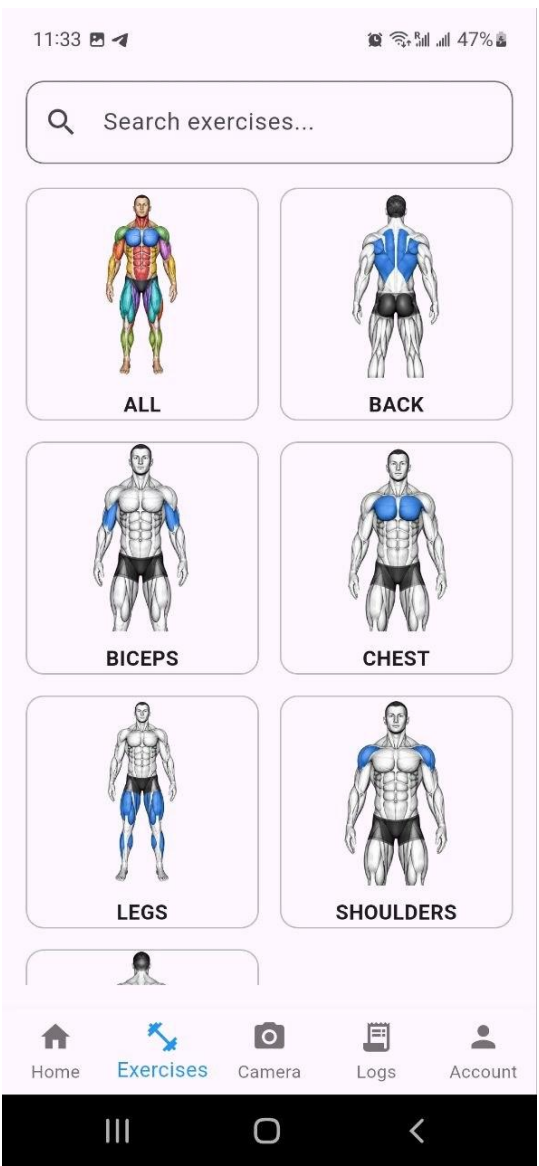


Figure 62: Application Exercises screen

Added a reusable bottom navigation bar widget for better navigation and usability. The navigation bar was shown at all times throughout the application and each page had an icon that could be used to rapidly navigate to that page. This made the overall user experience more pleasant by offering a familiar and easy to use mobile navigation system, while also eliminating duplicated navigation code across the project.

Because it was designed as a multi-user system, the authentication feature was also added when developing the frontend. The login and registration pages were developed so that the user can register under his/her personal login and password. Every user account was linked to a user-specific workout data and personalised application data. Input validation was also taken into account during implementation; this will increase the security aspect and prevent invalid account creation attempts.

User interface was refined in every stage of the frontend development process, to enhance usability, user navigation and uniformity among application screens. The hot reload feature in Flutter was an incredible boon to developers, enabling them to make changes to the interface and to the implementation directly without having to restart the app.



Figure 63: Application Camera screen

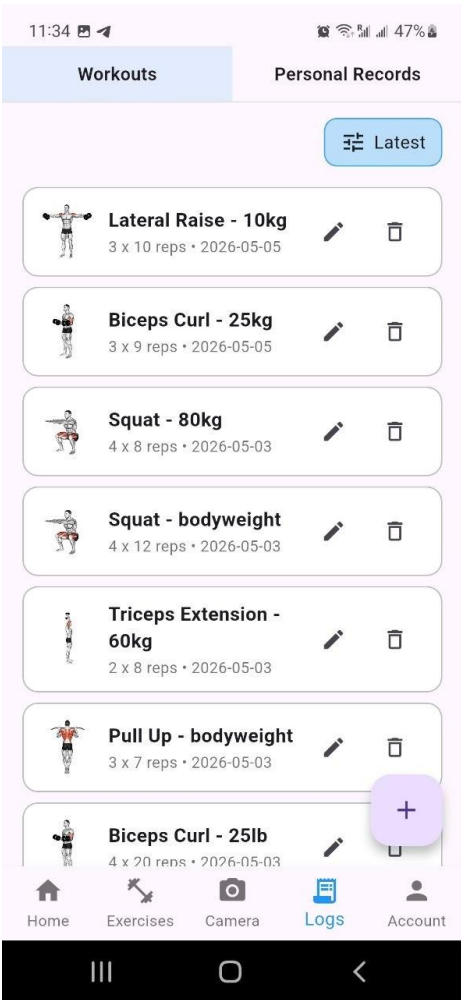


Figure 64: Application Workout Logs screen

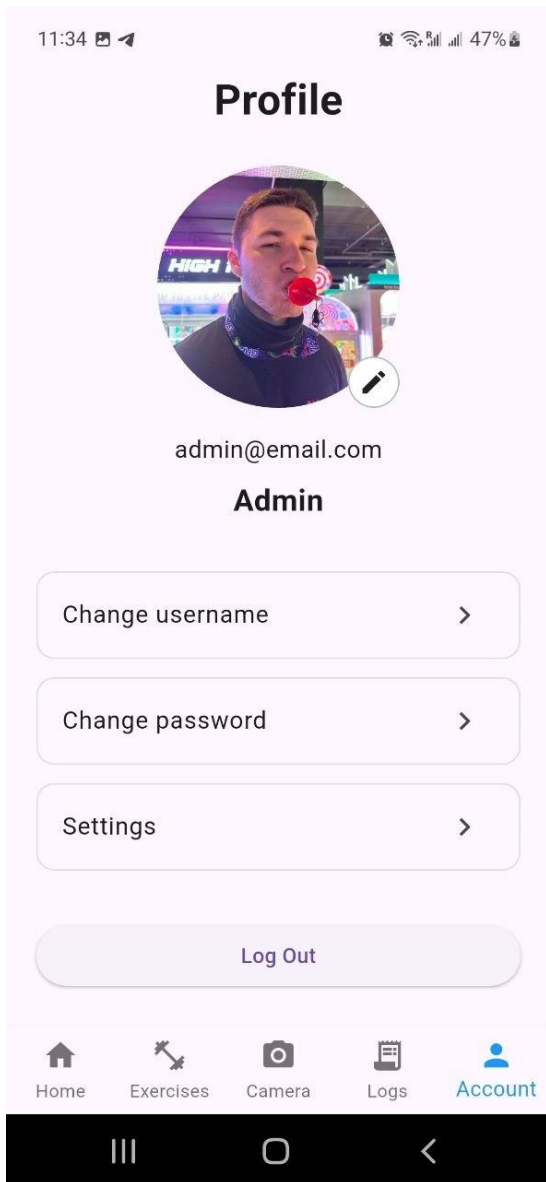


Figure 65: Application Account screen

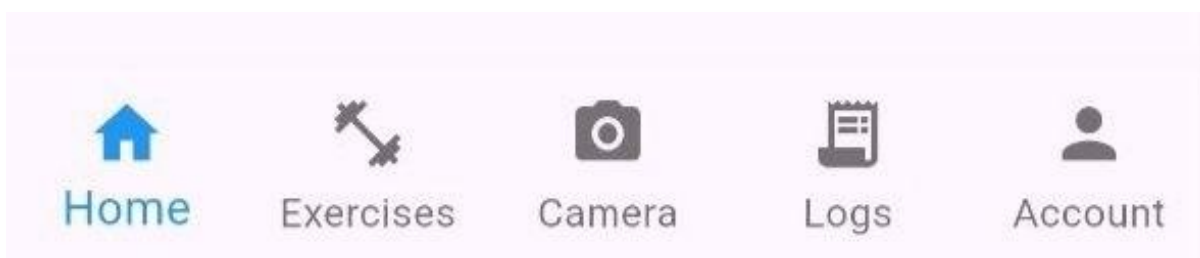


Figure 66: Application navigation bar

Backend Development

This led to the development of a separate backend system that enabled the mobile application on Flutter to support the authentication feature and the communication with the database. The backend was created in a separate Python file called `app.py`, which provided a communication

interface between the front end application and the MySQL database. This was done to split up the frontend and backend to organize the system architecture, making it easier to maintain, more organized and simpler to debug during development.

The backend server handled HTTP requests from the Flutter app, and provided suitable answers back to the user interface. The system was developed with authentication functionality, enabling users to register and log in to the system with their email address and password. The frontend application used an HTTP request to send the credentials entered by the user to the backend server when trying to log in.

The back end then worked on the requests, matched entered information with records in the MySQL database, and returned a response to the front end determining if the authentication was successful or not. When the credentials matched the account information stored for that application, the user logged in successfully into the application. The back end also validated other things when users logged in, like if their email already had an account when they signed up and/or if they had tried to log in with an invalid email address. This enhanced the reliability of the authentication mechanism and minimized the likelihood of storing inaccurate or duplicate user information in the database.

The Flutter application and the back end server were able to communicate with each other during development and testing using the same local Wi-Fi network connection. To help test the application on a real mobile device, the smartphone was first wired to the development laptop and then the connection was switched to the same local network for both devices. The backend server was then started on the laptop's local IP address and network port, with the ability for HTTP requests to be exchanged with the backend services in real-time between the mobile app and the backend server.

During the back-end development, various connectivity and debugging issues arose, such as changing IP addresses, communication between servers, and handling requests. These issues had to be tested and configured multiple times to ensure the stable communication between the frontend application, backend services, and database system.

```
PROBLEMS 27 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\artem\OneDrive - University of Roehampton\Final-Year Project\FYP-PoseFit_AI>
PS C:\Users\artem\OneDrive - University of Roehampton\Final-Year Project\FYP-PoseFit_AI> cd posefit_backend
PS C:\Users\artem\OneDrive - University of Roehampton\Final-Year Project\FYP-PoseFit_AI\posefit_backend> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.25.105.158:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 331-787-394
127.0.0.1 - - [08/May/2026 11:31:13] "GET /workout-logs?user_id=5 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:31:36] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:31:36] "GET /workout-logs?user_id=3 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:31:59] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:32:07] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:32:18] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:32:18] "GET /workout-logs?user_id=3 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:33:29] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:35:14] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:35:14] "GET /workout-logs?user_id=3 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:36:57] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:36:57] "GET /workout-logs?user_id=3 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:36:58] "GET /exercises HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 11:36:58] "GET /workout-logs?user_id=3 HTTP/1.1" 200 -
[]
```

Figure 67: app.py terminal launching

```
app.py 2 X main.dart
posefit_backend > app.py > forgot_password
122 |         cursor.close()
123 |
124 |
125 | @app.route('/login', methods=['POST'])
126 | def login():
127 |     data = request.get_json()
128 |
129 |     email = data.get('email')
130 |     password = data.get('password')
131 |
132 |     if not email or not password:
133 |         return jsonify({"success": False, "message": "Email and password are required."}), 400
134 |
135 |     cursor = get_db_cursor(dictionary=True)
136 |     cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
137 |     user = cursor.fetchone()
138 |
139 |     if not user:
140 |         return jsonify({"success": False, "message": "Invalid email or password."}), 401
141 |
142 |     stored_hash = user["password_hash"].encode('utf-8')
143 |
144 |     if not bcrypt.checkpw(password.encode('utf-8'), stored_hash):
145 |         return jsonify({"success": False, "message": "Invalid email or password."}), 401
146 |
147 |     return jsonify({
148 |         "success": True,
149 |         "message": "Login successful.",
150 |         "user": {
151 |             "id": user["id"],
152 |             "username": user["username"],
153 |             "email": user["email"],
154 |             "must_change_password": user["must_change_password"]
155 |         }
156 |     }), 200
157 |
```

Figure 68: part of the app.py code

Database Implementation

The decision to use MySQL relational database for primary data storage was driven by its proven reliability, scalability, and ability to handle structured data management, which was critical for the PoseFit AI application. MySQL has been widely used in both small-scale and large-scale software systems, which makes it an appropriate tool for the project, to manage user accounts, workout information, exercise data and machine learning feature samples. The database system was included in the back-end API to facilitate effective data storing, retrieving, and processing during the application operations.

Four primary SQL tables were created: `exercise_feature_samples`, `exercises`, `users`, and `workout_logs`. Every table was used to hold a set of application data necessary for the proper functioning of the system. The workouts table contained workout data like exercise names, reps, sets, weights, dates, and personal records, and the users table held information about the users, including their account details for authentication and user management. The exercises table provided the exercise information such as exercise name, description, instructions and target muscle exercise used across all application interface.

The `exercise_feature_samples` table is created to facilitate the machine learning and exercise recognition aspects of the project. The table stored the processed exercise features data, such as the body coordinate information, angle data, and exercise label that were extracted from the process of pose estimation and feature generation. In addition, CSV formatted data sets were also utilized in the experimentation and data preparation phase due to the ease of data storage and processing for machine learning training data sets prior to being integrated into the database system.

The communication between the mobile application and the MySQL database was managed through the backend API created using Python backend server. The backend was tasked with running SQL queries, checking the incoming requests, and sending back responses to the Flutter application. This enabled data regarding exercise and workout logs, account information, and other workout-related data to be saved and retrieved dynamically as the application is used.

During implementation, more attention was paid to the organisation of the database, maintainability and scalability in the future. The relational structure eliminated data duplication and ensured consistency by enforcing primary and foreign key constraints on the relationships between tables. This also provides the flexibility to include more exercises, users and workout data at subsequent development phases.

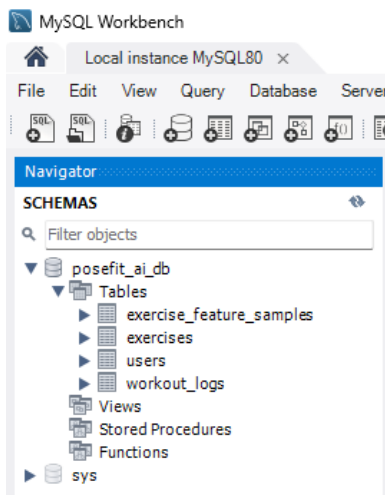


Figure 69: PoseFit AI database

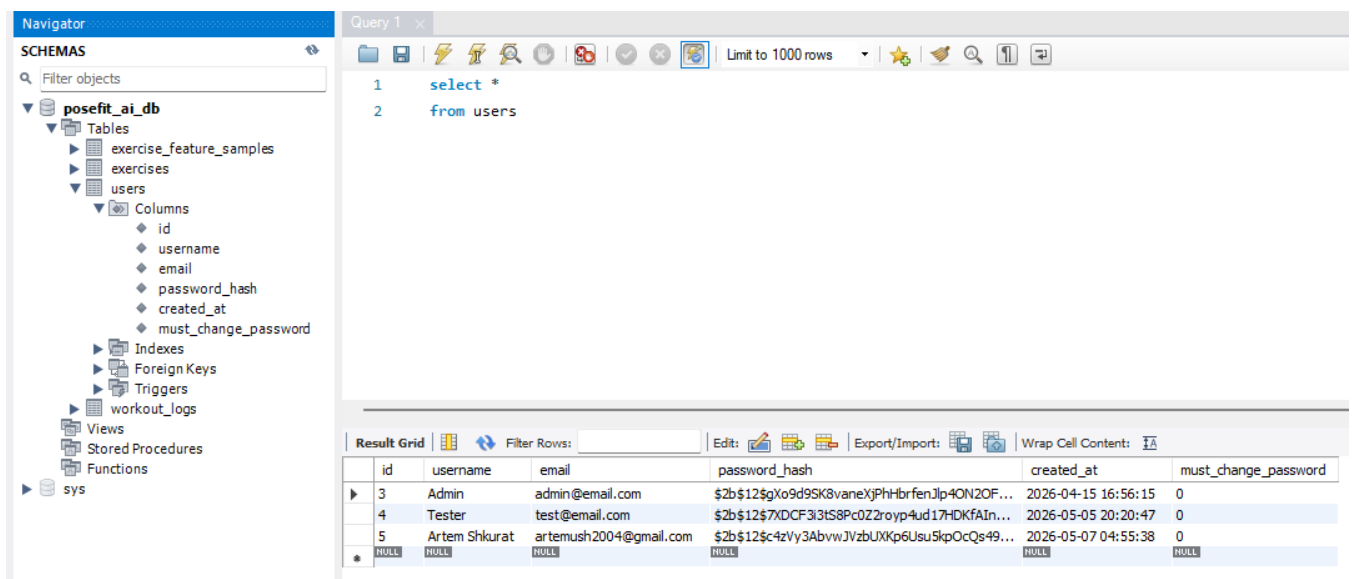


Figure 70: users database table and data

Navigator: SCHEMAS

Filter objects

posefit_ai_db

- Tables
 - exercise_feature_samples
 - exercises
 - users
 - workout_logs
- Columns
 - id
 - user_id
 - exercise_name
 - sets_count
 - reps_count
 - weight
 - weight_unit
 - is_pr
 - log_date
 - created_at
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

Table: workout_logs

Columns:

- id int AI PK
- user_id int
- exercise_name varchar(100)
- sets_count int
- reps_count int
- weight decimal(6,2)
- weight_unit varchar(10)
- is_pr tinyint(1)
- log_date date
- created_at timestamp

Query 1

```

1 select *
2 from workout_logs

```

Result Grid

id	user_id	exercise_name	sets_count	reps_count	weight	weight_unit	is_pr	log_date	created_at
3	3	Squads	6	12	50.00	kg	0	2026-05-01	2026-05-01 23:15:25
5	3	Bench Press	3	8	55.00	kg	0	2026-05-02	2026-05-02 20:51:04
7	3	Squat	4	12	0.00	bw	0	2026-05-03	2026-05-03 01:31:18
8	3	Lateral Raise	4	12	50.00	lb	0	2026-05-03	2026-05-03 01:34:55
9	3	Biceps Curl	4	20	25.00	lb	0	2026-05-03	2026-05-03 02:02:25
11	3	Squat	1	1	0.00	bw	1	2026-05-03	2026-05-03 02:06:23
12	3	Pull Up	3	7	0.00	bw	0	2026-05-03	2026-05-03 17:27:32
13	3	Triceps Extension	2	8	60.00	kg	0	2026-05-03	2026-05-03 21:28:03
14	3	Biceps Curl	1	10	20.00	kg	1	2026-05-03	2026-05-03 21:29:34
15	3	Squat	4	12	0.00	bw	0	2026-05-03	2026-05-03 21:31:32
16	3	Squat	4	8	80.00	kg	0	2026-05-03	2026-05-03 21:32:10
17	3	Squat	4	20	0.00	bw	1	2026-05-03	2026-05-03 21:33:06
18	3	Biceps Curl	1	20	18.00	kg	1	2026-05-05	2026-05-05 18:17:23
19	3	Biceps Curl	3	9	25.00	kg	0	2026-05-05	2026-05-05 18:26:29
20	3	Lateral Raise	3	10	10.00	kg	0	2026-05-05	2026-05-05 18:28:08
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

workout_logs2 x

Figure 71: workout_logs database table and data

Navigator: SCHEMAS

Filter objects

posefit_ai_db

- Tables
 - exercise_feature_samples
 - exercises
 - users
 - workout_logs
- Columns
 - id
 - user_id
 - exercise_name
 - sets_count
 - reps_count
 - weight
 - weight_unit
 - is_pr
 - log_date
 - created_at
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

Table: exercise_feature_samples

Columns:

- id int AI PK
- exercise_label varchar(100)
- features json
- left_elbow_angle double
- right_elbow_angle double
- left_knee_angle double
- right_knee_angle double
- left_hip_angle double
- right_hip_angle double

Query 1

```

1 select *
2 from exercise_feature_samples

```

Result Grid

id	exercise_label	features
3383	Lateral Raise	("leftHipAngle": 168.52135343551734, "leftKneeAngle": 160.77739790773677, "rightHipAngle": 158.86156734671442, "leftElbowAngle": 157.17993898430757, "rightKneeAngle": 179.55398413878336, "rightElbowAngle": 156.100531718449...
3384	Lateral Raise	("leftHipAngle": 165.94121782180184, "leftKneeAngle": 163.6152601209705, "rightHipAngle": 164.31454422554836, "leftElbowAngle": 158.674637880325, "rightKneeAngle": 171.32081195150363, "rightElbowAngle": 173.26719972141117...
3385	Lateral Raise	("leftHipAngle": 164.06502500125998, "leftKneeAngle": 179.88360768694312, "rightHipAngle": 169.77816250291136, "leftElbowAngle": 175.2649210654742, "rightKneeAngle": 176.1252924877206, "rightElbowAngle": 161.80386630042798...
3386	Lateral Raise	("leftHipAngle": 164.90527836257365, "leftKneeAngle": 175.34253083310426, "rightHipAngle": 171.81913995442116, "leftElbowAngle": 178.1331444884471, "rightKneeAngle": 177.2828975176578, "rightElbowAngle": 175.91412135128604...
3387	Lateral Raise	("leftHipAngle": 168.1662677905156, "leftKneeAngle": 177.7883373011441, "rightHipAngle": 169.80661380813173, "leftElbowAngle": 175.03323520723406, "rightKneeAngle": 177.085783348962, "rightElbowAngle": 179.46746569596738...
3388	Lateral Raise	("leftHipAngle": 167.3148625071269, "leftKneeAngle": 175.77091386634083, "rightHipAngle": 168.2897754188658, "leftElbowAngle": 176.23474089973223, "rightKneeAngle": 179.11925312168825, "rightElbowAngle": 179.34150466819133...
3389	Lateral Raise	("leftHipAngle": 163.23934279161116, "leftKneeAngle": 175.79902522930743, "rightHipAngle": 169.7967837711869, "leftElbowAngle": 176.68241535362867, "rightKneeAngle": 169.11136262215294, "rightElbowAngle": 167.8462885578126...
3390	Lateral Raise	("leftHipAngle": 165.29047605239066, "leftKneeAngle": 176.23134711784456, "rightHipAngle": 161.06226139945858, "leftElbowAngle": 176.53710179246465, "rightKneeAngle": 177.34930388080726, "rightElbowAngle": 167.786684825851...
3391	Lateral Raise	("leftHipAngle": 163.09671225949575, "leftKneeAngle": 178.68051392415785, "rightHipAngle": 165.517938751998, "leftElbowAngle": 168.81787810321543, "rightKneeAngle": 177.8209239497363, "rightElbowAngle": 174.5350043360784...
3392	Lateral Raise	("leftHipAngle": 159.13494012047906, "leftKneeAngle": 179.17181948794018, "rightHipAngle": 161.42297704627063, "leftElbowAngle": 170.2095810160984, "rightKneeAngle": 167.13702078306005, "rightElbowAngle": 169.2123349539298...
3393	Lateral Raise	("leftHipAngle": 162.7813579328682, "leftKneeAngle": 173.9466977826136, "rightHipAngle": 159.0376469881046, "leftElbowAngle": 147.15236970908296, "rightKneeAngle": 167.89582458560173, "rightElbowAngle": 119.5099228357796, "l...
3394	Lateral Raise	("leftHipAngle": 163.7268907653796, "leftKneeAngle": 175.09852123077923, "rightHipAngle": 162.4729250828954, "leftElbowAngle": 177.04763865144365, "rightKneeAngle": 175.29953055541768, "rightElbowAngle": 54.36932927931626, "l...
3395	Lateral Raise	("leftHipAngle": 160.89592875663747, "leftKneeAngle": 177.97018774278159, "rightHipAngle": 162.6395252178386, "leftElbowAngle": 173.62617159683265, "rightKneeAngle": 174.38233493819433, "rightElbowAngle": 63.35342845116361, "l...
3396	Lateral Raise	("leftHipAngle": 159.35849747921202, "leftKneeAngle": 178.9961700346804, "rightHipAngle": 162.73655702061295, "leftElbowAngle": 69.87949236175211, "rightKneeAngle": 171.78675025479863, "rightElbowAngle": 57.21187381423874, "l...
3397	Lateral Raise	("leftHipAngle": 159.6702526787966, "leftKneeAngle": 178.52378879950268, "rightHipAngle": 164.04640305988906, "leftElbowAngle": 159.82215853370798, "rightKneeAngle": 172.41575650265395, "rightElbowAngle": 162.048922325711...
3398	Lateral Raise	("leftHipAngle": 154.14117983897316, "leftKneeAngle": 178.0550528076836, "rightHipAngle": 165.4267435922201, "leftElbowAngle": 145.46404861291, "rightKneeAngle": 169.86468416270228, "rightElbowAngle": 156.51174993489724, "l...
3399	Lateral Raise	("leftHipAngle": 157.32150810992184, "leftKneeAngle": 176.15668235124798, "rightHipAngle": 164.8696147695884, "leftElbowAngle": 149.2397966530753, "rightKneeAngle": 175.60928108792558, "rightElbowAngle": 158.3121650410785, "l...
1400	Lateral Raise	("leftHipAngle": 155.6191956001092, "leftKneeAngle": 174.876301988997508, "rightHipAngle": 164.53012188555235, "leftElbowAngle": 176.00600977648792, "rightKneeAngle": 170.7524076679925, "rightElbowAngle": 157.99416551980076...

exercise_feature_samples 3 x

Figure 72: exercise_features_samples database table and data

Camera Integration and Permissions

The camera feature was one of the most significant and intricate features of the PoseFit AI app, as it was the backbone of the real-time exercise recognition system. One major part of the development involved the installation, testing and fine-tuning of the camera system to ensure that steady real-time processing and smooth user interactions are possible.

The first part of the implementation went towards providing access to the device camera from the Flutter app. As part of this, a new flutter camera dependency was added to the project to enable accessing the mobile device camera directly from the application. In the early stages of development, the implementation mainly focused on taking static images to ensure that the camera worked and could be used with the future pose estimation processing.

Once the basic camera functionality was successfully added, the system was extended to allow for streaming of the camera in real-time. The app took live camera frames instead of just a single image, a feature it needed for the MediaPipe pose estimation and exercise recognition functions. For this stage, extensive testing was carried out to assess the performance of the camera, the stability of the camera, the behaviour of the camera in the frame, and compatibility with the pose estimation system, prior to adding more AI features.

Important implementation considerations were the user privacy and how permissions are handled, as the application is highly dependent on camera access. The requests for camera permission were added at runtime to ensure that users are aware and specifically agree to this permission before the camera is on. The first time the camera page is opened the system will query the user and ask if the camera should be allowed. Other camera settings and permissions management were also embedded within the app to make it easy for users to control camera access if it's later denied from the device settings.

During the implementation process several technical issues arose in working with the camera system. The most noteworthy problem was that of app crashes due to resolution changes during runtime of the camera. When trying to dynamically change camera resolution directly from the application interface, more issues were found. This was a subject that needed to be continually debugged and tested to increase the stability of the camera and ensure a consistent behaviour in real-time processing. More advanced development stages also added more camera functionality such as camera switching and zoom controls, making it more usable and flexible when recognizing exercises from the camera.

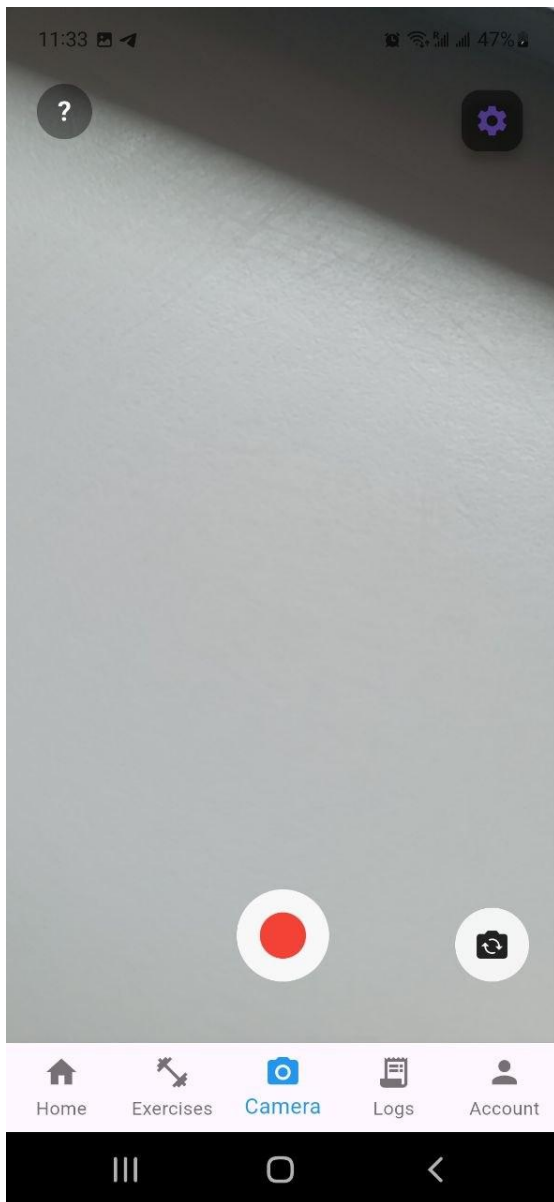


Figure 73: Application Camera screen



Figure 74: Camera Permission Interface

Pose Estimation and Exercise Recognition

Pose estimation and recognition of exercises was the most technically challenging part of the project, and one of the most important features of the PoseFit AI application. Once the real-time camera system was successfully implemented, the video stream from the camera was integrated with MediaPipe framework to detect body landmarks in real-time. MediaPipe was responsible for analysing live camera frames and placing landmarks across key body joints and positions. All in all, the framework offered some 33 body landmarks, spread across the main areas of the body, such as the shoulders, elbows, wrists, hips, knees, ankles, heels and facial reference points. The movement analysis system, which was applied across the project, was based on these landmarks.

After successfully detecting the body landmarks, TensorFlow was integrated into the system for exercise recognition using the movement data. TensorFlow analysed the body landmark coordinates generated by MediaPipe and calculated angle measurements between key body joints. These angle calculations enabled the system to identify the movement patterns during the performance of the exercises and how the body position changed over time for each exercise.

Once MediaPipe and TensorFlow worked together well, the camera system was scaled to include data collection and storage of exercise samples straight to the DB. For this phase of development the movements performed while moving in front of the camera were captured, processed and saved as training samples in the `exercise_feature_samples` table. First samples were taken for three activities to build a small training set for experimentation with the machine learning.

Data collection and preparation were among the major difficulties that were faced at this stage. To enhance the recognition accuracy and mitigate the model bias, exercise samples were required to be collected from different camera angles and in different exercise motion. Gathering enough quality training data was both time consuming and physically demanding as it was necessary to repeat each exercise multiple times in various ways to enhance the generalisation of the model.

There were several steps involving data pre-processing that had to be performed before the machine learning model could be trained. Firstly, the feature data are exported from the data base into a readable dataset suitable for machine learning processing. Further preprocessing was then performed to convert the raw landmarks and angle measurement data into structured feature vectors suitable for training the TensorFlow model. Finally, a dedicated training process was developed in order to train the exercise recognition model with the prepared dataset.

Once the model had been trained successfully, further functionality was added to enable the system to be able to determine prediction confidence levels while it was being used for exercise recognition in real-time. To minimize false classification and enhance the reliability of the system, thresholds were established for confidence. If the system had not obtained the desired level of confidence from the model, it might ask the person to move the camera again, or to move it in a different direction, to increase the accuracy of recognition.

The final implementation enabled the application to recognize exercises by using the smartphone camera to capture the body movement pattern of the exercise. Upon successful

recognition of an exercise, the system showed the user exercise-related information pages (e.g., the name of the exercise, the muscle groups it targets, instructions and more exercise details).

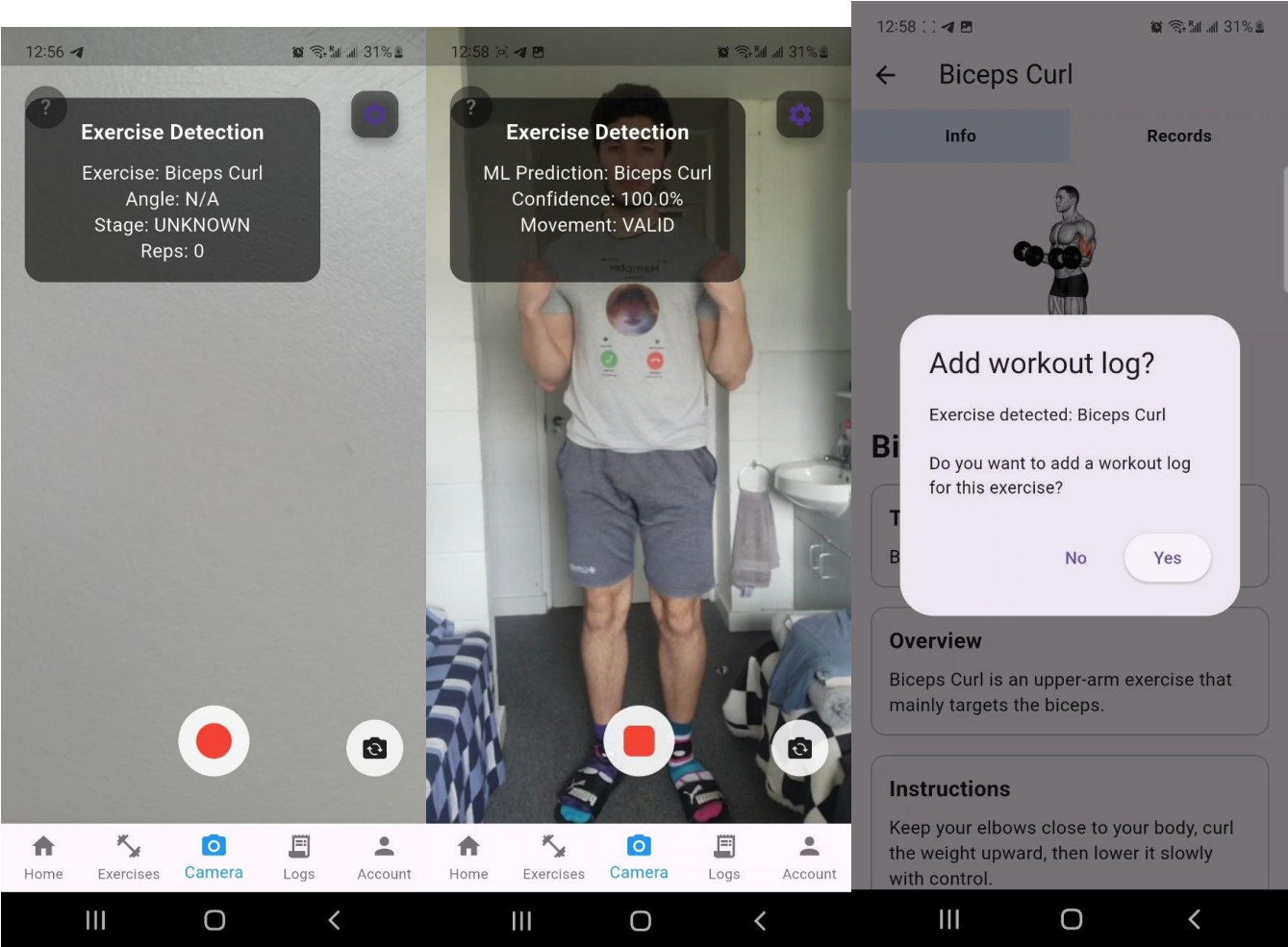


Figure 75: Real-Time Exercise Detection Camera Interface

Figure 76: Successful Exercise Recognition Using MediaPipe and TensorFlow

Figure 77: Exercises Info Page and Workout Log Confirmation After Exercise Recognition

Query 1

```

1 select *
2 from exercise_feature_sampled

```

Result Grid

id	exercise_label	features
3383	Lateral Raise	("leftHipAngle": 168.52135343551734, "leftKneeAngle": 160.77739790773677, "rightHipAngle": 158.86156734671442, "leftElbowAngle": 157.17993898430757, "rightKneeAngle": 179.55398413878336, "rightElbowAngle": 156.100531718449...
3384	Lateral Raise	("leftHipAngle": 165.94121782180184, "leftKneeAngle": 163.61526012098705, "rightHipAngle": 164.31454422554836, "leftElbowAngle": 158.674637880325, "rightKneeAngle": 171.32081195150363, "rightElbowAngle": 173.26719972141117...
3385	Lateral Raise	("leftHipAngle": 164.06502500125998, "leftKneeAngle": 179.88360768694312, "rightHipAngle": 169.77816250291136, "leftElbowAngle": 175.2649210654742, "rightKneeAngle": 176.1252924877206, "rightElbowAngle": 161.80386630042798...
3386	Lateral Raise	("leftHipAngle": 164.90527836257365, "leftKneeAngle": 175.34253083310426, "rightHipAngle": 171.81913995442116, "leftElbowAngle": 178.1331444684471, "rightKneeAngle": 177.2828975176578, "rightElbowAngle": 175.91412135128604...
3387	Lateral Raise	("leftHipAngle": 168.10625677905156, "leftKneeAngle": 177.7883337501441, "rightHipAngle": 169.80863580813173, "leftElbowAngle": 175.03332520733406, "rightKneeAngle": 177.085783348962, "rightElbowAngle": 179.46746609506738, ...
3388	Lateral Raise	("leftHipAngle": 167.3148625071269, "leftKneeAngle": 175.77091386634083, "rightHipAngle": 168.2897754188658, "leftElbowAngle": 176.23474089973223, "rightKneeAngle": 179.11925312168825, "rightElbowAngle": 179.34150466819133...
3389	Lateral Raise	("leftHipAngle": 163.23934279161116, "leftKneeAngle": 175.79902522930743, "rightHipAngle": 169.7967837711869, "leftElbowAngle": 176.68241535362867, "rightKneeAngle": 169.11136262215394, "rightElbowAngle": 167.8462885578126...
3390	Lateral Raise	("leftHipAngle": 165.29047605239066, "leftKneeAngle": 176.23134711784456, "rightHipAngle": 161.06226139945858, "leftElbowAngle": 176.53710179246465, "rightKneeAngle": 177.34930388008726, "rightElbowAngle": 167.786684828851...
3391	Lateral Raise	("leftHipAngle": 163.09671225949575, "leftKneeAngle": 178.68051392415785, "rightHipAngle": 165.51793897951998, "leftElbowAngle": 168.81787810321543, "rightKneeAngle": 177.8209239497363, "rightElbowAngle": 174.5350043360784...
3392	Lateral Raise	("leftHipAngle": 159.13949012047906, "leftKneeAngle": 179.17181948794018, "rightHipAngle": 161.42297704627063, "leftElbowAngle": 170.2095810160984, "rightKneeAngle": 167.13702078306005, "rightElbowAngle": 169.2123349539298...
3393	Lateral Raise	("leftHipAngle": 162.7813579328682, "leftKneeAngle": 173.9466977628136, "rightHipAngle": 159.0376469881046, "leftElbowAngle": 147.15236970908296, "rightKneeAngle": 167.89582458560173, "rightElbowAngle": 119.5099228357796, "1...
3394	Lateral Raise	("leftHipAngle": 163.7268907653796, "leftKneeAngle": 175.09852123077923, "rightHipAngle": 162.4729250828954, "leftElbowAngle": 170.04763885144365, "rightKneeAngle": 175.29953055541768, "rightElbowAngle": 54.36932927931626, "1...
3395	Lateral Raise	("leftHipAngle": 160.85952875663747, "leftKneeAngle": 177.97018774278158, "rightHipAngle": 162.6395252178386, "leftElbowAngle": 73.62617159683265, "rightKneeAngle": 174.3823493819433, "rightElbowAngle": 63.35342845116361, ...
3396	Lateral Raise	("leftHipAngle": 159.35849747921202, "leftKneeAngle": 178.9961700346804, "rightHipAngle": 162.73657502061295, "leftElbowAngle": 69.87949236175211, "rightKneeAngle": 171.78675025475863, "rightElbowAngle": 57.21187381423874, ...
3397	Lateral Raise	("leftHipAngle": 159.67022526787966, "leftKneeAngle": 178.52378879950268, "rightHipAngle": 164.04640305988906, "leftElbowAngle": 159.82215853370798, "rightKneeAngle": 172.41575650265395, "rightElbowAngle": 162.048922325711...
3398	Lateral Raise	("leftHipAngle": 154.14117983897316, "leftKneeAngle": 178.050528076836, "rightHipAngle": 165.4267435922201, "leftElbowAngle": 145.46404861291, "rightKneeAngle": 169.86468416270228, "rightElbowAngle": 156.51174993489724, "1...
3399	Lateral Raise	("leftHipAngle": 157.32150810992184, "leftKneeAngle": 176.15668235124798, "rightHipAngle": 164.8696147695884, "leftElbowAngle": 149.2397966530753, "rightKneeAngle": 175.60928108792558, "rightElbowAngle": 158.3121650410785, ...
3400	Lateral Raise	("leftHipAngle": 155.6191955601092, "leftKneeAngle": 174.87630598993508, "rightHipAngle": 164.53012188555235, "leftElbowAngle": 126.00600972648792, "rightKneeAngle": 170.7524076625925, "rightElbowAngle": 152.99361655388026...

Figure 78: SQL Query and Stored Exercise Feature Sample Dataset in MySQL Database

Workout Logging System

The exercise recognition system was added to the development process towards the end, as many users wanted to be able to track their workouts for extended periods of time. Other features had to be added to make the application more functional and useful in the day-to-day use of the gym and keep within the general thrust of the project to keep it fitness focused. The fact that workout logging was one of the most frequently-used features in typical fitness applications as well as a feature that would be useful in the context of the exercise recognition system that was already in place for this project led to the selection of this feature.

The primary intention of the workout logging system was that the users would be able to record and recall workout data without having to have to memorize the workout results from previous workouts manually. The features that were added enabled users to choose an exercise and record workout data, such as sets, reps, and weights used during workout sessions. After the workout was saved, users could access the workout data using the application logs section, where they would be able to view their workout history at any point.

An extra Personal Records (PR) feature was added, in addition to basic workout logging, to offer features not typically present in simpler workout monitoring apps. A Personal Records system enabled users to store and keep track of their best performance for each exercise in a different section, providing users with a much easier way to remember the maximum weight they lifted or the important improvements they have made that may not happen frequently in

normal training sessions. This feature made the application much more useful for more advanced gym users who would want to see how they had gone over time.

The MySQL database stored all workout logs, personal records and workout information related to the users via the back-end API system. Every workout record was associated to the user account that created it, thus permitting personalised workout history and record keeping between sessions. Further added functionality was included to enable users to edit or delete workout entries if the wrong data entered by accident. This gave more flexibility and minimized the chance for invalid workout data to be permanently stored.

In addition, the focus was on ensuring that the logging process is as simple and intuitive as possible, allowing users to complete their workouts smoothly and without too many hassles during classes. During implementation, it was important to keep the logging process as simple and user-friendly as possible, ensuring that users can complete their workouts without unnecessary complications during class time.

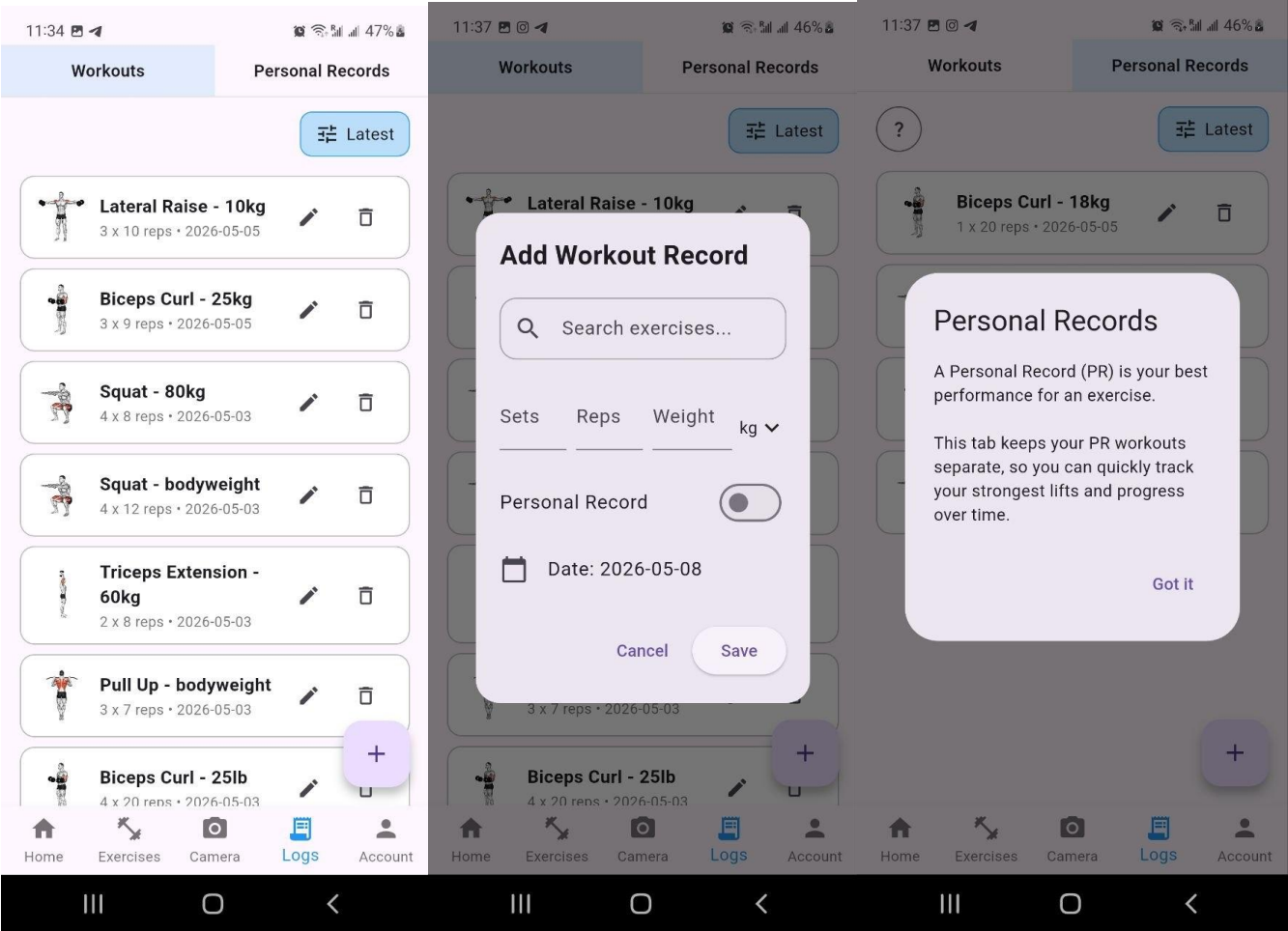
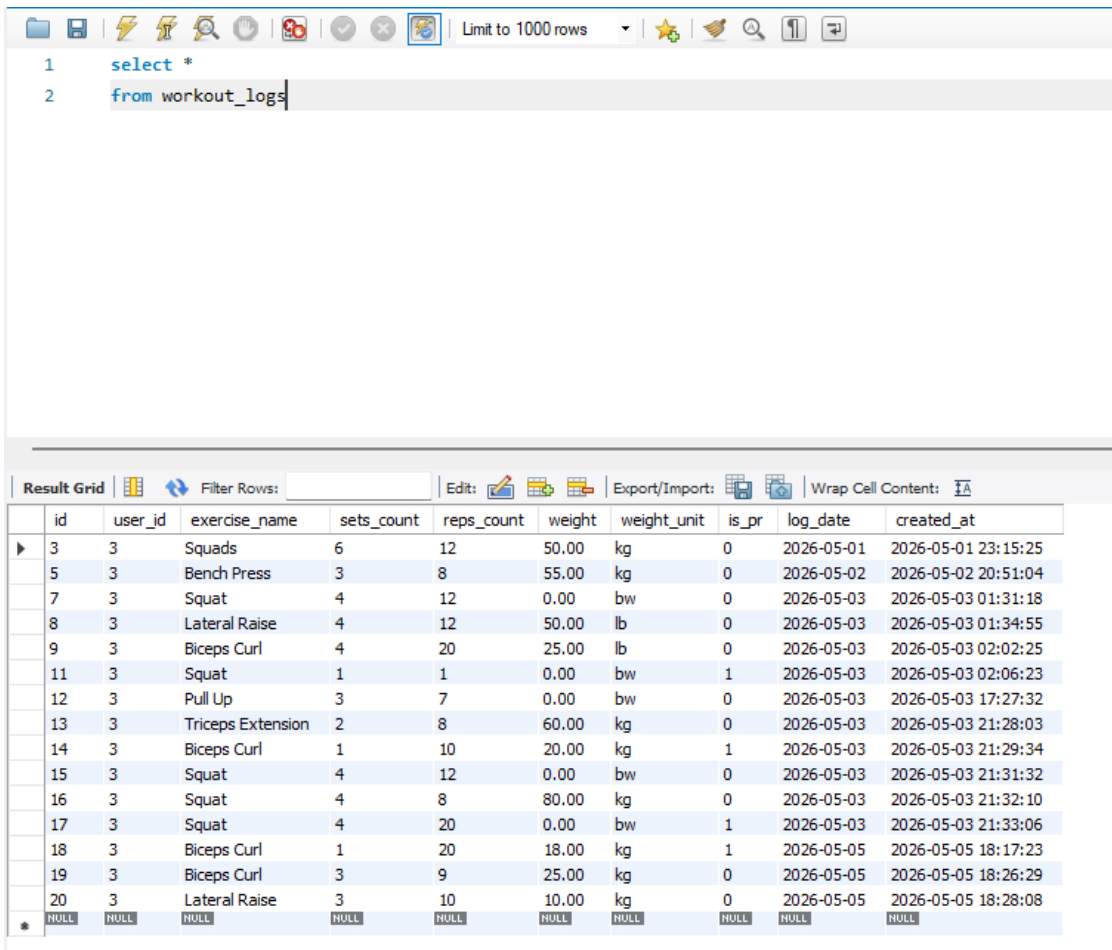


Figure 79: Workout Logs Interface

Figure 80: Add Workout Record Window

Figure 81: Personal Records Information Screen



The screenshot shows a MySQL database interface. At the top, there is a toolbar with various icons and a text input area containing the SQL query: `select * from workout_logs`. Below the query, a 'Result Grid' displays the data. The grid has a toolbar with options like 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The data is presented in a table with the following columns: `id`, `user_id`, `exercise_name`, `sets_count`, `reps_count`, `weight`, `weight_unit`, `is_pr`, `log_date`, and `created_at`. The table contains 20 rows of data, with the last row showing NULL values for all columns.

	id	user_id	exercise_name	sets_count	reps_count	weight	weight_unit	is_pr	log_date	created_at
▶	3	3	Squads	6	12	50.00	kg	0	2026-05-01	2026-05-01 23:15:25
	5	3	Bench Press	3	8	55.00	kg	0	2026-05-02	2026-05-02 20:51:04
	7	3	Squat	4	12	0.00	bw	0	2026-05-03	2026-05-03 01:31:18
	8	3	Lateral Raise	4	12	50.00	lb	0	2026-05-03	2026-05-03 01:34:55
	9	3	Biceps Curl	4	20	25.00	lb	0	2026-05-03	2026-05-03 02:02:25
	11	3	Squat	1	1	0.00	bw	1	2026-05-03	2026-05-03 02:06:23
	12	3	Pull Up	3	7	0.00	bw	0	2026-05-03	2026-05-03 17:27:32
	13	3	Triceps Extension	2	8	60.00	kg	0	2026-05-03	2026-05-03 21:28:03
	14	3	Biceps Curl	1	10	20.00	kg	1	2026-05-03	2026-05-03 21:29:34
	15	3	Squat	4	12	0.00	bw	0	2026-05-03	2026-05-03 21:31:32
	16	3	Squat	4	8	80.00	kg	0	2026-05-03	2026-05-03 21:32:10
	17	3	Squat	4	20	0.00	bw	1	2026-05-03	2026-05-03 21:33:06
	18	3	Biceps Curl	1	20	18.00	kg	1	2026-05-05	2026-05-05 18:17:23
	19	3	Biceps Curl	3	9	25.00	kg	0	2026-05-05	2026-05-05 18:26:29
	20	3	Lateral Raise	3	10	10.00	kg	0	2026-05-05	2026-05-05 18:28:08
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 82: Workout Logs Stored in the MySQL Database

Authentication and Security

Since the PoseFit AI system was intended as a multi-user application, where different users were able to access and view workout information, account management, and store data for themselves, authentication was considered a critical part of the system. While most apps and websites have an account creation and login system nowadays, they come in all levels of security depending on the importance of the data stored and the level of complexity of the platform. The application was not collecting any highly sensitive personal data, so a standard, yet reliable, authentication strategy was used, to ensure usability, simplicity, and security.

The authentication system enabled the users to establish their individual accounts and use email address and password to log into the application. When creating an account, password validation rules were added to enhance account security and prevent the use of weak passwords. The users had to write at least eight letters, including at least one capital letter, one number and one special symbol. These validation rules made the password string more secure against easy guessing or even basic attacks by increasing its password complexity.

Other security enhancements were made by not storing users' passwords in plain text in the database. Rather, the passwords were displayed as hash values that were stored in the MySQL database. This hashing also made it much harder to compromise account security since the actual passwords would not be visible or readable in a compromise of the database.

The authentication system was also enhanced with a “Forgot Password” option for better usability and support of account recovery. This feature enabled users to easily reset forgotten passwords and recover their accounts without the need for manual effort. Other validations were also carried out when logging into the site and when registering for an account, to prevent invalid logon and registration attempts and limit duplicate email registrations.

As the implementation unfolded, the issues regarding authentication and security were closely linked to the issues of legal, ethical, and professional responsibility, which were discussed earlier in the project. The application was designed as a prototype for academic purposes, but some basic data protection and account security concepts were taken into account during development.

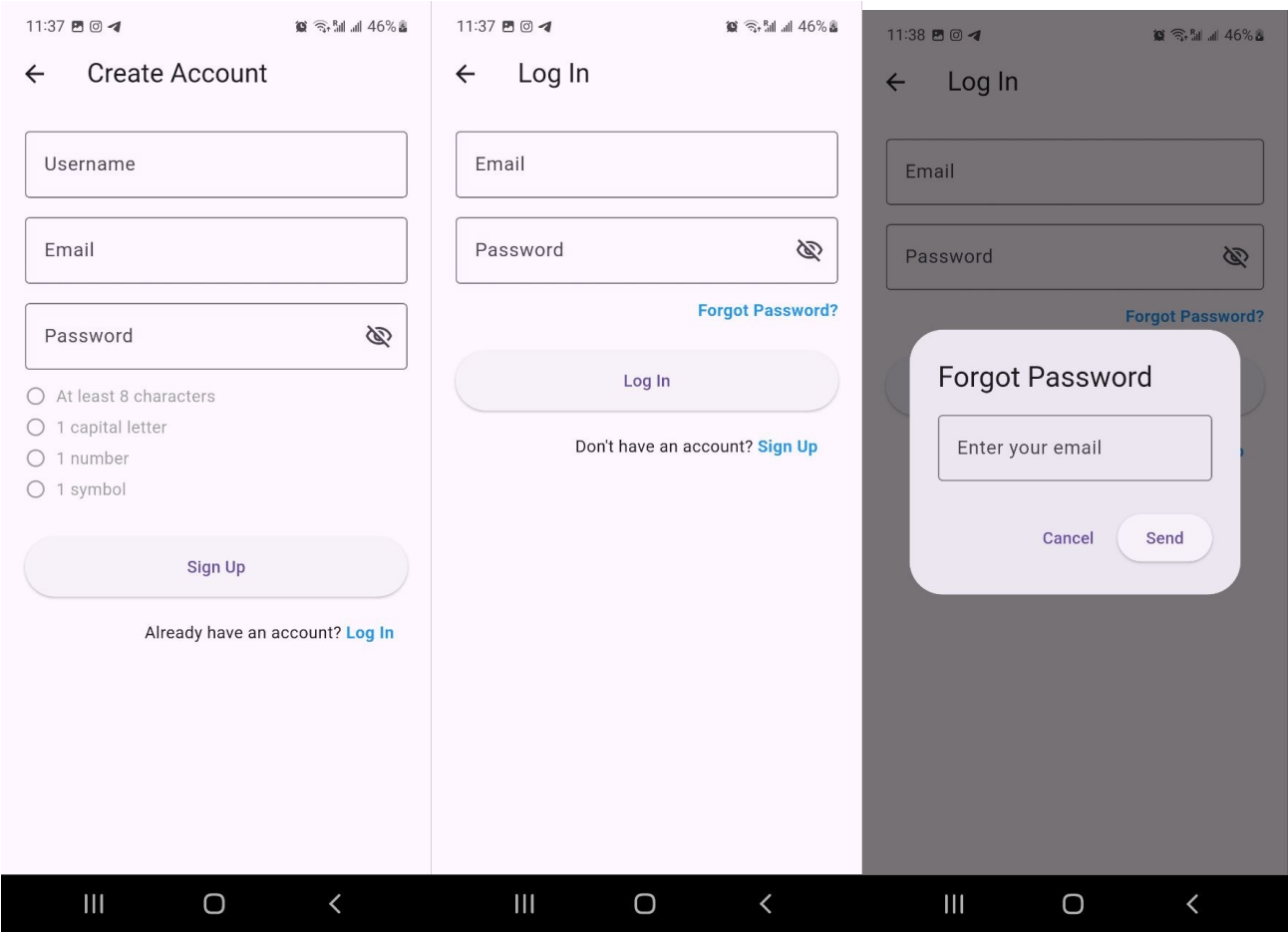
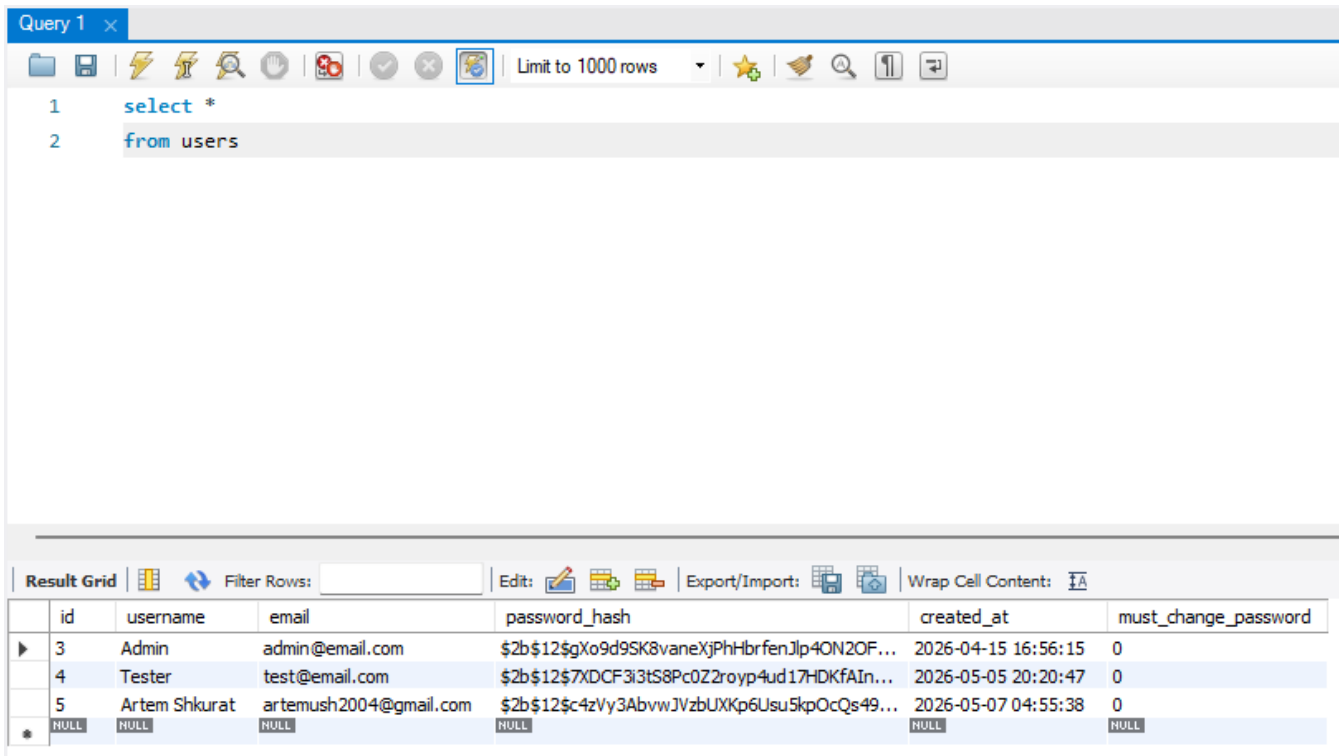


Figure 83: Create Account Interface

Figure 84: User Login Interface

Figure 85: Forgot Password Window



The screenshot shows a database query tool interface. At the top, there's a tab labeled 'Query 1'. Below it is a toolbar with various icons and a 'Limit to 1000 rows' dropdown. The query editor shows two lines of SQL: 'select *' and 'from users'. Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Edit' button, an 'Export/Import' button, and a 'Wrap Cell Content' checkbox. The result grid displays a table with the following data:

	id	username	email	password_hash	created_at	must_change_password
▶	3	Admin	admin@email.com	\$2b\$12\$gXo9d9SK8vaneXjPhHbrfenJlp4ON2OF...	2026-04-15 16:56:15	0
	4	Tester	test@email.com	\$2b\$12\$7XDCF3i3tS8Pc0Z2royp4ud17HDKfAIIn...	2026-05-05 20:20:47	0
	5	Artem Shkurat	artemush2004@gmail.com	\$2b\$12\$c4zVy3AbvwJVzbUXKp6Usu5kpOcQs49...	2026-05-07 04:55:38	0
•	NULL	NULL	NULL	NULL	NULL	NULL

Figure 86: User Account Data Stored in the MySQL Database

Challenges Encountered During Development

Throughout the development processes, they encountered challenges. They faced challenges during development process.

Several technical and organisational issues were faced during the implementation at various stages of the PoseFit AI application. The challenges had a positive effect on the progress of the development, entailed repeated debugging and testing and led to a deeper insight into the software engineering practices, system integration and project management in a real-world development environment.

The first difficulty came when we were setting up the project. The project's scope was difficult to define in the early stages of development, and the technology was difficult to identify, causing the project to start off at a slightly slower pace than planned. Before implementing this project with confidence, more time was needed for research, technology evaluation, and to be aware of the overall expectations of the project.

The front end development is one of the major challenges in terms of user interface and visual design. Multiple revisions and continuous refinements were needed to develop the mobile interface to make it clean, user friendly and uniform in appearance. Given its critical importance for the app, usability was considered and a considerable amount of effort was

invested in optimizing the page layouts, navigation structure and interaction between the user and the app, even though there was little previous experience with mobile UI design.

There were a number of technical issues with server communications and stability that arose when developing the backend. The servers were prone to crashing and encounters with connection problems were common during testing, especially when trying to establish a connection between the flutter application, backend API, and the MySQL database. Some debugging and configuration changes were necessary to keep the communications stable in the local network environment utilized for mobile testing.

The database implementation did not change significantly during the development process and while using the database, there were some challenges but they were indirect problems when integrating the database with other parts of the system, such as during communication with backend or data synchronisation.

There were also several implementation hurdles with the camera integration. Camera permission and consent management were one of the key problems. More features were required to guarantee that users can easily grant, deny and revoke camera permissions, without risk of the application getting unstable. A bit more debugging was needed when permissions changed while the app was running.

The machine learning challenges presented by the pose estimation and exercise recognition system were mainly prediction accuracy and especially due to the fact that the dataset was very limited. As a result, some predictions for exercises were wrong because of similarities between exercise movements, training data being too low, and differences in camera positioning and exercise execution. Further data collection, updating of the confidence thresholds and multiple model tests were needed to improve the reliability of the recognition.

The other usability issues that arose while developing the workout logging system were also noted. Implementation of "Add Workout" interface resulted in some problems, especially in the workout data entry behaviour and the handling of various units of weight measurement. Further enhancements were needed to better enable user interaction and to provide the ability to store workout information consistently and accurately.

Lastly, the addition of authentication and security measures added to the complexity of implementation, especially in terms of password recovery and required password change. The "Forgot Password" system, combined with the enforced password reset, involved further logic and state management of user accounts to ensure that users can regain access to their accounts securely without compromising the authentication system's security.

In general, the challenges played a significant role in the development process and have added to the problem-solving experience, technical understanding, and software engineering skills acquired during the project.

7. Solution testing/ evaluation

Functional Testing

During the development of the PoseFit AI application, functional testing was carried out to ensure that the features implemented were working correctly and were yielding the expected results. The core system functionality was tested, such as authentication, database interaction, integration of the camera, exercise recognition, logging exercise, and user interaction. Many tests were conducted on each feature, ranging from various scenarios to verify their correctness and robustness, to ensure communication between front-end, back-end and database components.

Testing Table

Table 6: Functional Testing Results of the PoseFit AI Application

Feature	Expected Result	Actual Result	Status
User registration	User account created successfully	Account stored in database correctly	Pass
User login	User logs into application	Successful authentication achieved	Pass
Forgot password	User resets password successfully	Password reset completed correctly	Pass
Camera permission	Permission request displayed	Permission handling worked correctly	Pass
Camera streaming	Real-time camera feed activated	Camera stream operated correctly	Pass
Pose estimation	Body landmarks, detected correctly	MediaPipe detected landmarks successfully	Pass
Exercise recognition	Exercise predicted correctly	Recognition accuracy varied depending on movement and angle	Partial
Workout logging	Workout saved to database	Workout information stored successfully	Pass
Personal records	RP information saved or removed	Personal record tracking operational successfully	Pass

Edit / Delete workout	Working information updated or removal	Database updated successfully	Pass
-----------------------	--	-------------------------------	------

In general, the most of the features implemented worked as expected in testing. The authentication, database communication, workout logging and camera integration worked very well for the majority of the testcases. The recognition accuracy of the exercise, however, depended on the camera position, the quality of the movement, light conditions and the limitations of the datasets acquired. These constraints affected the confidence levels of the predictions and sometimes led to wrong classification of exercises in real time testing.

Usability Testing

The usability testing was carried out to assess the user-friendliness and ease of use of the PoseFit AI app. User experience of the user interface, navigation, workout logging, authentication and clarity during workout recognition were the areas of testing. Usability and simplicity were key considerations during the development as the application was to be used mainly on mobile devices when training in the gym.

There was repeated self-testing and feedback from a number of external users throughout the development process, as informal usability testing was conducted. Most users managed to move from one page of the application to another without the need of extra explanation by using the bottom navigation system. The features like workout logging, account management and exercise information pages were easy, and easy to understand.

The camera interface and exercise recognition process were also received well because of the ease of interaction and the real-time pose estimation. However, during testing, certain usability issues were identified, including camera positioning considerations, consistency of exercise recognition and understanding how to increase prediction accuracy when low confidence. For these scenarios, further interface changes and instructions were made to make them more user friendly.

Users also noted that the workout logging function and the Personal Records feature were helpful features during the testing phase, as they helped them to avoid manually entering previous workout performance. This feature made it easier to use than only recognizing exercises on the application.

Exercise Recognition Evaluation

The ability to recognize the exercises with the real-time pose estimation and machine learning classification was evaluated through the exercise recognition evaluation. Given that exercise recognition was the main function of the application, the assessment of prediction consistency, responsiveness, confidence behaviour and the effect of various environmental conditions under real use conditions were given special attention.

The exercise recognition system was able to recognize the trained exercises well in the controlled test environment. MediaPipe pose estimation has been consistently able to track the body landmarks in real time, while TensorFlow has turned body angle measurements and movement patterns to create exercise predictions. The better the exercises were carried out, the more successful the recognition performance was. The better the exercises were carried out, the better the recognition performance was.

Several restrictions that impact the accuracy of prediction were uncovered during the evaluation process, however. The system made false predictions in certain cases because of the resemblance of motion patterns, the lack of consistency in exercise execution, the fast speed of motion, or part of the body being visible in the frame within the camera. The camera angle, the distance from the camera, and the lighting conditions of the environment also affected the recognition accuracy.

Some difficulties were also found because of the small training set. Due to manual recording of the exercise samples, only a limited number of exercises and variations of movements were available for training the model. This meant that the system was not able to fully generalise to different users, exercise styles and recording conditions.

Confidence thresholds that were used during development have helped to increase the reliability of the recognition, by lowering the number of low-confidence predictions shown to the user. If the level of confidence was not high enough, it asked the user to move his or her body or body parts again to increase the accuracy of the recognition. This has resulted in an overall enhanced usability and in minor misclassifications during tests.

Results Table

Table 7: Exercise Recognition Accuracy and Observation Results

Exercise	Recognition Results	Observation
Squat	Good / Situationally	Recognition accuracy was generally reliable when hand positioning and overall body posture was performed correctly
Lateral Raise	Moderate	Arm positioning occasionally reduced prediction consistency
Bicep Curl	Moderate	Rapid movement speed sometimes lowered prediction confidence

Performance and Reliability

Throughout development, performance and reliability testing were performed to assess the stability, responsiveness and behavior of the PoseFit AI application when used in real-time. The primary focus of the testing was on camera processing performance, stability of the communication behind the back of the camera, database interaction, responsiveness of the user interface, and reliability of the exercise recognition function in various usage scenarios.

Generally, the application exhibited good reliability with normal use during testing. In most of the tests, the navigation between the pages worked, the workout logging worked as expected, and the communication between the frontend application, backend server, and database system was successfully maintained. Additionally, the real-time camera processing and MediaPipe pose estimation also showed stable behaviour during exercise recognition, where the continuous detection of body landmarks was performed without significant delays in the normal exercise.

The TensorFlow prediction processing was overall responsive enough to enable near real-time exercise recognition while using the camera. The user's recognition was best when the entire body was within the camera frame and the exercises were conducted with body contours and lighting conditions clearly visible. But the low light, high speed of movement, or improper camera set-up sometimes resulted in inconsistent recognition and low prediction levels for the environment.

During the construction process, several reliability and stability problems were encountered and repeatedly debugged and tested. There are many important issues, one of the most important issues was a front end runtime crash due to state management and interface-related issues. This image shows an example runtime assertion failure that occurred in the front end tests, needing further debugging and interface state handling enhancements.



Figure 87: Flutter Runtime Assertion Failure During Testing

Further communication issues were also discovered at the time of testing between the frontend application and services. In certain cases, there were incomplete loading states where workout data was not presented accurately in the application interface, especially when communicating with the backend or loading data asynchronously.

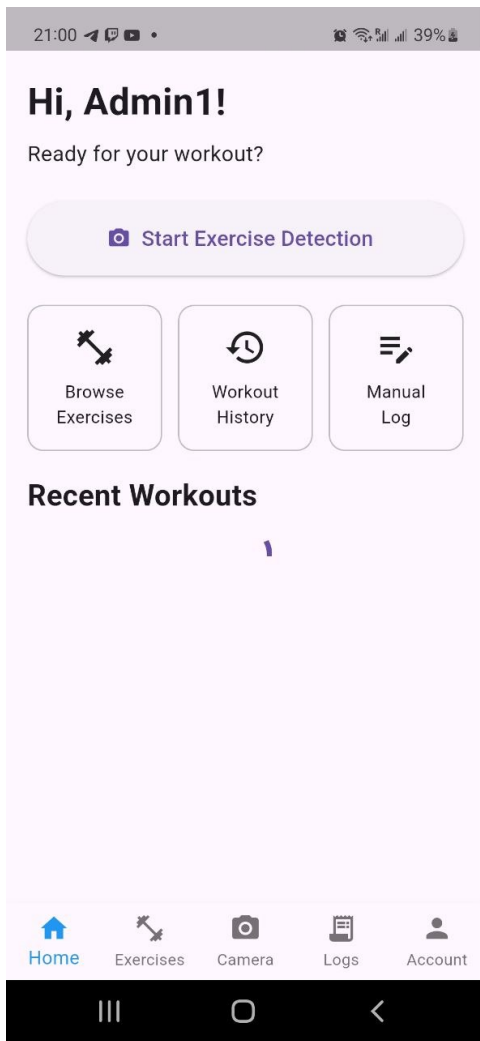


Figure 88: Workout Retrieval Loading Issue

There were also a few usability and layout testing issues of user interface responsiveness. Occasionally problems arose when dealing with various mobile screen sizes and keyboard interactions, causing some errors in the layout overflow while performing some of the operations of the interface.



Figure 89: Keyboard Overflow Error in the Help & Support Interface

Another instability was encountered from time to time when communicating back and forth, possibly related to the difference in the configuration of the local network and the different IP addresses configured for the physical mobile devices used during testing. Resolution changes, permission state changes, and other camera related issues were also seen, leading to some application instability in some cases until further validation and permission handling were added.

These constraints, however, were not an impediment to design for reliability because multiple iterations of debugging, testing and refining were successful in enhancing the overall reliability of the design over the course of development. Other validation rules, confidence limits, permission management and interface enhancements decreased crashes and increased the stability of the applications in the real-time environment.

Evaluation Against Objectives

At the end of the project, this section assesses the implementation of the PoseFit AI application, which aims to achieve the following objectives: The evaluation concentrates on identifying if the intended functionality, technical requirements and overall project objectives have been successfully met during the development process.

The main goal of the project was to create an application that can be operated from a mobile device, and that is capable of identifying gym exercises from real-time camera input and machine learning technologies. The use of Flutter, MediaPipe pose estimation, TensorFlow exercise recognition, backend API services, and a MySQL database system enabled this goal to be met. The application could locate body positions, process the movement data and predict exercises in real time based on the movement data and the camera on the smartphone.

One of the objectives was to create a mobile application that was able to store and manage workout information. The goal was met by introducing features such as workout logging, tracking user records, implementing user authentication, and connecting the database. Users could register, log workouts, monitor exercise performance and manage their own exercise information within the app.

The project also sought to explore the potential of machine learning and pose estimation for exercise recognition. This goal was accomplished by seamlessly embedding MediaPipe and TensorFlow into the camera processing pipeline. All the implementation of body landmark detection, angle calculation, feature extraction and machine learning classification was successful in the final system.

Furthermore, the project was designed to test the usability, reliability, and user interaction in a real-time mobile fitness scenario. The usability and responsiveness of the application were generally confirmed during normal use, through testing and evaluation. Navigation, workout logging and authentication functioned mostly properly during the majority of the tests. During the evaluation, there were some restrictions that affect recognition consistency of exercises and reliability of the camera in various conditions and movements.

While most of the initial goals were accomplished in a satisfactory way, there were a couple of restraints in the ultimate implementation. The exercise recognition system was trained with only a small number of exercises and variations of the exercises included in the data set. This

lowered the overall generalisation ability of the machine learning model, and sometimes impacted the prediction accuracy of the test set. In further versions of the application, more development time and bigger data sets would likely result in greater reliability in recognition.

The overall project has been successful in meeting its primary objectives, which include creating a functional AI powered fitness application that integrates real-time pose estimation with a machine learning exercise recognition system, a workout tracking system and a user account management system all within one mobile application.

Evaluation Table

Table 8: Evaluation of Project Objectives and Final Outcomes

Project Objective	Achieved	Evidence
Develop a mobile fitness application	Yes	Flutter mobile app successfully implemented
Implement exercise recognition	Partial	TensorFlow and MediaPipe successfully integrated, although recognition accuracy varied
Integrate real-time camera processing	Yes	Real-time camera stream and pose estimation implemented
Create workout logging system	Yes	Workout logs and Personal Records functionality implemented
Implement authentication system	Yes	Login, registration, and password recovery features implemented
Store workout and user data	Yes	MySQL database and backend API integrated successfully
Evaluate usability and reliability	Partial	Testing identified both successful functionality and system limitations

Limitations

The PoseFit AI application was able to deploy most of the desired features, but there were still some limitations in the final system. The main restrictions were on exercise recognition accuracy, the quality of the datasets, environmental reliability, and overall scalability of the application.

The size and diversity of the machine learning training data set was one of the critical challenges encountered. Samples were taken manually during the development process, limiting the number of exercise samples and exercises that were available for training. The machine learning model had some difficulty generalising well across varied movements, body shapes, and environments as only a relatively small number of exercise variations and camera angles were recorded.

Camera position and lighting conditions also played a significant role in the accuracy of the recognition of the exercises. For reliable operation the user's entire body should be visible within the frame of the camera, and the exercises carried out with a clear and stable posture. Prediction confidence levels were occasionally lower and exercise classifications were more likely to be wrong due to rapid movement, poor lighting, wrong camera position and only part of the body being visible.

One of the other constraints was the small amount of exercises supported by the recognition system. Because of the time constraints of the project and the difficulty of gathering and analyzing training data, only a small number of exercises could be included in the final implementation. It is likely that adding more exercise data and more types of exercises would greatly enhance the applicability of the application.

However, there were some technical challenges with the project too with device performance, camera actions and back-end communication. During the testing, the stability of the applications was sometimes compromised due to changes in camera resolution, permissions, and local network configuration. The system was not tested under large-scale production conditions, as tests were mostly carried out in a local development environment and physical mobile devices through a shared network.

Moreover, the authentication and security integration were comparatively simple as compared to commercial fitness applications. While password hashing, validation rules, and password

recovery were successfully added, other advanced security aspects like the need for multi-factor authentication, cloud account management, encrypted API communication, and deployment scales were not part of the project.

Lastly, usability testing was done mostly informally, instead of conducting extensive user studies. Feedback and repeated testing during development also helped to refine the navigation and interface usability, but more reliable data about user experience, accessibility and usability of the application in the long term could be obtained through additional testing with larger test groups.

Despite these constraints, the project successfully proved that it is possible to integrate real-time pose estimation, machine learning, mobile app development, and workout tracking in one AI fitness app.

8. Discussion

The PoseFit AI project successfully proved that it was possible to integrate the development of a mobile application, real-time pose estimation, machine learning, and workout tracking in a single platform focused on fitness. This final implementation combined Flutter, MediaPipe, TensorFlow, backend API services, and a MySQL database to enable exercise recognition, workout logging, authentication, and customised interaction with the user in a real-time mobile environment.

The project has been developed from a concept into a successful prototype that is able to recognize physical exercises from the analysis of body movements and camera-based pose estimation throughout development. Despite finding a number of technical and usability issues during testing and evaluation, the project was able to meet most of its original aims and objectives and gained the team valuable experience in software engineering, debugging, machine learning integration and mobile application development.

Perhaps the most important achievement of the project was the proof that lightweight mobile-based AI systems can still be effective in delivering valuable real-time exercise recognition functionality without the need for specialised hardware. The experiment with the integration of MediaPipe and TensorFlow demonstrated that computer vision and machine learning technologies can be effectively used in mobile applications for fitness, even at the prototype stage.

Discussion of Results

The end-to-end deployment of the PoseFit AI application proved that it is technically capable and practically beneficial to exercise recognition in real time with a fitness context and mobile devices and machine learning technologies. The successful integration of Flutter, MediaPipe, TensorFlow, backend API services and MySQL enabled the application to seamlessly integrate exercise recognition, exercise tracking, authentication and user management into a single mobile platform.

The ability to successfully implement real-time pose estimation and movement analysis with MediaPipe and TensorFlow were one of the best parts of the project. The app was able to identify body landmarks, measure body angle parameters and make exercise predictions from the smartphone camera feed. The workout logging system and the Personal Records system

also increased the usefulness of the application in practice by enabling the user to compare how they performed in the exercises in the past with how they are doing them now.

Overall, the system was reliable and scalable, but there were some limitations that affected the reliability and scalability of the system. The camera positioning, lighting, exercise quality of execution as well as the relatively small training set during development, had a significant effect on exercise recognition accuracy. As limited exercises and movement variations could be trained, sometimes inaccurate predictions were made during real-time use of the model.

The project also showed how essential it is to test, debug, and refine a software system during its development. Several technical issues such as camera permissions, backend communication, crash of front-end and reliability of ML were encountered and multiple tests and problem-solving were needed to make it function properly.

In sum, the project accomplished most of its goals and objectives, while also offering significant hands-on experience in the development of a mobile application, integration with the backend, database management, machine learning workflows and problem-solving in software engineering.

Reflection on Development

The PoseFit AI project offered significant hands-on experience in various aspects of software development, such as mobile app development, backend integration, database design and management, machine learning pipelines, debugging, and real-time camera processing, all throughout the development process. A major takeaway from this project was that testing and refinement are critical steps in the development process, especially when incorporating several technologies into a single system.

It also illustrated the difficulties of real-time pose estimation and exercise recognition in uncontrolled environments. The positioning of the camera, lighting conditions, differences in movement and data quality affected the reliability of the recognition, underscoring the need for large and diverse training sets when developing machine learning.

Besides, the development process brought problem solving and debugging skills to a great extent. Several technical problems with camera permissions, frontend crashes, backend communications, and interface responsiveness had to be repeatedly tested and resolved to get things working. These experiences gave a more in-depth understanding of application of theoretical software engineering concepts in real application development environments.

The project was a tremendous learning experience and a testament to the intricacies of integrating AI, computer vision, mobile app development, and user-centric software design into a real-time system.

9. Conclusion

The aim of the PoseFit AI project was to develop an AI-powered mobile fitness application capable of recognising gym exercises using real-time camera processing and machine learning technologies. The entire project has been smoothly implemented with the use of Flutter, MediaPipe, TensorFlow, back-end API services, and a MySQL database throughout the entire project. The final product would feature integrated exercise recognition, exercise logging, authentication and user account management in one mobile platform.

The integration of the real-time pose estimation and exercise recognition was one of the most notable accomplishments of the project, as it included using input from the smartphone camera. The application utilized the capabilities of MediaPipe body landmark detection and TensorFlow movement analysis to determine exercises by analysing body movements and angles. The system's usefulness and practicality were complemented by additional features like workout logging, tracking of Personal Records, and user authentication.

Several limitations were identified in the final implementation, including the limited accuracy of the exercise recognition on the computer vision system, limited size of the exercise dataset, and reliability of the environment, but the project did prove that it was possible to combine computer vision, machine learning tools, and the mobile application development into a fitness environment. The project also emphasized the need to test, debug, refine iteratively and consider usability in software development.

The PoseFit AI project has largely met its initial goals and objectives, and has delivered a valuable hands-on experience with software engineering, backend integration, database management, mobile development, machine learning workflows, and real-time system implementation. The project offers valuable lessons for future growth and is a testament to the capabilities of AI-driven fitness apps in today's mobile world.

Summary

The PoseFit AI project was a machine learning and real-time camera processing powered mobile fitness app that identifies gym exercises. Combined exercise recognition, workout logging, authentication, and user management in one mobile application, the application has been developed by Flutter, MediaPipe, TensorFlow, a Python-based backend API and a MySQL database.

The real-time pose estimation and exercise recognition function has been added throughout the development process through a body landmark detection and movement analysis. Other features like workout logging, Personal Records tracking, camera permission handling, database integration were also built to enhance the usability and usefulness of the app.

The application's recognition accuracy and reliability were successfully demonstrated during test and evaluation under controlled conditions, but some limitations were identified in the recognition accuracy and reliability during the development of the application. In conclusion, the project was a great success and achieved the desired objectives, with the participants gaining valuable experience in the field of computer vision, machine learning, and mobile application development in a fitness setting.

Recommendations for further work

The following are some enhancements that could be included in future versions of the PoseFit AI app to enhance its features: These are some of the enhancements that can be added in subsequent versions of the PoseFit AI application to make it even more useful, easier to use, and better at recognizing exercises: However, the most notable future enhancements would be to make it available to both Android and iOS platforms. Flutter has already supported cross-platform development, but there were mainly Android devices in its implementation and testing process.

A further key improvement would be to automatically collect training data for the machine learning model. In the development, the exercise samples were collected manually, using exercise movements in front of the camera, which is time consuming and physically exhausting. In future, the system may be able to automatically gather exercise data from video recordings, leading to the collection of more and higher-quality exercise data for training the system. A preliminary prototype of a video data collection method was investigated but not fully adopted to the final solution.

The current exercise recognition system is limited with respect to the number of exercises it can recognize because of the number of exercises included in the datasets and time taken to build the system. Future enhancements would include scaling the recognition system to recognize a much wider variety of gym movements and motion variations. More exercises supported will make the application far more useful and scalable in practical applications.

In addition, future development could focus on improving the overall visual design and user interface of the application. Usability and functionality were emphasized in the development process, but more interface enhancements, animation, visual improvements and such might make it look and feel more polished and professional to the user.

Reflection

Reflecting on this project I understand the extent to which my own technical knowledge, as well as the application, has evolved during the process. Initially, I had a simple concept to merge fitness and artificial intelligence, but I was not aware of the extent of complexity that would be involved in the implementation. During the course of the development, I got introduced to a wide range of technology and learned how to integrate the frontend with the backend, databases, machine learning, and real-time camera processing into a working application.

The most significant thing I learned in this project was that in software engineering independent learning is crucial. In many cases, I needed to consult technologies, read documentation, watch tutorials, try technologies and troubleshoot solutions myself before I could proceed with my development. I had a lot of time to learn how MediaPipe, TensorFlow, and Flutter work together with MySQL, particularly as I had not previously built a project that involved pose estimation on the fly and integration with machine learning.

In addition, I found out that software development doesn't always proceed according to the blueprint. Some of the simple tasks were actually quite challenging when it was implemented. Had multiple issues with backend communication, camera permissions, app crashes, machine learning prediction, and app UI behaviour. In some instances, I've been sitting for several hours or even days trying to figure out a particular problem then finally get a solution that worked. These were frustrating at times, but they helped me considerably in my debugging, patience and problem-solving skills

One thing I learned in developing the project was to make sure the project stays realistic and manageable. From the start, I was always wanting to add on more features and for the project to grow even more. But over the years I realized that if I could get something done and keep it running, then it was a worthwhile effort, as opposed to trying to make a bunch of unfinished features. This enabled me to concentrate more on getting the fundamental functionality of the app right than on adding features continually.

I feel this project is one of my most useful learning experiences while at university. I've also enhanced my technical knowledge of mobile dev, back-end systems, databases, and machine learning, and my confidence to solve complex problems independently, from concept to full implementation of a large software project.

10. References

[1]

Hevy, "Hevy." Accessed: Jan. 10, 2026. [Online]. Available: <https://www.hevyapp.com/>

[2]

M. & Strength, "Muscle & Strength Exercise Directory." Accessed: Jan. 10, 2026. [Online]. Available: <https://www.muscleandstrength.com/exercises>

[3]

G. AI, "MediaPipe (Pose Landmarker)." Accessed: Jan. 08, 2026. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

[4]

Strong, "Strong." Accessed: Jan. 10, 2026. [Online]. Available: <https://www.strong.app/>

[5]

G. AI, "MediaPipe (GitHub Repository)." Accessed: Jan. 08, 2026. [Online]. Available: <https://github.com/google-ai-edge/mediapipe?tab=readme-ov-file>

[6]

M. & Strength, "Total Package Workout." Accessed: Nov. 16, 2025. [Online]. Available: <https://www.muscleandstrength.com/sites/default/files/workouts/totalpackage.pdf>

[7]

R. Blog, "OpenPose (Official Website)." Accessed: Jan. 08, 2026. [Online]. Available: <https://blog.roboflow.com/what-is-openpose/>

[8]

F. AG, "Freeletics." Accessed: Jan. 08, 2026. [Online]. Available: <https://www.freeletics.com/en/>

[9]

C. P. C. Lab, "OpenPose (Documentation)." Accessed: Nov. 15, 2025. [Online]. Available: <https://cmu-perceptual-computing-lab.github.io/openpose/web/html/doc/index.html>

[10]

StrengthLog, "Exercise Directory." Accessed: Nov. 16, 2025. [Online]. Available: <https://www.strengthlog.com/exercise-directory/>

[11]

G. AI, "MediaPipe (Solutions Guide)." Accessed: Nov. 15, 2025. [Online]. Available: <https://ai.google.dev/edge/mediapipe/solutions/guide>

[12]

T. Fitness, "Tempo." Accessed: Jan. 08, 2026. [Online]. Available: <https://tempo.fit/>

11. Appendices

[Project proposal poster](#)

[PoseFit AI GitHub account](#)



Figure 90: PoseFit AI Application Logo

PoseFit Ai

Figure 91: PoseFit AI Application Name Branding

PM

ProjectManager

Project Name:

Project Manager

Date

Risk Matrix		Severity				
		Insignificant	Minor	Moderate	Major	Severe
Likelihood	Almost Certain	Medium	High	Very High	Very High	Very High
	Likely	Medium	High	High	Very High	Very High
	Possible	Low	Medium	High	High	Very High
	Unlikely	Low	Low	Medium	Medium	High
	Rare	Low	Low	Low	Low	Medium

Figure 92: Project Risk Assessment Matrix

Welcome to Projects

Built to be flexible and adaptable, Projects gives you a live canvas to filter, sort, and group issues and pull requests in a table, board, or roadmap. Tailor them to your needs with custom fields, saved views, workflows, and insights.

Learn more

Link a project

New project

Open 3

Closed 0

Sort

Report

Documentation and analysis of the PoseFit AI project, including requirements, system design, implementation, testing, and evaluation.

#10 updated 19 minutes ago

Artefact

Development of the PoseFit AI mobile application, including UI improvements, exercise guidance, camera-based recognition, workout logging, account management, and final usability refinements.

#9 updated yesterday

Mid-point review

A project focused on developing an AI-powered mobile system that recognises gym exercises using real-time pose detection. The mid-point review covers literature research, technical rationale, competitor analysis, ethical considerations, and early proof-of-concept development.

#6 updated 4 days ago

Figure 93: GitHub Project Management Board